

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Вычислительной техники
(полное название кафедры)

Утверждаю

Зав. кафедрой к.т.н., Якименко А.А.

(подпись, инициалы, фамилия)

«__» _____ 2024 г.

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ
по направлению высшего образования

09.04.01 Информатика и вычислительная техника
(код и наименование направления подготовки магистра)

Автоматики и вычислительной техники
(факультет)

Горбанев Владимир Викторович
(фамилия, имя, отчество студента – автора работы)

Разработка программной системы обезличивания персональных
(полное название темы магистерской диссертации)
данных в реляционных базах данных

Руководитель
от НГТУ

Альсова Ольга Константиновна
(фамилия, имя, отчество)

к.т.н., доцент
(ученая степень, ученое звание)

(подпись, дата)

Автор выпускной
квалификационной работы

Горбанев Владимир Викторович
(фамилия, имя, отчество)

АВТФ, АММ-22
(факультет, группа)

(подпись, дата)

Новосибирск 2024

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Вычислительной техники
(полное название кафедры)

УТВЕРЖДАЮ

Зав. кафедрой к.т.н., Якименко А.А.
(фамилия, имя, отчество)

(подпись, дата)

**ЗАДАНИЕ
НА МАГИСТЕРСКУЮ ДИССЕРТАЦИЮ**

студенту Горбаневу Владимиру Викторовичу
(фамилия, имя, отчество)

факультета Автоматики и вычислительной техники
(полное название факультета)

Направление подготовки 09.04.01 Информатика и вычислительная техника
(код и наименование направления подготовки магистра)

Магистерская программа Прикладные информационные системы и технологии
(наименование магистерской программы)

Тема Разработка программной системы обезличивания персональных данных в
реляционных базах данных.
(полное название темы)

Цели работы Разработка и исследование алгоритмов и программного обеспечения
для обезличивания персональных данных в реляционных базах данных.

- Задачи 1. Аналитический обзор существующих методов и программных средств, применяемых для решения задачи обезличивания персональных данных.
2. Разработка алгоритмов обезличивания персональных данных в реляционных базах данных.
3. Проектирование и разработка программной системы, реализующей алгоритмы обезличивания данных.
4. Исследование эффективности разработанных алгоритмов обезличивания персональных данных (оценка скорости, оценка информационных потерь) на примере анализа медицинской БД.

Задание согласовано и принято к исполнению.

**Руководитель
от НГТУ**

Альсова Ольга Константиновна

(фамилия, имя, отчество)

к.т.н., доцент

(ученая степень, ученое звание)

(подпись, дата)

Студент

Горбанев Владимир Викторович

(фамилия, имя, отчество)

АВТФ, АММ-22

(факультет, группа)

(подпись, дата)

Тема утверждена приказом по НГТУ № 1632/2 от «28» марта 2024г.

изменена приказом по НГТУ № 4854/2 от «20» октября 2022 г.

Диссертация сдана в ГЭК № _____, тема сверена с данными приказа

(подпись секретаря государственной экзаменационной комиссии по защите ВКР, дата)

(фамилия, имя, отчество секретаря государственной
экзаменационной комиссии по защите ВКР)

АННОТАЦИЯ

Пояснительная записка к магистерской диссертации Горбанева Владимира Викторовича на тему «Разработка программной системы обезличивания персональных данных в реляционных базах данных» изложена на 104 странице, содержит 77 иллюстраций и 4 таблицы. Диссертация состоит из введения, 4 глав с промежуточными выводами, заключения, списка литературы из 13 наименований.

КОНФИДЕНЦИАЛЬНОСТЬ, ЗАЩИТА ДАННЫХ, ОБЕЗЛИЧИВАНИЕ, ПЕРСОНАЛЬНЫЕ ДАННЫЕ, БАЗА ДАННЫХ, ДЕОБЕЗЛИЧИВАНИЕ, ИНФОРМАЦИОННАЯ ПОТЕРЯ.

Цель работы: Разработка и исследование алгоритмов и программного обеспечения для обезличивания персональных данных в реляционных базах данных.

Данная работа посвящена разработке программного обеспечения по обезличивания персональных данных и исследованию разработанных методов по обезличиванию.

В рамках магистерской диссертации подробно анализируются основные технологии, используемые для обезличивания данных, а также те конкретные технологии, которые выбраны для реализации в рамках данного проекта. Кроме того, магистерская диссертация включает в себя описание разработанных методов обезличивания, а также проведение анализа скорости их работы и оценку информационных потерь, связанных с применением этих методов.

Также разработано программное обеспечение для решения задачи обезличивания данных в реляционных базах данных.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ОБЗОР ТЕХНОЛОГИЙ И ПРОГРАММНЫХ СРЕД ДЛЯ ОБЕЗЛИЧИВАНИЯ ДАННЫХ	12
1.1 Основные понятия.....	12
1.2 Технологии обезличивания данных	13
1.2.1 Статическое обезличивание.....	13
1.2.2 Динамическое обезличивание	15
1.3 Реализация технологий обезличивания данных в современных программных средах.....	17
1.4 Исследование возможностей SQL Server Management Studio (SSMS) по обезличиванию данных на примере БД по инфекционной заболеваемости ...	20
1.5 Выводы по главе 1.....	24
2 ОБЗОР МЕТОДОВ ОБЕЗЛИЧИВАНИЯ ДАННЫХ	25
2.1 Классификации методов обезличивания данных	26
2.1.1 Метод введения идентификаторов.....	26
2.1.2 Метод изменения состава или семантики	27
2.1.3 Метод декомпозиции.....	28
2.1.4 Метод перемешивания	29
2.2 Основные критерии оценки эффективности методов обезличивания	30
2.3 Выводы по главе 2.....	32
3 РЕАЛИЗАЦИЯ МЕТОДОВ ОБЕЗЛИЧИВАНИЯ ДАННЫХ.....	33
3.1 Выбор и обоснование программных средств реализации процедуры обезличивания данных	33
3.2 Описание реализованных методов обезличивания данных	38
3.2.1 DateAging	39
3.2.2 GeneralizationString	40
3.2.3 GeneralizationValue	42
3.2.4 Identifier.....	44
3.2.5 MicroAggregation.....	45
3.2.6 MicroAggregationBySingleAxis	48

3.2.7 Round	50
3.2.8 Shuffle	50
3.2.9 ValueReplacement	52
3.2.10 ValueReplacementByPattern	52
3.2.11 ValueReplacementFromFile	54
3.2.12 ValueVariance.....	55
3.2.13 Decomposition	56
3.3 Оценка скорости обезличивания данных на основе реализованных методов.....	57
3.4 Оценка информационных потерь, связанных с применением метода обезличивания	69
3.5 Выводы по главе 3.....	71
4 РАЗРАБОТКА ПРОГРАММНОЙ СИСТЕМЫ ПО ОБЕЗЛИЧИВАНИЮ ДАННЫХ	73
4.1 Анализ требований	73
4.2 Проектирование программного обеспечения	74
4.2.1 Описание классов.....	74
4.2.2 Архитектура программного обеспечения	78
4.2.3 Алгоритм решения задачи обезличивания данных	81
4.2.4 Интерфейс программной системы	82
4.3 Тестирование программного обеспечения	93
4.4 Выводы по главе 4.....	99
ЗАКЛЮЧЕНИЕ	101
СПИСОК ЛИТЕРАТУРЫ.....	103

ВВЕДЕНИЕ

Актуальность

В современном цифровом мире, где данные стали одним из самых ценных ресурсов, вопрос обезличивания данных становится все более актуальным и важным. Реляционные базы данных играют ключевую роль в хранении и обработке информации в различных сферах деятельности, начиная от бизнеса и заканчивая научными исследованиями и государственными службами. Обезличивание данных в реляционных базах данных представляет собой критически важный процесс в сфере информационной безопасности. С увеличением объемов персональной информации, хранящейся в базах данных, растет и необходимость защиты этой информации от несанкционированного доступа и злоупотреблений. Обезличивание данных представляет собой методологию, при которой идентифицирующая информация удаляется из базы данных или заменяется идентификаторами, не связанными с конкретными личностями. Этот процесс обеспечивает анонимность и защиту конфиденциальности данных, при этом сохраняя их ценность для аналитики и последующей обработки.

Одной из основных задач обезличивания данных является обеспечение анонимности пользователей или субъектов данных, чья информация хранится в базе данных. Это особенно важно в контексте соблюдения нормативных требований, таких как Приказ Роскомнадзора от 05.09.2013 N 996 «Об утверждении требований и методов по обезличиванию персональных данных», требования ФСТЭК по защите персональных данных. Нарушение этих норм может повлечь серьезные последствия для организаций, включая значительные штрафы и утрату доверия со стороны клиентов и партнеров.

Однако, несмотря на широкое использование методов обезличивания данных, существуют определенные ограничения и риски. Во-первых, существует вероятность обратного преобразования, особенно при использовании слабых методов шифрования или недостаточно эффективных алгоритмов обезличивания. Во-вторых, даже обезличенные данные могут быть

подвергнуты анализу и использованы для идентификации личности, особенно при объединении с другими наборами данных.

Технологии обезличивания позволяют решить эту проблему и предоставить данные для опубликования или передачи разработчикам, тестировщикам, третьим лицам для проведения экономических и статистических исследований, клинических испытаний, в учебных целях и т.п.

На данный момент существует множество ПО для обезличивания данных, к примеру, PostgreSQL Anonymizer, Data Masking in MySQL, K2View Data Masking, DATPROF Data Masking Tool и другие. Но все они имеют значительные недостатки, например, платная подписка, ограниченный функционал, набор методов, который не полностью соответствует требованиям Российского законодательства.

Поэтому актуальна разработка собственного программного обеспечения для обезличивания данных, чему и посвящена магистерская диссертация.

Разрабатываемое программное обеспечение не только защитит конфиденциальные данные пользователей, но и будет соответствовать всем требованиям Роскомнадзора по защите данных. Программное обеспечение войдет в состав системы «Интеллектуальное управление, обогащение и обезличивание данных», разрабатываемой в рамках платформы «Доверенная среда обмена информации» (проекты Центра компетенций НГТУ).

Целью работы является разработка и исследование алгоритмов и программного обеспечения для обезличивания персональных данных в реляционных базах данных.

Для достижения поставленной цели необходимо было решить следующие **задачи**:

1. Аналитический обзор существующих методов и программных средств, применяемых для решения задачи обезличивания персональных данных.
2. Разработка программных алгоритмов обезличивания персональных данных в реляционных базах данных.

3. Проектирование и разработка программной системы, реализующей алгоритмы обезличивания данных.

4. Исследование эффективности разработанных алгоритмов обезличивания персональных данных (оценка скорости, оценка информационных потерь) на примере анализа медицинской БД.

Объектом исследования являются методы обезличивания персональных данных.

Предметом исследования является процесс обезличивания персональных данных с целью исключения прямой или косвенной идентификации отдельных субъектов с сохранением при этом ценности и полезности данных для последующего использования.

Методами исследования являются методы обезличивания, методы оценки информационных потерь, методы предварительной обработки данных.

Научная новизна

1. Разработан программный алгоритм обезличивания персональных данных на основе применения разных классов методов обезличивания. Алгоритм разработан с учетом требований российского законодательства и международных рекомендаций в сфере защиты данных.

2. Исследована эффективность разработанных алгоритмов обезличивания персональных данных (оценка скорости, оценка информационных потерь) на примере анализа медицинской БД.

Практическая значимость работы заключается в создании библиотеки функций и модулей программной системы для обезличивания персональных данных, которая войдет в состав системы «Интеллектуальное управление, обогащение и обезличивание данных», разрабатываемой в рамках платформы «Доверенная среда обмена информации» (проекты Центра компетенций НГТУ).

Публикации по теме работы

Результаты работы были представлены на международной научно-практической конференции «Цифровые системы и модели: теория и практика проектирования, разработки и применения» (Казань, 10-11 апреля 2024 г.):

Горбанев В.В. Сравнительный анализ методов обезличивания данных по временному показателю /В.В. Горбанев, О.К. Альсова//Международная научно-практическая конференция «Цифровые системы и модели: теория и практика проектирования, разработки и применения»: Казань, 10-11 апреля 2024 г. – С. 1261–1265. – ISBN 978-5-89873-660-6

Структура и объём диссертации

Структура диссертационной работы включает введение, четыре основные главы, заключение и список использованной литературы. Общий объём работы составляет 104 страницу, включая библиографический список, содержащий 13 источников. В тексте представлены 77 рисунков, а также включены четыре таблицы.

Основное содержание работы

Во введении обосновывается актуальность темы, определяется цель и задачи исследования, описываются основные полученные результаты и практическая значимость работы. Кратко излагается содержание диссертации по главам, а также описывается структура и объём магистерской диссертации.

В первой главе диссертации выполнен анализ существующих технологий обезличивания персональных данных и их реализации в современных программных средах. В ходе исследования выделяются преимущества, недостатки и ограничения использования указанных программных продуктов. Также в работе обосновывается необходимость разработки собственного программного обеспечения для обезличивания данных, учитывая особенности и требования российского законодательства в сфере защиты данных.

Вторая глава диссертационного исследования представляет собой анализ методов обезличивания персональных данных, описываются основные свойства методов обезличивания и критерии оценки их эффективности, приводятся преимущества и недостатки каждого метода.

Третья глава. В третьей главе выполнено обоснование выбора программных средств и технологий для реализации программного обеспечения обезличивания данных.

Описаны реализации методов обезличивания данных с использованием СУБД PostgreSQL и языка программирования Java, которые включены в состав программной системы. Проведены экспериментальные исследования времени и скорости обезличивания данных в зависимости от размера исходной базы данных и выбранного метода. Описаны методы оценки информационных потерь после обезличивания данных.

Четвертая глава диссертации начинается с представления требований к программному обеспечению, включая как функциональные, так и нефункциональные. Затем следует описание архитектуры и модульной структуры программного продукта, а также программной реализации графической оболочки и представление пользовательского интерфейса с его функциональными возможностями.

В заключительной части главы подробно рассматривается функционал каждого модуля программного обеспечения, включая как базовые функции, так и специально разработанные методы обезличивания данных. Этот подход к описанию позволяет полностью ознакомиться с возможностями и преимуществами разработанного программного продукта, обеспечивая понимание его функциональности и потенциала.

В заключении сформулированы основные результаты диссертации, выводы по результатам исследования и перспективные направления дальнейших исследований.

1 ОБЗОР ТЕХНОЛОГИЙ И ПРОГРАММНЫХ СРЕД ДЛЯ ОБЕЗЛИЧИВАНИЯ ДАННЫХ

1.1 Основные понятия

Обезличивание персональных данных — это действия, в результате которых становится невозможным без использования дополнительной информации определить принадлежность персональных данных конкретному субъекту, что обеспечивает защиту от несанкционированного использования, а также возможность их обработки [2].

Обезличенные данные должны обладать свойствами, сохраняющими основные характеристики обезличиваемых персональных данных.

Свойства обезличенных данных:

- полнота (сохранение всей информации, которая имелаась до обезличивания);
- структурированность (сохранение структурных связей между обезличенными данными, имеющимся до обезличивания);
- релевантность (возможность обработки запросов в одинаковой семантической форме);
- семантическая целостность (сохранение семантики персональных данных при их обезличивании);
- применимость (возможность решения задач обработки персональных данных, стоящих перед оператором, осуществляющим обезличивание персональных данных);
- анонимность (невозможность однозначной идентификации субъектов данных, полученных в результате обезличивания, без применения дополнительной информации).

Основная цель обезличивания данных — создание альтернативной версии данных, которую невозможно легко идентифицировать или реконструировать, защищая данные, классифицируемые как конфиденциальные. При этом

обезличенная информация выглядит реалистично и непротиворечиво и может использоваться для решения прикладных задач.

1.2 Технологии обезличивания данных

Существует множество технологий обезличивания данных, но на данный момент выделяют две основные и наиболее распространенные технологии: статическое и динамическое обезличивание.

Статическое обезличивание данных постоянно заменяет конфиденциальные данные, изменяя данные в состоянии покоя.

Динамическое обезличивание данных направлено на замену конфиденциальных данных при передаче, оставляя исходные данные в состоянии покоя нетронутыми и неизменными.

1.2.1 Статическое обезличивание

Самый распространенный способ технической реализации статического обезличивания заключается в использовании специализированных программных решений, размещенных на выделенном сервере. При настройке указывается сетевой адрес базы данных, содержащей конфиденциальную информацию, а также предоставляются учетные данные для доступа к ней. Для определения полей, содержащих конфиденциальные данные, проводится автоматизированный аудит. После этого выбирается метод обезличивания. Система выполняет запросы к базе данных и, используя функциональные возможности СУБД, либо обезличивает саму базу данных, либо создает копию с измененными данными [3].

Статическое обезличивание данных обычно выполняется для золотой копии базы данных, но также может применяться к значениям в других источниках, включая файлы.

Оно в основном используется для предоставления высококачественных, то есть реалистичных, данных для разработки и тестирования приложений без раскрытия конфиденциальной информации. Реализм важен, потому что он позволяет командам разработчиков и тестировщиков более эффективно

выявлять дефекты на ранних этапах цикла разработки, тем самым снижая затраты и повышая общее качество.

Дополнительные области применения включают защиту данных для использования в аналитике и обучении.

Статическое обезличивание также способствует внедрению облачных технологий. Обезличивание данных в локальной среде перед их загрузкой в облако снижает риск для организаций, связанных с раскрытием облачных данных.

В начале производственные данные используются для создания копии базы данных с золотой маской (рисунок 1.1), которая затем реплицируется в различные среды. Кроме того, на этой диаграмме отражено понятие нескольких копий производственных данных, что помогает защитить данные с помощью статического обезличивания.

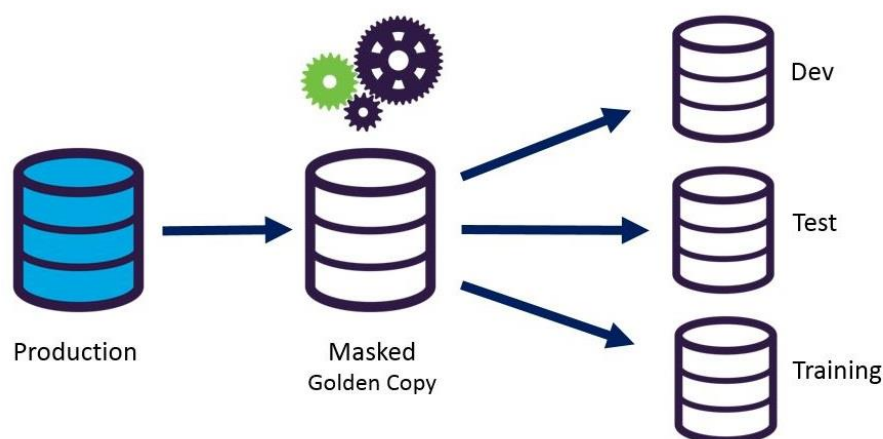


Рисунок 1.1 – Архитектура статистического обезличивания

На рисунке 1.2 представлен пример статического обезличивания конфиденциальных данных. Как видно, изменились все строки в столбцах, но при этом они не потеряли своей реалистичности и их можно использовать в тестировании.

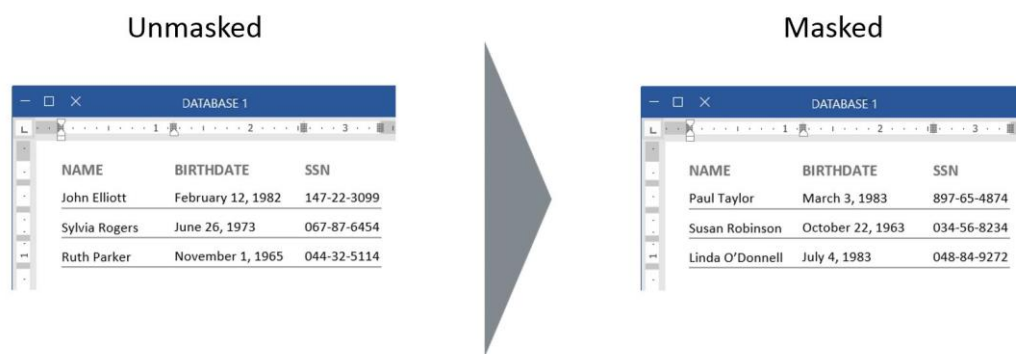


Рисунок 1.2 – Данные до и после статистического обезличивания

Преимущества статистического обезличивания:

- конфиденциальные данные удаляются навсегда. Если злоумышленник скомпрометирует статически обезличенную базу данных, он не получит никаких конфиденциальных данных;
- нет большой нагрузки на систему. Все данные преобразуются заранее, поэтому после того, как обезличенная база данных станет доступной, производительность не пострадает;
- значительно упрощает безопасность копирования данных. Нет необходимости внедрять детализированную защиту, поскольку все конфиденциальные данные заменены.

Недостатки статического обезличивания:

- обезличивание данных длится довольно большой промежуток времени, которых может занять минуты или часы в зависимости от размера данных;
- необратимо изменяются исходные данные, которые не подлежат восстановлению.

1.2.2 Динамическое обезличивание

Динамическое обезличивание данных происходит во время выполнения, динамически и по запросу, поэтому нет необходимости во втором источнике данных для хранения обезличенных данных (рисунок 1.3) [3].

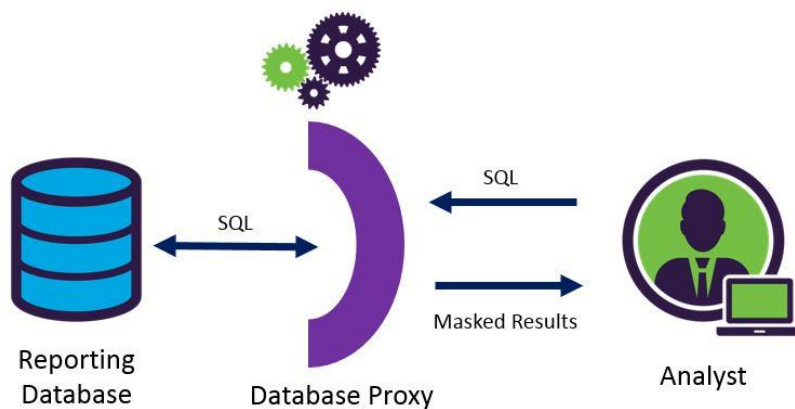


Рисунок 1.3 – Архитектура динамического обезличивания

Динамическое обезличивание данных, также известное как обезличивание данных «на лету», обезличивает конфиденциальные данные в пути, оставляя исходные данные в состоянии покоя неповреждёнными и неизменными. Динамическое обезличивание используется, чтобы скрыть определённые данные от определённого пользователя. Данные в базе данных не обезличиваются физически, а обезличиваются в результате запроса. Не обезличенные данные остаются видимыми в базе данных (рисунок 1.4).

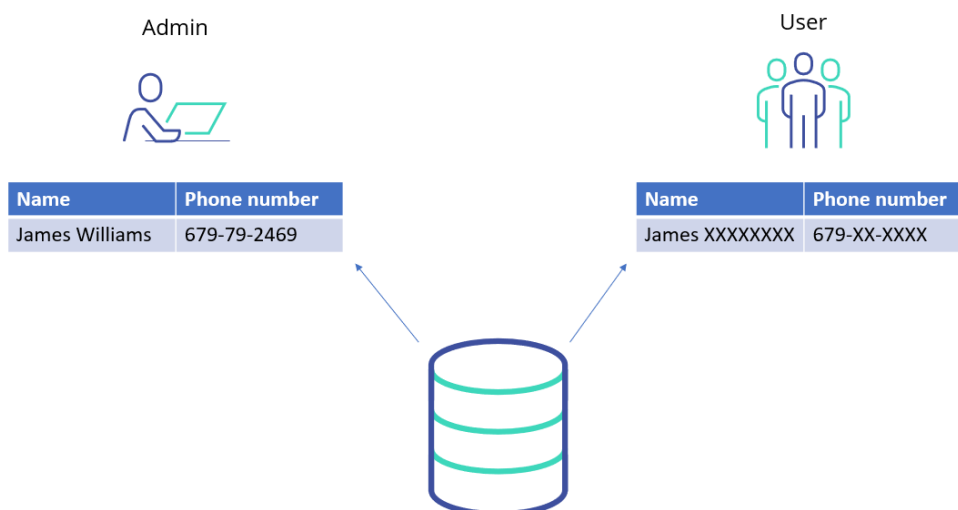


Рисунок 1.4 – Запрос с динамическим обезличиванием

Преимущества динамического обезличивания:

- добавляет дополнительный уровень безопасности и контроля конфиденциальности для защиты данных;
- защищает данные в сценариях только для чтения;
- работает в реальном времени;

- не требует предварительного обезличивания всех данных.

Недостатки динамического обезличивания:

- не подходит для использования в динамической среде для чтения и записи, поскольку обезличенные данные могут быть записаны обратно в базу данных, что приведет к повреждению данных;
- высокая нагрузка на системы, так как обезличивание происходит при каждом запросе;
- прокси-сервер является единственной точкой отказа, и пользователи, подключающиеся напрямую к базе данных, могут обойти его, что может привести к раскрытию исходных данных, хранящихся в базе данных;
- работники могут украсть оригинальную базу данных;
- хранимые процедуры нельзя динамически обезличить. Обезличивание хранимых процедур требует перезаписи результатов запроса, а не самого запроса.

1.3 Реализация технологий обезличивания данных в современных программных средах

Обезличивание делает данные бесполезными для кибератак, сохраняя при этом удобство использования и согласованность, а также снижает риски, связанные с совместным использованием данных с интегрированными сторонними приложениями и облачными миграциями. Поэтому данная технология является очень важной для сохранения конфиденциальности данных, в результате чего было разработано множество сервисов и библиотек для пользователей, корпораций, компаний и т.д.

Методы для обезличивания данных предоставляют различные компании и проекты с открытым кодом. Рассмотрим некоторые из них.

PostgreSQL Anonymizer

В PostgreSQL есть готовое решение по обезличиванию данных — библиотека PostgreSQL Anonymizer. В проекте реализован декларативный подход к обезличиванию. Это означает, что вы можете объявить правила

обезличивания с помощью языка определения данных PostgreSQL (DDL) и указать свою стратегию обезличивания внутри самого определения таблицы. После определения правил обезличивания можно получить доступ к обезличенным данным как статистическим, так и динамическим образом [4].

В данной библиотеке реализованы некоторые методы обезличивания, такие как перемешивание, наложение шума, обобщение и другие.

Одним из минусов PostgreSQL Anonymizer можно отметить то, что данная библиотека разрабатывалась иностранцами, а значит она плохо работает с русскими символами. Тем самым реализованные в ней методы, которые предполагают обезличивание имен, фамилий, компаний и т. д., становятся бесполезными и не применимыми.

Другие методы, более универсальные, также имеют свои недостатки, поскольку существует несколько способов реализации методов обезличивания, которые могут не соответствовать желаниям пользователя, использующего их. Например, метод `shuffle`, предоставляемый данной библиотекой, может перемешать только один столбец в таблице, и в случае, если необходимо сохранить связь между определенными столбцами таблицы, данный метод не является решением задачи.

Существующий метод обобщения в PostgreSQL Anonymizer также может не устраивать пользователя своей реализацией, поскольку если требуется реализация метода обобщения через таблицу справочник, данный метод может оказаться неэффективным. Кроме того, значительным недостатком данного метода является то, что невозможно настроить более гибкие параметры для обезличивания путем обобщения. Например, если необходимо обезличить возраст согласно классификации ВОЗ (Всемирной организации здравоохранения), используя 5 категорий:

- 1 категория – дети до 1 года (среднее равно 0,5);
- 2 категория – дети от 1 года до 3-х лет (среднее равно 2);
- 3 категория – дети от 3 до 18 лет (среднее равно 10,5);
- 4 категория – от 18 до 60 лет (среднее равно 39);

5 категория – старше 60 лет (среднее равно 77,5).

Метод из библиотеки не способен обеспечить такую гибкость, и категории обезличиваемого столбца будут иметь одинаковый числовой промежуток.

Data Masking in MySQL

Обезличивание в СУБД MySQL имеет такие же преимущества и недостатки, как и в PostgreSQL Anonymizer. Но так как проект является решением от компании Oracle, он имеет свои особенности.

Поскольку Oracle - крупная компания, в их продукте гораздо больше методов обезличивания, и они более универсальны. Например, в MySQL существует механизм словарей для обезличивания данных. Вместо того, чтобы данные генерировались случайным образом, есть возможность составить словарь значений. Словари обезличивания данных позволяют выбирать случайное значение из набора предопределенных значений, которые можно "загрузить" в механизм обезличивания. Таким образом, открывается возможность обезличивать не только числа, но и слова и предложения, в том числе и на русском языке [5].

Но хотя в MySQL и больше методов обезличивания и они более универсальны из-за того, что разработкой занимается Oracle, есть и свой минус, а именно, методы платные и имеют ежемесячную подписку.

На рынке существуют и другое программное обеспечение: K2View Data Masking, DATPROF Data Masking Tool, IRI FieldShield (Profile/Mask/Test), Accutiv Data Discovery & Masking, IRI DarkShield (Unstructured Data Masking), IRI CellShield EE (Excel Data Masking), Oracle - Data Masking and Subsetting, IBM InfoSphere Optim Data Privacy, Delphix и т.д.

Данное программное обеспечение позволяет работать не только с различными СУБД, а также в NoSQL, но они все имеют дорогую подписку.

Исходя из представленных выше технологий обезличивания, можно сделать вывод, что наиболее качественные и надежные методы обезличивания требуют платных решений. В бесплатных открытых проектах функционал

может быть недостаточно гибким и подходить только для маленьких проектов, так как для больших баз данных функционала бесплатных технологий обезличивания недостаточно.

1.4 Исследование возможностей SQL Server Management Studio (SSMS) по обезличиванию данных на примере БД по инфекционной заболеваемости

Рассмотрим возможности SQL Server Management Studio (SSMS) версии 18.0 и выше в контексте статистического обезличивания данных [6].

В SSMS присутствуют 5 методов обезличивания (рисунок 1.5).

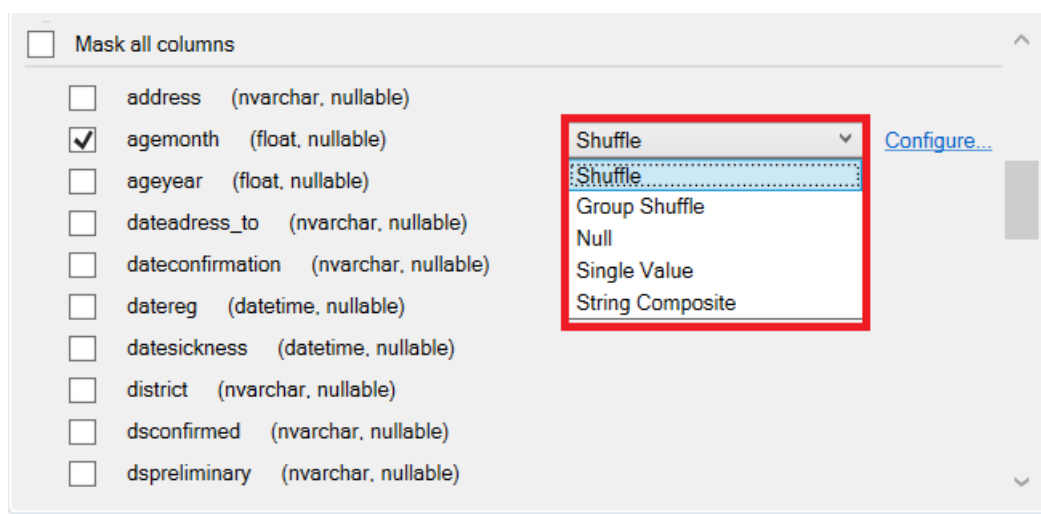


Рисунок 1.5 – Все доступные методы обезличивания в SSMS выделены **красным**

Первые два метода, shuffle (перемешивание) и group shuffle (групповое перемешивание), работают точно так же, как описано в разделе 2.1.4, и не имеют особых недостатков.

Метод NULL обнуляет все записи в выбранном атрибуте и также, как и первые два метода, не содержит недостатков.

Давайте проверим эти методы (рисунок 1.6). На атрибуте district мы проведем обезличивание при помощи перемешивания и заменим значения атрибута job на null с использованием метода NULL.

	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного 45	ОУ № 45.86	2008-05-08 00:00:00.000
2	NULL	0	6	Калининский	Курчатова 11/3	NULL	2008-02-22 00:00:00.000
3	NULL	0	7	Заельцовский	Ельцовская 35	NULL	2008-02-25 00:00:00.000
4	NULL	2	33	Дзержинский	Рылеева 92	ДОУ № 17 "Улыбка", младшая садиковая группа	2008-02-25 00:00:00.000
5	NULL	2	27	Ленинский	1п. Троллейный	NULL	2008-02-25 00:00:00.000
6	NULL	0	8	Заельцовский	Дачная 23/1	NULL	2008-02-25 00:00:00.000
7	NULL	1	13	Кировский	4перБрон	не орган	2008-02-25 00:00:00.000
8	NULL	3	43	Калининский	Земнухова 3	ДОУ №14 "Аленка"	2008-02-25 00:00:00.000
9	NULL	0	11	Кировский	Сиб-Гвар 80	неорган	2008-02-25 00:00:00.000
10	NULL	0	5	Октябрьский	Высоцкого 5	не организованный	2008-02-26 00:00:00.000
11	NULL	82	984	Дзержинский	2 ая Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	73	876	Ленинский	Блюхера 31	NULL	2008-02-25 00:00:00.000
13	NULL	1	15	Калининский	Новоуральская 13	NULL	2008-02-26 00:00:00.000
14	NULL	2	24	Ленинский	Новосибирская 20	NULL	2008-02-22 00:00:00.000
15	NULL	20	240	Заельцовский	Рельсовая 3	НГАВТ.4 кур	2008-02-25 00:00:00.000



	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного 45	NULL	2008-05-08 00:00:00.000
2	NULL	0	6	Октябрьский	Курчатова 11/3	NULL	2008-02-22 00:00:00.000
3	NULL	0	7	Кировский	Ельцовская 35	NULL	2008-02-25 00:00:00.000
4	NULL	2	33	Ленинский	Рылеева 92	NULL	2008-02-25 00:00:00.000
5	NULL	2	27	Калининский	1п. Троллейный	NULL	2008-02-25 00:00:00.000
6	NULL	0	8	Заельцовский	Дачная 23/1	NULL	2008-02-25 00:00:00.000
7	NULL	1	13	Калининский	4перБрон	NULL	2008-02-25 00:00:00.000
8	NULL	3	43	Первомайский	Земнухова 3	NULL	2008-02-25 00:00:00.000
9	NULL	0	11	Советский	Сиб-Гвар 80	NULL	2008-02-25 00:00:00.000
10	NULL	0	5	Ленинский	Высоцкого 5	NULL	2008-02-26 00:00:00.000
11	NULL	82	984	Кировский	2 ая Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	73	876	Заельцовский	Блюхера 31	NULL	2008-02-25 00:00:00.000
13	NULL	1	15	Дзержинский	Новоуральская 13	NULL	2008-02-26 00:00:00.000
14	NULL	2	24	Центральный	Новосибирская 20	NULL	2008-02-22 00:00:00.000
15	NULL	20	240	Советский	Рельсовая 3	NULL	2008-02-25 00:00:00.000

Рисунок 1.6 – Результат применения методов shuffle и null

Как видно из рисунка 1.6, эти методы продемонстрировали оптимальную эффективность и, как уже упоминалось ранее, не имеют недостатков.

Следующие методы в SSMS, которые будут рассмотрены, называются Single value и string composite. Они имеют некоторые недостатки. Давайте рассмотрим эти методы подробнее.

Суть метода single value (рисунок 1.7) состоит в замене числового атрибута другим значением. Недостаток данного метода заключается в том, что невозможно указать диапазон значений, а также изменить текущее значение для сохранения данных для дальнейшей обработки.

	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного 45	ОУ № 45,86	2008-05-08 00:00:00.000
2	NULL	0	6	Калининский	Курчатова 11/3	NULL	2008-02-22 00:00:00.000
3	NULL	0	7	Заельцовский	Ельцовская 35	NULL	2008-02-25 00:00:00.000
4	NULL	2	33	Дзержинский	Рылеева 92	ДОУ № 17 "Улыбка", младшая садовая группа	2008-02-25 00:00:00.000
5	NULL	2	27	Ленинский	1п. Троллейный	NULL	2008-02-25 00:00:00.000
6	NULL	0	8	Заельцовский	Дачная 23/1	NULL	2008-02-25 00:00:00.000
7	NULL	1	13	Кировский	4перБрон	не орган	2008-02-25 00:00:00.000
8	NULL	3	43	Калининский	Земнухова 3	ДОУ №14 "Аленка"	2008-02-25 00:00:00.000
9	NULL	0	11	Кировский	Сиб-Гвар 80	неорган	2008-02-25 00:00:00.000
10	NULL	0	5	Октябрьский	Высоцкого 5	не организованный	2008-02-26 00:00:00.000
11	NULL	82	984	Дзержинский	2 ая Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	73	876	Ленинский	Блюхера 31	NULL	2008-02-25 00:00:00.000
13	NULL	1	15	Калининский	Новоуральская 13	NULL	2008-02-26 00:00:00.000
14	NULL	2	24	Ленинский	Новосибирская 20	NULL	2008-02-22 00:00:00.000
15	NULL	20	240	Заельцовский	Рельсовая 3	НГАБТ.4 кур	2008-02-25 00:00:00.000



	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного 45	ОУ № 45,86	2008-05-08 00:00:00.000
2	NULL	15	6	Калининский	Курчатова 11/3	NULL	2008-02-22 00:00:00.000
3	NULL	15	7	Заельцовский	Ельцовская 35	NULL	2008-02-25 00:00:00.000
4	NULL	15	33	Дзержинский	Рылеева 92	ДОУ № 17 "Улыбка", младшая садовая группа	2008-02-25 00:00:00.000
5	NULL	15	27	Ленинский	1п. Троллейный	NULL	2008-02-25 00:00:00.000
6	NULL	15	8	Заельцовский	Дачная 23/1	NULL	2008-02-25 00:00:00.000
7	NULL	15	13	Кировский	4перБрон	не орган	2008-02-25 00:00:00.000
8	NULL	15	43	Калининский	Земнухова 3	ДОУ №14 "Аленка"	2008-02-25 00:00:00.000
9	NULL	15	11	Кировский	Сиб-Гвар 80	неорган	2008-02-25 00:00:00.000
10	NULL	15	5	Октябрьский	Высоцкого 5	не организованный	2008-02-26 00:00:00.000
11	NULL	15	984	Дзержинский	2 ая Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	15	876	Ленинский	Блюхера 31	NULL	2008-02-25 00:00:00.000
13	NULL	15	15	Калининский	Новоуральская 13	NULL	2008-02-26 00:00:00.000
14	NULL	15	24	Ленинский	Новосибирская 20	NULL	2008-02-22 00:00:00.000
15	NULL	15	240	Заельцовский	Рельсовая 3	НГАБТ.4 кур	2008-02-25 00:00:00.000

Рисунок 1.7 – Результат применения метода single value на атрибуте age_year

Следующий метод называется string composite, и заключается в том, чтобы заменить строку по определенному заранее заданному паттерну (рисунок 1.8).

Pattern: ☒ Advanced

Sample:

Source Regex:

Backup Pattern:

Sample Input:

Partial Replacement (Advanced):
 Define a Regular Expression for your input text that contains a capture group. The input text captured will be replaced by text generated from your pattern. Multiple capture groups/patterns are currently not supported.

Examples (click to try):
[Social Security Number \(keep last four digits\)](#)
[Phone Number \(keep area code\)](#)
[Email Address \(keep domain\)](#)

Рисунок 1.8 – Создание паттерна для обезличивания

Как видно на рисунке 1.9, значения, которые соответствовали паттерну, числовые обозначения в адресе были заменены звездой. Так же присутствуют пустые значения и значения, в которых звезда была поставлена не в том месте.

Это и является одним из минусов паттерна, так как он не универсален и малейшее отклонение от него приводит к ошибкам, поэтому значения атрибута

должны заполняться строго определенным образом. В случае если есть множество разных видов заполнения, это приведет к разрастанию логики паттерна и неудобству его чтения и изменения в случае необходимости.



	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного 45	ОВ № 45,86	2008-05-08 00:00:00.000
2	NULL	0	6	Калининский	Курчатова 11/3	NULL	2008-02-22 00:00:00.000
3	NULL	0	7	Завельцовский	Ельцовская 35	NULL	2008-02-25 00:00:00.000
4	NULL	2	33	Дзержинский	Рылеева 92	ДОУ № 17 "Улыбка", младшая садовая группа	2008-02-25 00:00:00.000
5	NULL	2	27	Ленинский	1п. Тропильный	NULL	2008-02-25 00:00:00.000
6	NULL	0	8	Завельцовский	Дачная 2311	NULL	2008-02-25 00:00:00.000
7	NULL	1	13	Кировский	4лтерБрон	не орган	2008-02-25 00:00:00.000
8	NULL	3	43	Калининский	Земнухова 3	ДОУ №14 "Аленка"	2008-02-25 00:00:00.000
9	NULL	0	11	Кировский	Сиб-Гаар 80	неорган	2008-02-25 00:00:00.000
10	NULL	0	5	Октябрьский	Высоцкого 5	не организованный	2008-02-26 00:00:00.000
11	NULL	82	984	Дзержинский	2 ая Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	73	876	Ленинский	Блюхера 31	NULL	2008-02-25 00:00:00.000
13	NULL	1	15	Калининский	Новоуральская 13	NULL	2008-02-26 00:00:00.000
14	NULL	2	24	Ленинский	Новосибирская 20	NULL	2008-02-22 00:00:00.000
15	NULL	20	240	Завельцовский	Рельсовая 3	НГАБТ,4 кур	2008-02-25 00:00:00.000

	id_sex	age_year	age_month	district	address	job	date_sickness
1	NULL	15	180	Ленинский	Плахотного *	ОВ № 45,86	2008-05-08 00:00:00.000
2	NULL	0	6	Калининский	Курчатова *	NULL	2008-02-22 00:00:00.000
3	NULL	0	7	Завельцовский	Ельцовская *	NULL	2008-02-25 00:00:00.000
4	NULL	2	33	Дзержинский	Рылеева *	ДОУ № 17 "Улыбка", младшая садовая группа	2008-02-25 00:00:00.000
5	NULL	2	27	Ленинский	1п. *	NULL	2008-02-25 00:00:00.000
6	NULL	0	8	Завельцовский	Дачная *	NULL	2008-02-25 00:00:00.000
7	NULL	1	13	Кировский		не орган	2008-02-25 00:00:00.000
8	NULL	3	43	Калининский		ДОУ №14 "Аленка"	2008-02-25 00:00:00.000
9	NULL	0	11	Кировский	Сиб-Гаар *	неорган	2008-02-25 00:00:00.000
10	NULL	0	5	Октябрьский		не организованный	2008-02-26 00:00:00.000
11	NULL	82	984	Дзержинский	2 * Национальная	NULL	2008-02-26 00:00:00.000
12	NULL	73	876	Ленинский	Блюхера *	NULL	2008-02-25 00:00:00.000
13	NULL	1	15	Калининский	Новоуральская *	NULL	2008-02-26 00:00:00.000
14	NULL	2	24	Ленинский	Новосибирская *	NULL	2008-02-22 00:00:00.000
15	NULL	20	240	Завельцовский		НГАБТ,4 кур	2008-02-25 00:00:00.000

Рисунок 1.9 – Результат обезличивания атрибута address по паттерну

На данный момент — это все методы обезличивания, которые предоставляет нам SSMS 18.0. И, несмотря на то, что даже текущие методы содержат недостатки, так как они являются довольно универсальными, существует еще множество методов обезличивания, которые не реализованы в SSMS.

В SSMS полностью отсутствует метод введения идентификатора, который является довольно удобным в случае необходимости деобезличивания персональных данных. Также отсутствуют методы обезличивания даты и декомпозиции.

В методе изменения состава и семантики реализованы только наиболее универсальные методы, которые можно использовать практически во всех базах данных. Однако отсутствуют более узко специализированные методы, которые позволили бы не только обезличить базу данных, но и сохранить её в таком формате для проведения исследовательской работы.

Примерами таких методов являются замена имени и фамилии на реальные, а не на набор случайных букв, как в методе string composite. Также

отсутствует возможность задавать изменения числовых значений и дат относительно текущих. Не предусмотрена возможность задавать диапазон случайных чисел для замены.

1.5 Выводы по главе 1

В рамках первой главы рассмотрено понятие обезличивания персональных данных, а также статические и динамические технологии по обезличиванию персональных данных.

Приведены примеры технологий по обезличиванию данных, используемых в популярных базах данных, таких как *Data Masking in MySQL*, *PostgreSQL Anonymizer* в MySQL и PostgreSQL соответственно. Были сделаны выводы об их возможностях.

На примере SQL Server Management Studio и его пакета по обезличиванию персональных данных была продемонстрирована работа методов по обезличиванию данных. Были сделаны выводы о возможностях данного пакета, его недостатках и достоинствах.

Из анализа программного обеспечения, представленного в данной главе, можно сделать вывод о необходимости разработки собственного программного продукта. Большинство существующих программных систем обладают рядом недостатков, включая наличие платной подписки и отсутствие реализации полного списка методов обезличивания данных, рекомендуемых к применению Роскомнадзором. Бесплатное программное обеспечение, в свою очередь, обладает лишь минимальными возможностями по обезличиванию данных. Таким образом, необходима разработка собственной программной системы, обладающей универсальностью, полным функционалом с учетом требований российского законодательства в сфере защиты данных.

2 ОБЗОР МЕТОДОВ ОБЕЗЛИЧИВАНИЯ ДАННЫХ

Основным требованием к обезличиванию персональных данных государственными и муниципальными органами является обеспечение не только защиты от несанкционированного использования, но и возможности их обработки, для чего обезличенные данные должны обладать свойствами, сохраняющими основные характеристики этих данных [1].

К свойствам методов обезличивания персональных данных, относятся:

- обратимость (возможность обратного преобразования обезличенных данных);
- вариативность (возможность внесения изменений в параметры метода);
- изменяемость (возможность внесения изменений в массив обезличенных данных);
- стойкость (стойкость метода к атакам на идентификацию субъекта персональных данных);
- возможность косвенного деобезличивания (возможность проведения деобезличивания с использованием информации других операторов);
- совместимость (возможность интеграции персональных данных);
- параметрический объем (объем дополнительной информации, необходимой для реализации метода обезличивания и деобезличивания);
- возможность оценки качества данных (возможность проведения контроля качества обезличенных данных).

В России приказом Роскомнадзора были определены четыре основных метода обезличивания.

- метод введения идентификаторов. Замена части сведений идентификаторами с созданием таблицы соответствия идентификаторов исходным данным;
- метод изменения состава или семантики, изменение состава или семантики персональных данных путем замены результатами статистической обработки, обобщения или удаления части сведений;

- метод декомпозиции, разбиение массива персональных данных на несколько частей с их последующим отдельным хранением;
- метод перемешивания, перестановка отдельных записей, а также групп записей в массиве персональных данных.

2.1 Классификации методов обезличивания данных

2.1.1 Метод введения идентификаторов

Данный метод осуществляется путем замены чувствительных атрибутов персональных данных на идентификаторы, которые ссылаются на справочную таблицу с соответствующими персональными данными (рисунок 2.1) [1].

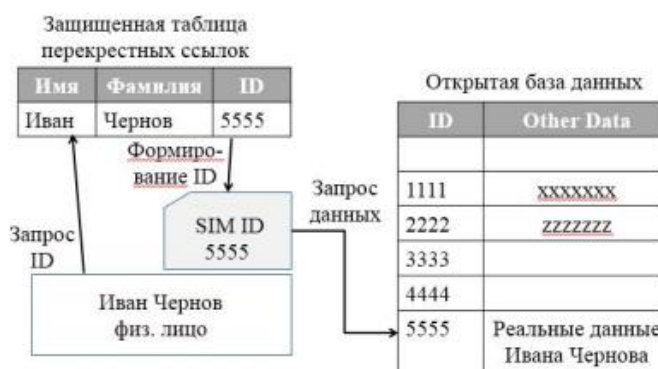


Рисунок 2.1 – Метод введения идентификаторов

Персональные данные, преобразованные методом введения идентификаторов, обладают следующими свойствами:

- полнота (персональная информация о субъекте не удаляется из таблицы, а переносится в другую);
- структурированность (каждый идентификатор однозначно указывает на скрытый набор персональных данных);
- семантическая целостность (данные не изменяются, а переносятся в таблицу соответствия);
- применимость (можно осуществлять обработку обезличенных данных без деобезличивания).

Этот метод оказывается уязвимым к атакам, направленным на справочники идентификаторов, а также к атакам, использующим информацию из этих справочников. При этом его устойчивость не зависит от объема

обезличиваемых данных. Поэтому для обеспечения анонимности персональных данных необходимо соблюдать определенные правила при выборе идентификаторов и их замене на персональные данные.

Метод введения идентификаторов сохраняет в записях связи между атрибутами обезличенных данных и соответствующими атрибутами персональных данных. Его эффективность проявляется особенно при небольшом количестве атрибутов и записей персональных данных, так как объем справочников зависит от этих параметров. Однако вычислительная эффективность метода снижается при частых изменениях в составе данных и значениях атрибутов.

В разработанном программном продукте был реализован метод введения идентификаторов, соответствующий данному классу методов обезличивания данных.

2.1.2 Метод изменения состава или семантики

Суть данного метода состоит в изменении значений атрибутов персональных данных на другие или удаление части сведений, позволяющих идентифицировать субъекта (рисунок 2.2) [1].

Обезличенные данные, получаемые после применения данного метода, обладают следующими свойствами:

- структурированность (связь между атрибутами персональных данных субъекта не нарушается);
- анонимность (удаление или обобщение части данных приводит к неоднозначности при идентификации с использованием обезличенных данных);
- полнота (обезличиваемые данные могут обладать данным свойством, если провести изменения, которые дают гарантию их сохранности, в случае удаления некоторых сведений, обезличенные данные, теряют данное свойство);
- семантическая целостность (при условии, если будут проведены изменения, сохраняющие семантику данных);

- частичная релевантность (в некоторых случаях есть возможность получить семантическое соответствие поискового запроса и полученного ответа на запрос);
- применимость (можно осуществлять обработку, не требующую деобезличивания всего объема данных)

	Customer ID	First name	Last name	Sex	Birth number	Telephone	Email	Document ID
1	519875	Roland	Johnson	M	7406197625	00458 36 853746	johnson.r23@mail.com	FX 731 020



Маскированные данные

	Customer ID	First name	Last name	Sex	Birth number	Telephone	Email	Document ID
1	519875	John	Howard	M	7406198032	99999 87 024464	ok7hmpb.qbv@z6+w.mlp	AG 700 510

Рисунок 2.2 – Метод изменения состава и семантики

Применение метода изменения состава и семантики позволяет сохранить часть связей между атрибутами обезличиваемых персональных данных. Данный метод наиболее эффективен в случае, когда нет необходимости в обратном деобезличивании, так как он необратим и не позволяет восстановить исходные персональные данные из обезличенных.

В разработанном программном обеспечении данному классу методов соответствуют следующие методы: старение данных, обобщение для качественных и количественных признаков или дат, замена значений атрибута, микроагрегирование по выбранным атрибутам, микроагрегирование по выбранным атрибутам с использованием сортировки одного из них, округление, замена значений атрибута в соответствии с заданным образцом (паттерном), замена значений атрибута на случайное значение из файла, а также добавление шума.

2.1.3 Метод декомпозиции

Суть метода разделить атрибуты персональных данных на несколько таблиц и подмножеств, с последующим отдельным хранением записей, соответствующих подмножествам этих атрибутов [1].

Данный метод позволит получить обезличенные данные, со следующими свойствами:

- полнота (персональная информация не удаляется, а переносится в другие хранилища);
- структурированность (связи между записями, которые содержатся в разных хранилищах, сохраняются, что позволяет однозначно сопоставить их);
- семантическая целостность (вид персональных данных о субъекте не изменяется);
- релевантность (так как возможно получить семантическое соответствие поискового запроса и полученного ответа на запрос);
- применимость (можно осуществить обработку данных, располагаемых в одном хранилище, так как оно не зависит от других).

Метод декомпозиции обеспечивает анонимность только при сложных связях между хранилищами и защите данных хранилищ от несанкционированного доступа, так как метод не устойчив к атакам, направленным на деанонимизацию путем анализа данных из различных хранилищ и косвенной деанонимизации

Данный метод позволяет сохранить в каждой записи связи между атрибутами обезличенных и персональных данных. Эффективность метода проявляется при большом объеме атрибутов персональных данных и редких изменениях в составе данных и значениях атрибутов.

В разработанном программном решении был реализован метод декомпозиции, соответствующий данному типу методов обезличивания данных.

2.1.4 Метод перемешивания

Суть данного метода – обезличить значения атрибута или группы атрибутов персональных данных путем перемешивания данных между собой [1].

Обезличенные данные, полученные методом перемешивания, обладают такими свойствами, как:

- полнота (все персональные данные о субъектах сохраняются);
- структурированность (при деобезличивании связи можно полностью восстановить);
- семантическая целостность (вид представления данных никак не изменяется);
- анонимность (данные перемешиваются по каждому отдельному атрибуту записи о субъекте, что не позволяет без доступа к служебной информации определить принадлежность тех или иных данных конкретному субъекту);
- релевантность (возможность получить семантическое соответствие поискового запроса и полученного ответа на запрос);
- применимость (при наличии доступа к служебной информации, есть возможность осуществить обработку).

При использовании метода перемешивания не сохраняются связи между атрибутами обезличенных и персональных данных. Этот метод рекомендуется для применения в случае, если имеется большое количество записей и атрибутов персональных данных. Стойкость данного метода к атакам, направленным на деобезличивание, пропорциональна объему данных.

В созданном программном продукте был реализован метод перемешивания, который соответствует данному классу методов обезличивания данных.

2.2 Основные критерии оценки эффективности методов обезличивания

При оценке методов обезличивания для защиты конфиденциальных данных следует учитывать несколько ключевых критериев, обеспечивающих их эффективность и безопасность. Вот некоторые общие критерии оценки методов обезличивания:

1. Безопасность данных. Основная цель методов обезличивания — защитить конфиденциальные данные. Выбранный метод должен обеспечивать высокий уровень защиты от несанкционированного доступа, гарантируя, что обезличенные данные невозможно будет легко обратить вспять или расшифровать.

2. Влияние на качество данных. Методы обезличивания должны обеспечивать качество и целостность данных. Обезличенные данные по-прежнему должны быть пригодны для использования по назначению, например, для аналитики, тестирования или составления отчетов, без ущерба для их точности и актуальности.

3. Обратимость. В зависимости от конкретного сценария использования, метод обезличивания может оказаться обратимым. Он должен позволять при необходимости восстанавливать исходные данные, обеспечивая при этом безопасность обезличенных данных.

4. Производительность. Эффективность и производительность методов обезличивания имеют решающее значение, особенно при обработке больших объемов данных. Оценка скорости обезличивания и ее влияния на время обработки данных может помочь определить практичность подхода к обезличиванию.

5. Соответствие. Методы обезличивания должны соответствовать соответствующим правилам конфиденциальности данных, таким как приказ Роскомнадзора [1], GDPR[7], HIPAA[8] или PCI DSS[9]. Убедитесь, что выбранный метод соответствует конкретным требованиям и стандартам, применимым к обезличиванию данных.

6. Масштабируемость. Оценка масштабируемости методов обезличивания, чтобы определить их пригодность для обработки данных разного объема и сложности. Метод должен быть в состоянии учитывать рост и изменение требований к данным с течением времени.

7. Устойчивость к атакам. Оценка устойчивости методов обезличивания к потенциальным атакам, таким как статистический вывод, атаки методом

перебора или другие методы повторной идентификации данных. Метод должен эффективно защищать от этих угроз.

8. Простота реализации. Учитывайте простоту внедрения и поддержки метода обезличивания в существующих процессах и системах обработки данных. Метод, который плавно интегрируется с существующими рабочими процессами и инструментами, может оптимизировать усилия по защите данных.

9. Стоимость и ресурсы: Оценка финансовых последствий внедрения и управления методом обезличивания, включая любые необходимые ресурсы или специальные навыки. Баланс между преимуществами метода и его затратами имеет важное значение для принятия решений.

Оценивая методы обезличивания по этим критериям, организации могут выбрать наиболее подходящий подход к защите конфиденциальных данных, обеспечивая при этом соблюдение правил защиты данных и поддерживая операционную эффективность.

2.3 Выводы по главе 2

В рамках второй главы были рассмотрены свойства методов обезличивания персональных данных, такие как: обратимость; вариативность; изменяемость; стойкость; возможность косвенного деобезличивания; совместимость; параметрический объем; возможность оценки качества данных.

Были представлены 4 основные группы методов обезличивания данных, их основная суть, свойства, которыми они должны обладать и эффективность применения данных методов.

Рассмотрены основные критерии оценки методов обезличивания для защиты конфиденциальных данных, которые учитывают несколько ключевых свойств, обеспечивающих их эффективность и безопасность.

3 РЕАЛИЗАЦИЯ МЕТОДОВ ОБЕЗЛИЧИВАНИЯ ДАННЫХ

3.1 Выбор и обоснование программных средств реализации процедуры обезличивания данных

Основной частью разрабатываемого программного обеспечения для обезличивания персональных данных в реляционных базах данных является модуль обезличивания, который войдет в более широкую систему интеллектуального управления, обогащения и обезличивания данных (рисунок 3.1). Информация из разных источников и пулов данных в соответствии со схемой поступает сначала на вход модуля объединения пулов в конечную выборку, где происходит формирование единой исходной БД, а затем уже сформированная БД подается на вход модуля обезличивания данных. Один из важных вопросов, который необходимо решить на этапе проектирования архитектуры платформы, – какую модель и какой тип БД использовать.

Предлагается использовать реляционные базы данных (РБД), которые предназначены для хранения и организации взаимосвязанных данных. РБД основаны на реляционной модели, которая представляет данные в виде таблиц, что облегчает их понимание и обработку. Каждая строка таблицы представляет собой запись с уникальным идентификатором, известным как ключ. Столбцы таблицы содержат атрибуты данных, а каждая запись обычно включает значение для каждого атрибута, обеспечивая простое установление связей между различными элементами данных [10].

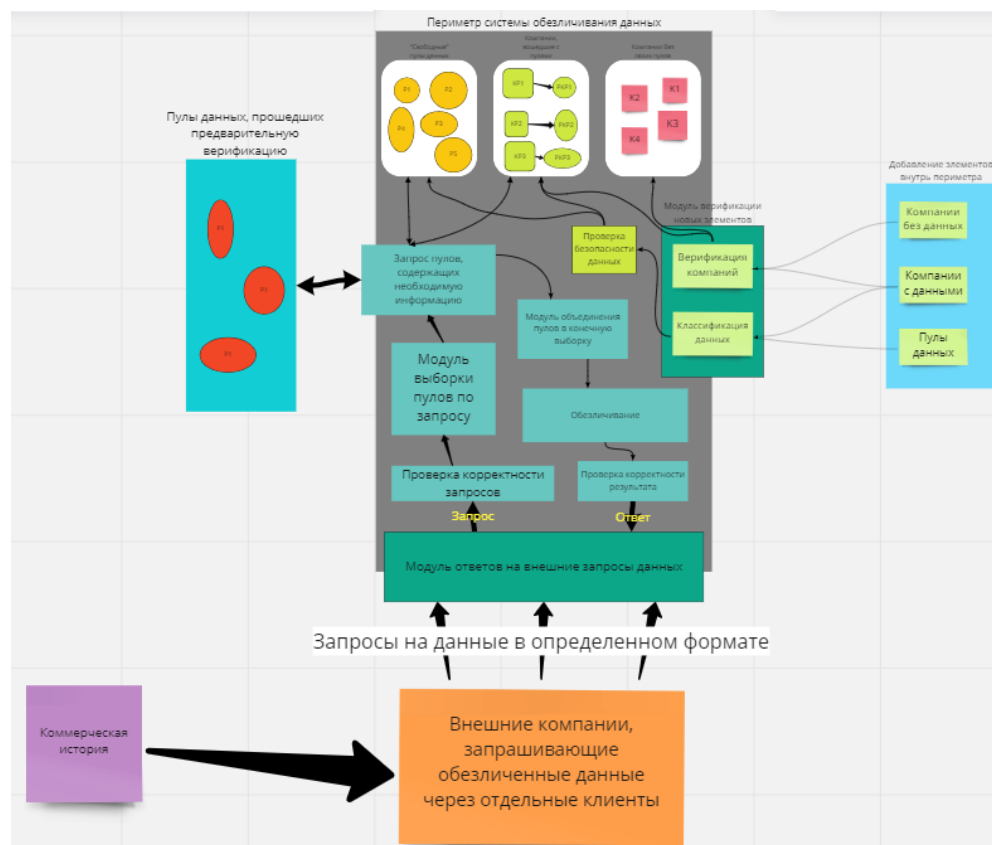


Рисунок 3.1 – Схема работы системы интеллектуального управления, обогащения и обезличивания данных

На сегодняшний день реляционный подход является наиболее распространенной моделью для организации баз данных в мире благодаря своим преимуществам. Реляционные базы данных обеспечивают обслуживание информационных потребностей, где данные взаимосвязаны и требуется обеспечить их целостность. Эта модель предоставляет стандартный способ представления и запроса данных, пригодный для использования в различных приложениях. Основным преимуществом реляционных БД являются таблицы, которые обеспечивают эффективное хранение структурированной информации и обеспечивают простой доступ к ней. Для взаимодействия с данными используется язык структурированных запросов (SQL), который обеспечивает простоту и эффективность при выполнении запросов к базе данных [10]. Реляционные БД обычно масштабируются вертикально, добавляя больше ресурсов на один сервер. Они демонстрируют высокую производительность при интенсивных операциях чтения и записи с небольшими и средними

объемами данных, а также обеспечивают быстрый доступ к данным с помощью индексов и объединения таблиц.

Наиболее известные реляционные СУБД: Oracle, MySQL, PostgreSQL, MS SQL Server. Каждая СУБД обеспечивает ИФ между пользователями и приложениями и БД, выполнение основных административных функций по управлению хранением данных и доступа к ним, но имеет свои достоинства и недостатки. Например, программное обеспечение Oracle платное, что уже ограничивает его использование.

В рамках проекта предлагается использовать СУБД PostgreSQL, учитывая ее основные достоинства и преимущества в сравнении с другими СУБД. Рассмотрим СУБД PostgreSQL более подробно.

PostgreSQL имеет открытый исходный код, который находится в свободном доступе, и является полностью бесплатным. Это дает возможность любому разработчику свободно его использовать, внедрять и модифицировать в соответствии с решаемыми задачами и потребностями.

Поддержкой PostgreSQL занимается не одна компания, а целое сообщество, которое развивает СУБД уже более 30-ти лет, быстро внедряет инновации и исправляет возможные ошибки в версиях.

Одно из основных преимуществ PostgreSQL является то, что она поддерживает БД неограниченных размеров по количеству записей и может хранить таблицы размером в 32 Тбайта.

В PostgreSQL применяются объектно-реляционные базы данных, что подразумевает наличие функций, таких как наследование таблиц и перегрузка функций, которые могут быть полезны в различных приложениях. Кроме того, PostgreSQL более строго соблюдает стандарты SQL по сравнению с другими системами управления базами данных.

PostgreSQL обладает высокой расширяемостью. Она поддерживает ряд расширенных типов данных, недоступных в других СУБД, таких как геометрические, сетевые адреса, индексируемый JSONB, собственный UUID, временные метки с учетом часового пояса. В сравнении с большинством СУБД

PostgreSQL хранит в своих каталогах гораздо больше информации: не только информацию о таблицах и столбцах, но также информацию о типах данных, функциях, методах доступа и так далее. Поэтому пользователи могут изменять эти таблицы. Это означает, что PostgreSQL может быть расширена пользователем, в отличие от большинства СУБД, которые требуют жестко запрограммированных изменений в исходном коде для модификации программного обеспечения.

PostgreSQL изначально поддерживает NoSQL, а также широкий набор типов данных, включая числовые типы, логические типы, сетевые адреса, типы битовых строк, массивы, составные типы, типы идентификаторов объектов, псевдотипы и даже геометрические типы, такие как точки, сегменты линий, прямоугольники, пути, многоугольники и круги. Она также поддерживает типы JSON, hstore и XML, и пользователи могут даже добавлять новые типы.

PostgreSQL была первой СУБД, внедрившей многоверсионный контроль параллелизма (MVCC), это означает, что чтение никогда не блокирует запись, и наоборот. Соблюдение этого требования является одной из основных причин, по которой разработчики предпочитают PostgreSQL. Дополнительным преимуществом является то, что контроль параллелизма обеспечивает значительное повышение эффективности работы.

Все вышеперечисленные достоинства PostgreSQL обосновывают ее выбор в качестве СУБД.

Для реализации методов обезличивания данных также необходимо выбрать язык программирования, так как стандартных средств языка SQL, встроенного в PostgreSQL, недостаточно для реализации сложных методов обезличивания.

В качестве языка программирования был выбран Java по следующим причинам.

В Java реализованы разнообразные функции, которые обеспечивают надежное управление памятью, высокую производительность, обратную совместимость и безопасность.

Java один из самых фундаментальных языков, который используется для создания программного обеспечения для нескольких платформ. Единоразово написанный код на Java может быть запущен на любой операционной системе, которая поддерживает этот язык, кроме того, Java — это язык программирования со статической типизацией, что делает его быстрее, чем другие языки.

Если посмотреть на практическую гибкость языка, Java обеспечивает более надежную поддержку рефакторинга, чем другие языки, благодаря своей статической системе типов. Java рассматривает безопасность как часть своего дизайна. Язык Java, компилятор, интерпретатор и среда выполнения были разработаны с учетом требований безопасности.

Java предлагает высокую кросс-функциональность и переносимость, поскольку программы, написанные на одной платформе, могут работать на настольных компьютерах, мобильных устройствах и встроенных системах. Большинство приложений Java встроены в Windows и могут работать в Linux.

Исходный код, то есть файл .java, скомпилирован и преобразован в байт-код. Байт-код можно запустить на любой машине, на которой есть виртуальная машина Java. Виртуальная машина Java изменяет байт-код в код, который может выполняться аппаратным обеспечением компьютера.

Большинство других языков программирования, таких как COBOL, C, C++, Smalltalk или Visual Basic, компилируют большую часть кода в двоичные файлы. Двоичные файлы зависят от платформы, т.е. программы, скомпилированные для компьютеров на Windows, нельзя запускать на компьютерах с операционными системами Linux или Mac.

В Java реализованы все необходимые инструменты для создания полноценного приложения, например, такие как работа с графикой и базами данных.

Сочетание реляционной СУБД PostgreSQL и языка программирования Java является хорошим выбором для создания и поддержки сложных

программных решений, поэтому эта связка технологий была выбрана для реализации функционала модуля обезличивания данных.

3.2 Описание реализованных методов обезличивания данных

Приведем примеры реализованных методов обезличивания данных с использованием выбранных программных средств и технологий. Исходная БД создана средствами СУБД PostgreSQL, методы обезличивания реализованы на языке программирования Java. В исследовании использовалась БД по инфекционной заболеваемости по городам России, всего 333038 записей.

В ходе выполнения обезличивания Java программа подключается к базе данных PostgreSQL и создает ее копию с именем "mask_имя исходной БД". После того, как была создана копия исходной базы данных, программа разрывает соединение с оригинальной базой и начинает применять методы обезличивания к клонированной базе, проводя все необходимые преобразования. Таким образом, в PostgreSQL создается и хранится новая обезличенная база данных, а также таблицы идентификаторов, если метод обезличивания предполагает их создание. Такое решение является временным, поскольку исходная и обезличенная БД, а также таблицы идентификаторов не могут храниться в одном месте. Для обеспечения информационной безопасности и надежной защиты от возможных атак злоумышленников необходимо обеспечить раздельное хранение всех баз данных.

Разработанные методы обезличивания в Java выглядит следующим образом. Каждый метод обезличивания представляет собой класс, который в полях хранит данные, а в методах реализацию необходимую для проведения обезличивания. В полях как правило хранятся такие данные, как имя таблицы, столбец или столбцы, над которыми будет производиться обезличивание, и дополнительные данные для реализации конкретного метода обезличивания. Каждый класс также содержит обязательный метод start(), который служит в качестве вызова метода для начала обезличивания. Данный метод в свои параметры принимает controllerDB для подключения к конкретной базе данных.

В работе были реализованы следующие методы обезличивания:

- *DateAging* – старение данных;
- *GeneralizationString* – обобщение для качественных признаков;
- *GeneralizationValue* – обобщение для количественных признаков или дат;
- *ValueReplacement* – замена значений атрибута на заданное значение;
- *Identifier* – введение идентификатора;
- *MicroAggregation* – Микроагрегирование по выбранным атрибутам;
- *MicroAggregationBySingleAxis* – Микроагрегирование по выбранным атрибутам путем сортировки одного из них;
- *Round* – округление;
- *Shuffle* – перемешивание;
- *ValueReplacementByPattern* – замена значений атрибута согласно заданному образцу (паттерну);
- *ValueReplacementFromFile* – замена значений атрибута на случайное значение из файла, который задается пользователем;
- *ValueVariance* – добавление шума;
- *Decomposition* – декомпозиция атрибута таблицы.

Методы *DateAging*, *ValueReplacement*, *GeneralizationString*, *GeneralizationValue*, *ValueReplacementByPattern*, *MicroAggregation*, *MicroAggregationBySingleAxis*, *ValueReplacementFromFile*, *Round* относятся к группе методов изменения состава и семантики, метод *Identifier* относится к методам введения идентификаторов, метод *Shuffle* относится к методам перемешивания согласно классификации Роскомнадзора, метод *Decomposition* относится к методу декомпозиции [1].

3.2.1 DateAging

Параметры метода:

- *nameTable* – название таблицы, которую нужно обезличить;
- *nameColumn* – название столбца в выбранной таблице;

– countDays – количество дней.

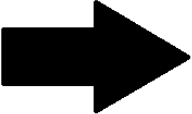
Краткое описание:

Суть метода старение даты заключается в изменении значения даты путем увеличения или уменьшения количества дней. Пользователь задаёт переменную countDays, которая является отрицательным или положительным целым числом, и определяет, на сколько дней уменьшится или увеличится исходная дата. После чего последовательно каждая запись будет изменяться на новое значение (рисунок 3.2) [11].

Пример создания объекта:

```
new DateAging("infection", "date_sickness", 10);
```

Результат работы:



id_inf	address	date_sickness
10939	Плахотного 45	2008-05-08
10940	Курчатова 11/3	2008-02-22
10941	Ельцовская 35	2008-02-25
10942	Рылеева 92	2008-02-25
10943	1п. Троллейный	2008-02-25
10944	Дачная 23/1	2008-02-25
10945	4перБрон	2008-02-25
10946	Земнухова 3	2008-02-25
10947	Сиб-Гвар 80	2008-02-25
10948	Высоцкого 5	2008-02-26
10949	2 ая Национальная	2008-02-26
10950	Блюхера 31	2008-02-25
10951	Новоуральская 13	2008-02-26
10952	Новосибирская 20	2008-02-22
10953	Рельсовая 3	2008-02-25
10954	Учительская 33	2008-02-25
10955	Пр. Дзержинского 5	2008-02-24
10956	Титова 119	2008-02-25
10957	Вертковского 39/1	2008-02-26
10958	Блюхера 61/1	2008-02-26

id_inf	address	date_sickness
10939	Плахотного 45	2008-05-18
10940	Курчатова 11/3	2008-03-03
10941	Ельцовская 35	2008-03-06
10942	Рылеева 92	2008-03-06
10943	1п. Троллейный	2008-03-06
10944	Дачная 23/1	2008-03-06
10945	4перБрон	2008-03-06
10946	Земнухова 3	2008-03-06
10947	Сиб-Гвар 80	2008-03-06
10948	Высоцкого 5	2008-03-07
10949	2 ая Национальная	2008-03-07
10950	Блюхера 31	2008-03-06
10951	Новоуральская 13	2008-03-07
10952	Новосибирская 20	2008-03-03
10953	Рельсовая 3	2008-03-06
10954	Учительская 33	2008-03-06
10955	Пр. Дзержинского 5	2008-03-05
10956	Титова 119	2008-03-06
10957	Вертковского 39/1	2008-03-07
10958	Блюхера 61/1	2008-03-07

Рисунок 3.2 – Результат обезличивания методом DateAging

3.2.2 GeneralizationString

Параметры метода:

- nameTable – название таблицы, которую нужно обезличить;
- nameColumn – название столбца в выбранной таблице;
- generalizationTable – название создаваемой методом таблицы, которая хранить соответствия обобщенных значений;
- value – хеш таблица, в которой ключ обозначает заменяемую строку, а значение обобщенную строку.

Краткое описание:

Суть метода обобщения строк заключается в изменении значения на данные, которые были предоставлены пользователем. В ходе выполнения данного метода, в базе данных создается таблица соответствий, в которой будут храниться прошлые значения и их обобщенные значения.

В ходе работы данного метода будут пройдены все записи в указанной таблице, значения которых в виде ключа будут искаться в переданной пользователям хеш-таблице. Если такой ключ будет найден, то его значение заменится на значение из хеш-таблицы.

В конце выполнения метода будет создана таблица «generalizationTable». Таблица соответствий «generalizationTable» будет создана в той же базе данных, где находится таблица, над которой происходит процесс обезличивания. В случае если в таблице содержится значение, которое пользователь не обобщил, оно пропускается и остается без изменений (рисунок 3.3).

Пример создания объекта:

```
HashMap<String, String> value = new HashMap<>();  
    value.put("A00.0", "A00");  
    value.put("A00.1", "A00");  
    value.put("A00.9", "A00");  
    value.put("A01.0", "A01");  
    value.put("A01.1", "A01");  
    value.put("A01.2", "A01");  
    value.put("A01.3", "A01");  
    value.put("A01.4", "A01");  
    value.put("A02.0", "A02");  
    value.put("A02.1", "A02");  
    value.put("A02.2+", "A02");  
    value.put("A02.8", "A02");  
    value.put("A02.9", "A02");
```

```
MaskItem item = new GeneralizationString("mkb10", "mkb10",
"gen_mkb10", value);
```

Результат работы:



id_mkb10	groups	mkb10
1	A00-A09	A00
2	A00-A09	A00.0
3	A00-A09	A00.1
4	A00-A09	A00.9
5	A00-A09	A01
6	A00-A09	A01.0
7	A00-A09	A01.1
8	A00-A09	A01.2
9	A00-A09	A01.3
10	A00-A09	A01.4
11	A00-A09	A02
12	A00-A09	A02.0
13	A00-A09	A02.1
14	A00-A09	A02.2+
15	A00-A09	A02.8
16	A00-A09	A02.9

id_mkb10	groups	mkb10
1	A00-A09	A00
2	A00-A09	A00
3	A00-A09	A00
4	A00-A09	A00
5	A00-A09	A01
6	A00-A09	A01
7	A00-A09	A01
8	A00-A09	A01
9	A00-A09	A01
10	A00-A09	A01
11	A00-A09	A02
12	A00-A09	A02
13	A00-A09	A02
14	A00-A09	A02
15	A00-A09	A02
16	A00-A09	A02

generalization	value
A01	A01.4
A02	A02.2+
A00	A00.9
A02	A02.8
A00	A00.0
A01	A01.2
A02	A02.1
A01	A01.3
A00	A00.1
A01	A01.0
A01	A01.1
A02	A02.0

Рисунок 3.3 – Результат обезличивания методом GeneralizationString

3.2.3 GeneralizationValue

Параметры метода:

Методу для работы необходима следующие данные:

- nameTable – название таблицы, которую нужно обезличить;
- nameColumn – название столбца в выбранной таблице;
- generalizationTable – название создаваемой методом таблицы, которая хранить соответствия обобщенных значений;
- generalizationName – название столбца в таблице обобщенных значений;
- minValue – лист, в котором храниться начало диапазона обобщения;
- maxValue – лист, в котором храниться конец диапазона обобщения;
- isDate – флаг, обозначающий обобщение даты.

Краткое описание:

Метод принимает два списка значений, которые хранят начальные значение «х» и конечные значения «у» диапазонов. При этом диапазон значений будет разделен как [х, у). То есть заданное начальное значение будет входить в диапазон, а конечное не будет. В случае, если нужно обобщить даты, то нужно установить флаг «isDate» в значение true, иначе false.

В начале будет создана таблица соответствий «generalizationTable», которая будет хранить в себе идентификаторы диапазонов и их обозначения. После этого в обезличиваемой таблице будет создан новый столбец с идентификаторами. Они будут заполняться на основе значений из обезличиваемого столбца. Как только все идентификаторы будут назначены, обезличиваемый столбец будет удален.

Таблица соответствий «generalizationTable» будет создана в той же базе данных, где находится таблица, над которой происходит процесс обезличивания. В этой таблице будут храниться идентификаторы и указанные пользователем названия обобщенных значений (рисунок 3.4).

Пример создания объекта:

```
ArrayList<String> gen = new ArrayList<>();
    gen.add("До 1 года");
    gen.add("От 1 года до 3-х лет");
    gen.add("От 3 до 18 лет");
    gen.add("от 18 до 60 лет");
    gen.add("старше 60 лет");
    ArrayList<Integer> min = new ArrayList<>();
    min.add(Integer.MIN_VALUE);
    min.add(1);
    min.add(3);
    min.add(18);
    min.add(60);
    ArrayList<Integer> max = new ArrayList<>();
    max.add(1);
    max.add(3);
    max.add(18);
    max.add(60);
    max.add(Integer.MAX_VALUE);
```

```
MaskItem item = new GeneralizationValue("test", "age_year", "gen_age",
gen, min, max, false);
```

Результат работы:

id_inf	id_city	date_reg	ds_confirmed	district	age_year
10939	1	2008-05-10	ОКИ неуставленной этиологии	Ленинский	15
10940	1	2008-02-26	Citrobacter ; Ротавирус ; Энтерит ротавирусный	Калининский	0
10941	1	2008-02-26	ОКИ неуставленной этиологии	Заельцовский	0
10942	1	2008-02-26	ОКИ неуставленной этиологии	Дзержинский	2
10943	1	2008-02-26	ОКИ неуставленной этиологии	Ленинский	2
10944	1	2008-02-26	ОКИ неуставленной этиологии	Заельцовский	0
10945	1	2008-02-26	ОКИ неуставленной этиологии	Кировский	1
10946	1	2008-02-26	ОКИ неуставленной этиологии	Калининский	3
10947	1	2008-02-26	ОКИ неуставленной этиологии	Кировский	0
10948	1	2008-02-26	ОКИ неуставленной этиологии	Октябрьский	0



id_inf	id_city	date_reg	ds_confirmed	district	id_age_generalization
10939	1	2008-05-10	ОКИ неуставленной этиологии	Ленинский	3
10940	1	2008-02-26	Citrobacter ; Ротавирус ; Энтерит ротавирусный	Калининский	1
10941	1	2008-02-26	ОКИ неуставленной этиологии	Заельцовский	1
10942	1	2008-02-26	ОКИ неуставленной этиологии	Дзержинский	2
10943	1	2008-02-26	ОКИ неуставленной этиологии	Ленинский	2
10944	1	2008-02-26	ОКИ неуставленной этиологии	Заельцовский	1
10945	1	2008-02-26	ОКИ неуставленной этиологии	Кировский	1
10946	1	2008-02-26	ОКИ неуставленной этиологии	Калининский	2
10947	1	2008-02-26	ОКИ неуставленной этиологии	Кировский	1
10948	1	2008-02-26	ОКИ неуставленной этиологии	Октябрьский	1



id	class_name
1	До 1 года
2	От 1 года до 3-х лет
3	От 3 до 18 лет
4	от 18 до 60 лет
5	старше 60 лет

Рисунок 3.4 – Результат обезличивания методом GeneralizationValue

3.2.4 Identifier

Параметры метода:

- nameTable – название таблицы, которую нужно обезличить;
- namesColumn – массив имен столбцов в выбранной таблице;
- newNameTable – название таблицы идентификаторов.

Описание метода:

Метод Identifier реализует обезличивание путем введения идентификатора. Он создает новую таблицу, название которой указывает пользователь, куда переносятся все выбранные из исходной таблицы столбцы. Затем создаются Primary key (столбцы-идентификаторы) в обеих таблицах, которые связывают записи из предыдущей таблицы с записями из новой. На последнем этапе столбцы с исходными значениями удаляются (рисунок 3.5). Идентификатор вычисляется как наибольшее значение в таблице и его порядковый номер.

Пример создания объекта:

```
new Identifier("infection", new String[]{"address"}, "address");
```

Результат работы:

id_inf	id_city	date_reg	district	address
10939	1	2008-05-10	Ленинский	Плахотного 45
10940	1	2008-02-26	Калининский	Курчатова 11/3
10941	1	2008-02-26	Завельцовский	Ельцовская 35
10942	1	2008-02-26	Дзержинский	Рылеева 92
10943	1	2008-02-26	Ленинский	1п. Троллейный
10944	1	2008-02-26	Завельцовский	Дачная 23/1
10945	1	2008-02-26	Кировский	4перБрон
10946	1	2008-02-26	Калининский	Земнухова 3
10947	1	2008-02-26	Кировский	Сиб-Гвар 80
10948	1	2008-02-26	Октябрьский	Высоцкого 5
10949	1	2008-02-26	Дзержинский	2 ая Национальная
10950	1	2008-02-26	Ленинский	Блюхера 31
10951	1	2008-02-26	Калининский	Новоуральская 13
10952	1	2008-02-26	Ленинский	Новосибирская 20
10953	1	2008-02-26	Завельцовский	Рельсовая 3
10954	1	2008-02-26	Калининский	Учительская 33
10955	1	2008-02-26	Дзержинский	Пр. Дзержинского 5
10956	1	2008-02-26	Ленинский	Титова 119
10957	1	2008-02-26	Ленинский	Вертковского 39/1
10958	1	2008-02-26	Ленинский	Блюхера 61/1



id_inf	id_city	date_reg	district	id_address
10939	1	2008-05-10	Ленинский	4591967dc48280787ef4de13d629af6d7bf4f17a2f4778c1152b7187e44f3732
10940	1	2008-02-26	Калининский	abd09a32e917ad1d2819148c27ded827667de8ab8990097ef6974cb267f95682
10941	1	2008-02-26	Завельцовский	d0fef177afdee61d041a625589178a9d087f88a3e5209a8f39cebcfb2aae958
10942	1	2008-02-26	Дзержинский	e5611f7aa3fcd929e1dc7966630cababd99fe8de668c205bccf3a55ef32f9e70
10943	1	2008-02-26	Ленинский	b976eed1942c177c57184ac5b5e2103ab44a9d7f7f16917ba1b38ef8ad2d9990
10944	1	2008-02-26	Завельцовский	1925c848fc88d2ddab11f17b2b56da70aa31c9917fd1e49b76a12e8846b7c7f7
10945	1	2008-02-26	Кировский	4eb32d007f5a2be84f3f94e80be8190bead3a7c831fe16ee5586875267b81b4
10946	1	2008-02-26	Калининский	0b523b4122b5391f2ac0a8281f9d6102020f6bf1f03599e403f2de23e2ace2f8
10947	1	2008-02-26	Кировский	dabc82ad393a545b3d509fa0cf8416b269f2e580e065ee3488bb0082270bd399
10948	1	2008-02-26	Октябрьский	07ce7f45a978829ab5a355bb9a99cc19fc86d42a43c995eb1c71900efbdfc021
10949	1	2008-02-26	Дзержинский	cff00a863c416d40e8b76e7263705e19623003c9b3d07c1403405b9eda2e00e9
10950	1	2008-02-26	Ленинский	1c2ead41e54fa812ab4d4cc551ff2450c62b7e0517956513a8d24280ea8353d8b
10951	1	2008-02-26	Калининский	f57a21aebfaa76345dd1ba3d9f1d2b76eca5bf2ca2c4f57615d81bd5a9963076
10952	1	2008-02-26	Ленинский	23f48183429720314eb41d41236886651ea080db6275cc53daebf3024851140c
10953	1	2008-02-26	Завельцовский	a7d9bb1b896fd5821db74dd21c3062cf8e1daef99ad7b7c66cfa20196545e8
10954	1	2008-02-26	Калининский	90d6a074dbb7da2535e4868fa34ce1b09068af28416010f12c109f6204a76e69
10955	1	2008-02-26	Дзержинский	58e403b8f3531b3c2e8a2d236cd284aa0c6114974de9387b55f46b41a5fe2ff6
10956	1	2008-02-26	Ленинский	2fd4a75affbb49ed03669edd9f15adb5047cb311a3f59c1dc34a89f3eb6267
10957	1	2008-02-26	Ленинский	885c44a4cc210a9079f3b45fd76f90b8bc4376dc16c6fd1c6c25980636be546e
10958	1	2008-02-26	Ленинский	426d5cc0c373bb8640cf635a3365822bd65d7b1417ef28bf6de8b0e4e0cd432f



address	id_address
Плахотного 45	4591967dc48280787ef4de13d629af6d7bf4f17a2f4778c1152b7187e44f3732
Курчатова 11/3	abd09a32e917ad1d2819148c27ded827667de8ab8990097ef6974cb267f95682
Ельцовская 35	d0fef177afdee61d041a625589178a9d087f88a3e5209a8f39cebcfb2aae958
Рылеева 92	e5611f7aa3fcd929e1dc7966630cababd99fe8de668c205bccf3a55ef32f9e70
1п. Троллейный	b976eed1942c177c57184ac5b5e2103ab44a9d7f7f16917ba1b38ef8ad2d9990
Дачная 23/1	1925c848fc88d2ddab11f17b2b56da70aa31c9917fd1e49b76a12e8846b7c7f7
4перБрон	4eb32d007f5a2be84f3f94e80be8190bead3a7c831fe16ee5586875267b81b4
Земнухова 3	0b523b4122b5391f2ac0a8281f9d6102020f6bf1f03599e403f2de23e2ace2f8
Сиб-Гвар 80	dabc82ad393a545b3d509fa0cf8416b269f2e580e065ee3488bb0082270bd399
Высоцкого 5	07ce7f45a978829ab5a355bb9a99cc19fc86d42a43c995eb1c71900efbdfc021
2 ая Национальная	cff00a863c416d40e8b76e7263705e19623003c9b3d07c1403405b9eda2e00e9
Блюхера 31	1c2ead41e54fa812ab4d4cc551ff2450c62b7e0517956513a8d24280ea8353d8b
Новоуральская 13	f57a21aebfaa76345dd1ba3d9f1d2b76eca5bf2ca2c4f57615d81bd5a9963076
Новосибирская 20	23f48183429720314eb41d41236886651ea080db6275cc53daebf3024851140c
Рельсовая 3	a7d9bb1b896fd5821db74dd21c3062cf8e1daef99ad7b7c66cfa20196545e8
Учительская 33	90d6a074dbb7da2535e4868fa34ce1b09068af28416010f12c109f6204a76e69
Пр. Дзержинского 5	58e403b8f3531b3c2e8a2d236cd284aa0c6114974de9387b55f46b41a5fe2ff6
Титова 119	2fd4a75affbb49ed03669edd9f15adb5047cb311a3f59c1dc34a89f3eb6267
Вертковского 39/1	885c44a4cc210a9079f3b45fd76f90b8bc4376dc16c6fd1c6c25980636be546e
Блюхера 61/1	426d5cc0c373bb8640cf635a3365822bd65d7b1417ef28bf6de8b0e4e0cd432f

Рисунок 3.5 – Результат обезличивания методом Identifier

3.2.5 MicroAggregation

Параметры метода:

- *nameTable* – название таблицы, которую нужно обезличить;
- *namesColumn* – массив имен столбцов в выбранной таблице;
- *k* – число неразличимых (одинаковых) записей.

Описание метода:

Метод микроагрегации создает одинаковые группы записей. Эти группы формируются из записей, которые не вносят слишком большой вклад в группу. Для каждой переменной вычисляется среднее значение по каждой группе, которое используется для замены каждого из исходных усредненных значений. Группы формируются по критерию максимального сходства.

В начале формируется вектор $X_{\text{среднее}}$, который вычисляется как среднее значение всех записей. Затем относительно $X_{\text{среднее}}$ при помощи квадрата Евклидова расстояния находится X_r (Самая дальняя запись), а относительно X_r находится X_s (Самая дальняя запись относительно X_r). Формируются две группы размерности $k-1$, одна из которых ближе к значениям X_r , а другая к значениям X_s . После формирования групп значений записей меняются на среднее значение группы. Если нераспределенных записей остается больше, чем $3k$, цикл повторяется. Если нераспределенных записей остается между $3k-1$ и $2k$, то первая группа записей формируется, как и всегда, а вторая собирается из оставшихся записей. Если же нераспределенных записей будет $2k$ и менее, то все они формируются в одну группу (рисунок 3.6) [12].

Пример создания объекта:

```
new MicroAggregation("population", new String[]{"years", "newborn",  
"total"},5);
```

Результат работы:

id_city	years	newborn	p1_6	p0_15	p15_more	p7_15	man	woman	total
6	2003	13477	60500	192900	1094100	118923	580800	706200	1287000
6	2004	13756	64600	189900	1114400	111544	587800	716500	1304300
6	2005	13396	68200	185700	1122700	104104	589200	719200	1308400
6	2006	13867	71700	183900	1131200	98333	591900	723200	1315100
6	2007	14492	75200	184600	1138400	94908	595300	727700	1323000
6	2008	15820	79000	188200	1144100	93380	598500	733800	1332300
5	2003	5551	24700	83500	506800	53249	289600	327200	590300
5	2004	5705	25900	80600	506200	48995	287400	326000	586800
5	2005	5614	27300	78000	505700	45086	285400	324800	583700
5	2006	5648	28700	75800	505000	41452	283700	323600	580800
5	2007	5993	30000	75100	503700	39107	282400	323000	578800
5	2008	6113	31600	75500	503100	37787	282200	323000	578600
3	2003	11091	53200	169200	901800	104909	494800	603300	1071000
3	2004	11389	57400	168700	926400	99911	492600	602500	1095100
3	2005	11039	60200	164700	928300	93461	490800	602200	1093000
3	2006	11242	62700	161800	929700	87858	489600	601900	1091500
3	2007	11580	64800	161100	931400	84720	489900	602600	1092500
3	2008	12578	67100	162600	931100	82922	490200	603500	1093700
8	2003	9950	44700	147400	765400	92750	412600	500200	912800
8	2004	9997	47300	143500	773700	86203	414200	503000	917200
8	2005	10038	50000	140300	780600	80262	415100	505800	920900
8	2006	10292	53100	138400	788800	75008	417800	509400	927200
8	2007	10683	56400	138500	797900	71417	421700	514700	936400
8	2008	12046	59200	140900	806900	69654	426900	520900	947800
2	2003	6754	30800	94200	541600	56646	283800	352000	635800
2	2004	7041	32400	91200	540000	51759	281300	349900	631200
2	2005	6926	33000	86500	517700	46574	269000	335200	604200
2	2006	6562	34700	85100	515000	43838	267000	333100	600100
2	2007	6930	35700	84900	512300	42270	265500	331700	597200
2	2008	7678	37000	86400	510400	41722	265100	331700	596800
1	2003	13706	61600	201900	1211100	126594	644000	769000	1413000
1	2004	14337	65200	195600	1210000	116063	639800	765800	1405600
1	2005	14110	69600	190900	1206100	107190	634800	762200	1397000
1	2006	13780	73800	187600	1204300	100020	631700	760200	1391900
1	2007	14600	77400	186900	1203600	94900	630100	760400	1390500
1	2008	16347	81500	190400	1206800	92553	632700	764500	1397200



id_city	years	newborn	p1_6	p0_15	p15_more	p7_15	man	woman	total
3	2006	11156	53200	169200	901800	104909	494800	603300	1011067
3	2006	11156	62700	161800	929700	87858	489600	601900	1011067
3	2006	11156	64800	161100	931400	84720	489900	602600	1011067
8	2006	11156	53100	138400	788800	75008	417800	509400	1011067
8	2006	11156	56400	138500	797900	71417	421700	514700	1011067
1	2005	14456	61600	201900	1211100	126594	644000	769000	1400940
1	2005	14456	65200	195600	1210000	116063	639800	765800	1400940
1	2005	14456	69600	190900	1206100	107190	634800	762200	1400940
1	2005	14456	81500	190400	1206800	92553	632700	764500	1400940
1	2005	14456	73800	187600	1204300	100020	631700	760200	1400940
8	2006	11156	59200	140900	806900	69654	426900	520900	1011067
5	2006	5815	31600	75500	503100	37787	282200	323000	581740
5	2006	5815	30000	75100	503700	39107	282400	323000	581740
5	2006	5815	28700	75800	505000	41452	283700	323600	581740
5	2006	5815	27300	78000	505700	45086	285400	324800	581740
5	2006	5815	25900	80600	506200	48995	287400	326000	581740
1	2007	14435	77400	186900	1203600	94900	630100	760400	1333860
6	2007	14435	79000	188200	1144100	93380	598500	733800	1333860
6	2007	14435	75200	184600	1138400	94908	595300	727700	1333860
6	2007	14435	71700	183900	1131200	98333	591900	723200	1333860
6	2007	14435	68200	185700	1122700	104104	589200	719200	1333860
5	2006	6729	24700	83500	506800	53249	289600	327200	597720
2	2006	6729	37000	86400	510400	41722	265100	331700	597720
2	2006	6729	35700	84900	512300	42270	265500	331700	597720
2	2006	6729	34700	85100	515000	43838	267000	333100	597720
2	2006	6729	33000	86500	517700	46574	269000	335200	597720
2	2004	8756	32400	91200	540000	51759	281300	349900	803580
2	2004	8756	30800	94200	541600	56646	283800	352000	803580
8	2004	8756	44700	147400	765400	92750	412600	500200	803580
8	2004	8756	47300	143500	773700	86203	414200	503000	803580
8	2004	8756	50000	140300	780600	80262	415100	505800	803580
6	2005	12448	64600	189900	1114400	111544	587800	716500	1174620
6	2005	12448	60500	192900	1094100	118923	580800	706200	1174620
3	2005	12448	57400	168700	926400	99911	492600	602500	1174620
3	2005	12448	67100	162600	931100	82922	490200	603500	1174620
3	2005	12448	60200	164700	928300	93461	490800	602200	1174620

Рисунок 3.6 – Результат обезличивания методом MicroAggregation

3.2.6 *MicroAggregationBySingleAxis*

Параметры метода:

- *nameTable* – название таблицы, которую нужно обезличить;
- *namesColumn* – массив имен столбцов в выбранной таблице;
- *axisColumn* – столбец, который был ранее указан в “*namesColumn*” и по которому будет осуществляться формирование групп;
- *k* – число неразличимых (одинаковых) записей.

Описание метода:

Метод микроагрегации по одному атрибуту создает группы одинаковых записей размерностью *k*. Эти группы формируются относительно атрибута «*axisColumn*», поэтому в начале работы алгоритма данный атрибут сортируется, после чего берутся первые подряд идущие записи в количестве *k*. По данным записям вычисляется среднее значение их столбцов, и оно присваивается всем записям в группе. Данный алгоритм продолжается, пока не останется нераспределенных записей (рисунок 3.7) [13].

Пример создания объекта:

```
new MicroAggregationBySingleAxis("population",10, "years", new String[]{  
"years", "newborn", "total"});
```

Результат работы:

id_city	years	newborn	p1_6	p0_15	p15_more	p7_15	man	woman	total
6	2003	13477	60500	192900	1094100	118923	580800	706200	1287000
6	2004	13756	64600	189900	1114400	111544	587800	716500	1304300
6	2005	13396	68200	185700	1122700	104104	589200	719200	1308400
6	2006	13867	71700	183900	1131200	98333	591900	723200	1315100
6	2007	14492	75200	184600	1138400	94908	595300	727700	1323000
6	2008	15820	79000	188200	1144100	93380	598500	733800	1332300
5	2003	5551	24700	83500	506800	53249	289600	327200	590300
5	2004	5705	25900	80600	506200	48995	287400	326000	586800
5	2005	5614	27300	78000	505700	45086	285400	324800	583700
5	2006	5648	28700	75800	505000	41452	283700	323600	580800
5	2007	5993	30000	75100	503700	39107	282400	323000	578800
5	2008	6113	31600	75500	503100	37787	282200	323000	578600
3	2003	11091	53200	169200	901800	104909	494800	603300	1071000
3	2004	11389	57400	168700	926400	99911	492600	602500	1095100
3	2005	11039	60200	164700	928300	93461	490800	602200	1093000
3	2006	11242	62700	161800	929700	87858	489600	601900	1091500
3	2007	11580	64800	161100	931400	84720	489900	602600	1092500
3	2008	12578	67100	162600	931100	82922	490200	603500	1093700
8	2003	9950	44700	147400	765400	92750	412600	500200	912800
8	2004	9997	47300	143500	773700	86203	414200	503000	917200
8	2005	10038	50000	140300	780600	80262	415100	505800	920900
8	2006	10292	53100	138400	788800	75008	417800	509400	927200
8	2007	10683	56400	138500	797900	71417	421700	514700	936400
8	2008	12046	59200	140900	806900	69654	426900	520900	947800
2	2003	6754	30800	94200	541600	56646	283800	352000	635800
2	2004	7041	32400	91200	540000	51759	281300	349900	631200
2	2005	6926	33000	86500	517700	46574	269000	335200	604200
2	2006	6562	34700	85100	515000	43838	267000	333100	600100
2	2007	6930	35700	84900	512300	42270	265500	331700	597200
2	2008	7678	37000	86400	510400	41722	265100	331700	596800
1	2003	13706	61600	201900	1211100	126594	644000	769000	1413000
1	2004	14337	65200	195600	1210000	116063	639800	765800	1405600
1	2005	14110	69600	190900	1206100	107190	634800	762200	1397000
1	2006	13780	73800	187600	1204300	100020	631700	760200	1391900
1	2007	14600	77400	186900	1203600	94900	630100	760400	1390500
1	2008	16347	81500	190400	1206800	92553	632700	764500	1397200



id_city	years	newborn	p1_6	p0_15	p15_more	p7_15	man	woman	total
6	2003	10432	60500	192900	1094100	118923	580800	706200	1012380
5	2003	10432	24700	83500	506800	53249	289600	327200	1012380
3	2003	10432	53200	169200	901800	104909	494800	603300	1012380
8	2003	10432	44700	147400	765400	92750	412600	500200	1012380
2	2003	10432	30800	94200	541600	56646	283800	352000	1012380
1	2003	10432	61600	201900	1211100	126594	644000	769000	1012380
5	2003	10432	25900	80600	506200	48995	287400	326000	1012380
8	2003	10432	47300	143500	773700	86203	414200	503000	1012380
1	2003	10432	65200	195600	1210000	116063	639800	765800	1012380
6	2003	10432	64600	189900	1114400	111544	587800	716500	1012380
3	2005	9549	57400	168700	926400	99911	492600	602500	914150
2	2005	9549	32400	91200	540000	51759	281300	349900	914150
1	2005	9549	69600	190900	1206100	107190	634800	762200	914150
2	2005	9549	33000	86500	517700	46574	269000	335200	914150
8	2005	9549	50000	140300	780600	80262	415100	505800	914150
3	2005	9549	60200	164700	928300	93461	490800	602200	914150
6	2005	9549	68200	185700	1122700	104104	589200	719200	914150
5	2005	9549	27300	78000	505700	45086	285400	324800	914150
8	2005	9549	53100	138400	788800	75008	417800	509400	914150
5	2005	9549	28700	75800	505000	41452	283700	323600	914150
3	2007	10973	62700	161800	929700	87858	489600	601900	1031700
1	2007	10973	73800	187600	1204300	100020	631700	760200	1031700
2	2007	10973	34700	85100	515000	43838	267000	333100	1031700
6	2007	10973	71700	183900	1131200	98333	591900	723200	1031700
6	2007	10973	75200	184600	1138400	94908	595300	727700	1031700
2	2007	10973	35700	84900	512300	42270	265500	331700	1031700
5	2007	10973	30000	75100	503700	39107	282400	323000	1031700
3	2007	10973	64800	161100	931400	84720	489900	602600	1031700
8	2007	10973	56400	138500	797900	71417	421700	514700	1031700
1	2007	10973	77400	186900	1203600	94900	630100	760400	1031700
8	2008	11764	59200	140900	806900	69654	426900	520900	991067
3	2008	11764	67100	162600	931100	82922	490200	603500	991067
2	2008	11764	37000	86400	510400	41722	265100	331700	991067
1	2008	11764	81500	190400	1206800	92553	632700	764500	991067
5	2008	11764	31600	75500	503100	37787	282200	323000	991067
6	2008	11764	79000	188200	1144100	93380	598500	733800	991067

Рисунок 3.7 – Результат обезличивания методом MicroAggregationBySingleAxis

3.2.7 Round

Параметры метода:

- *nameTable* – название таблицы, которую нужно обезличить;
- *nameColumn* – название столбца в выбранной таблице;
- *precision* – число знаков после запятой.

Описание метода:

Этот метод округляет значение атрибута до точности, указанной пользователем в переменной «precision». Значение точности не может быть отрицательным (рисунок 3.8).

Пример создания объекта:

```
new Round("water_day", "troubl_n", 1);
```

Результат работы:



id_water	troubl_n
1	0.12
2	0.13
3	0.15
4	0.11
5	0.088
6	0.071
7	0.08
8	0.11
9	0.12
10	0.06
11	0.03
12	0.06
13	0.018
14	0.045
15	0.039
16	0.06
17	0.06
18	0.06
19	0.09
20	0.09
21	0.07
22	0.1
23	0.08
24	0.08
25	0.13

id_water	troubl_n
1	0.1
2	0.1
3	0.2
4	0.1
5	0.1
6	0.1
7	0.1
8	0.1
9	0.1
10	0.1
11	0
12	0.1
13	0
14	0
15	0
16	0.1
17	0.1
18	0.1
19	0.1
20	0.1
21	0.1
22	0.1
23	0.1
24	0.1
25	0.1

Рисунок 3.8 – Результат обезличивания методом Round

3.2.8 Shuffle

Параметры метода:

- *nameTable* – имя таблицы, которую нужно обезличить;
- *namesColumn* – массив имен столбцов в выбранной таблице.

Описание метода:

Метод Shuffle реализует алгоритм подстановки в поля выбранного столбца «чужих» значений (из другой записи этой же БД). Отличие от предыдущего метода заключается в том, что изменяются не сами значения признака (атрибута), а их места. Алгоритм подстановки может быть применен как к одному столбцу, так и к группе столбцов (групповое перемешивание). В этом случае соответствие значений полей столбцов в рамках группы сохраняется, но не сохраняется относительно других столбцов БД.

Схема работы метода перемешивания представлена на рисунке 3.9. Выбирается случайная строка, и значение поля в обезличенном столбце меняется на значение поля этого же столбца из последней строки таблицы. Затем также выбирается случайная строка, но обмен значениями полей происходит уже с предпоследней строкой таблицы и так далее. Строка, с которой происходит обмен данными со случайной строкой, не может быть выбрана как случайная строка. Также нельзя выбирать случайными строками те строки, которые уже были выбраны при предыдущих обменах (рисунок 3.10).

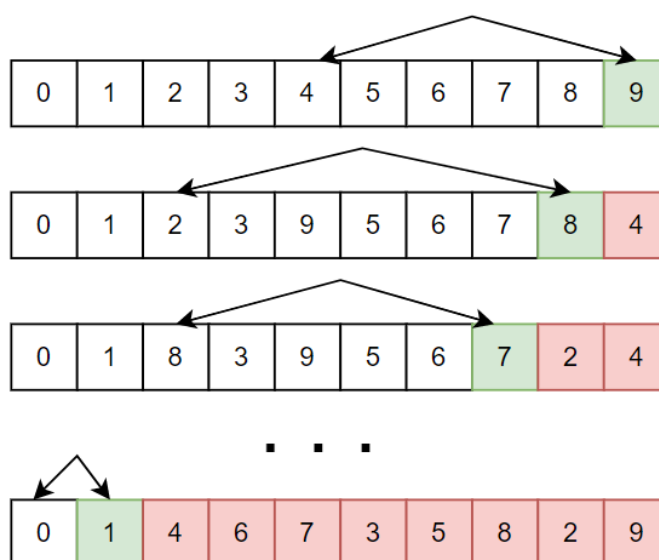


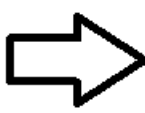
Рисунок 3.9 – Схема работы алгоритма перемешивания

Пример создания объекта:

```
new Shuffle("city", new String[]{"city_name"});
```

Результат работы:

id_city	name_city
1	Novosibirsk
2	Barnaul
3	Chelyabinsk
4	Irkutsk
5	Vladivostok
6	Ekaterinburg
7	Habarovsk
8	Krasnoyarsk



id_city	name_city
1	Irkutsk
2	Vladivostok
3	Novosibirsk
4	Krasnoyarsk
5	Ekaterinburg
6	Habarovsk
7	Chelyabinsk
8	Barnaul

Рисунок 3.10 – Результат обезличивания методом *shuffle*

3.2.9 ValueReplacement

Параметры метода:

- `nameTable` – имя таблицы, которую нужно обезличить;
- `nameColumn` – имя столбца (атрибута) в выбранной таблице;
- `value` – значение, которым будут обезличиваться поля.

Описание метода:

Метод последовательно обезличивает все поля выбранного атрибута значениями, указанными пользователем (рисунок 3.11).

Пример создания объекта:

```
new ValueReplacement("city", "name_city", "none");
```

Результат работы:

id_city	name_city
1	Novosibirsk
2	Barnaul
3	Chelyabinsk
4	Irkutsk
5	Vladivostok
6	Ekaterinburg
7	Habarovsk
8	Krasnoyarsk

(8 строк)



id_city	name_city
1	None
2	None
3	None
4	None
5	None
6	None
7	None
8	None

(8 строк)

Рисунок 3.11 – Результат обезличивания методом *ValueReplacement*

3.2.10 ValueReplacementByPattern

Параметры метода:

- `nameTable` – название таблицы, которую нужно обезличить;
- `nameColumn` – название столбца в выбранной таблице;

– *regex* – регулярное выражение (паттерн), по которому будут искаться символы;

– *replacement* – строка, на которые будут заменяться найденные символы.

Описание метода:

Метод `ValueReplacementByPattern` реализует замену значений полей в обезличиваемом столбце по заданному образцу (паттерну).

Для задания регулярных выражений используются стандартные правила написания паттернов (рисунок 3.12).

	Символ	Значение	Пример выражения
Основные		Большинство символов соответствуют сами себе	кот
	.	Любой знак (как правило, за исключением новой строки)	
		Или	Привет Превед
	()	Группа символов для захвата — см. в разделах "квантификаторы"	
	[]	Класс символов (находится любой один из символов, указанный в скобках)	[ауоыиэяюёе]
	^	Внутри []: исключение ([^ауоыиэяюёе] найдёт всё, кроме строчных гласных)	[^ауоыиэяюёе]
	-	Внутри []: диапазон ([а-я] найдёт все строчные буквы)	[0-5]
	^	Начало строки	^во
	\$	Конец строки	за\$
	\	Эранирование (поиск символа, который обычно используется как метасимвол)	\.
Классы символов	\b	граница слова (может не работать с русским)	a\b
	\w	буква (буквы латинского алфавита и цифры, не работает с другими письменностями и расширенной латиницей, т.е. символами с диакритическими знаками типа "è", "ö")	
	\d	цифра	\d
	\s	пробел	1\s5
	\W \D \S	не буква, не цифра, не пробел	
	\t	tab	
	\n	новая строка	
Квантификаторы	*	Совпадение 0 или более раз	па*
	?	Совпадение 0 или более раз (ленивое, т.е. минимально возможное)	па
	+	Совпадение 1 и более раз	a.+a
	+?	Совпадение 1 и более раз (ленивое)	a.+?a
	?	Совпадение 0 или 1 раз	аб?
	{n}	Повтор ровно n раз	,{2}
	{n,}	Повтор минимум n раз	приве{2}т
	{n, m}	Повтор минимум n и максимум m раз	(ха){2,5}

Рисунок 3.12 – Правила написания паттернов

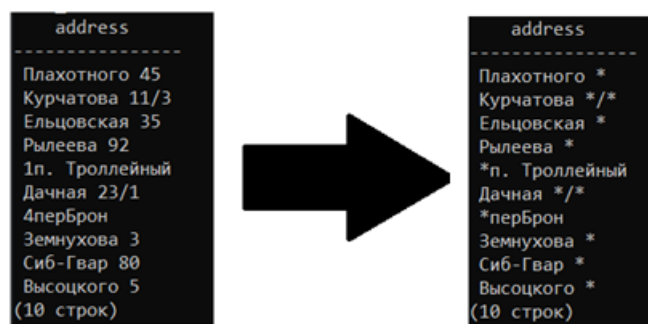
Таким образом, метод обезличивает значения полей путем нахождения в них регулярных выражений и замены их на символы, заданные пользователем.

Для примера, обезличим столбец `address` в таблице `infection` по регулярному выражению `'[0-9]+'`, и заменим найденные символы на `"*"` (рисунок 3.13). В результате обезличивания все числовые последовательности заменяются на `"*"`.

Пример создания объекта:

```
new ValueReplacementByPattern("infection", "address", "[0-9]+", "*");
```

Результат работы:



address
Плахотного 45
Курчатова 11/3
Ельцовская 35
Рылеева 92
1п. Троллейный
Дачная 23/1
4перБрон
Земнухова 3
Сиб-Гвар 80
Высоцкого 5
(10 строк)

address
Плахотного *
Курчатова */*
Ельцовская *
Рылеева *
*п. Троллейный
Дачная */*
*перБрон
Земнухова *
Сиб-Гвар *
Высоцкого *
(10 строк)

Рисунок 3.13 – Результат обезличивания методом ValueReplacementByPattern

3.2.11 ValueReplacementFromFile

Параметры метода:

- nameTable – название таблицы, которую нужно обезличить;
- nameColumn – название столбца в выбранной таблице;
- nameFile – название файла с записями для замены.

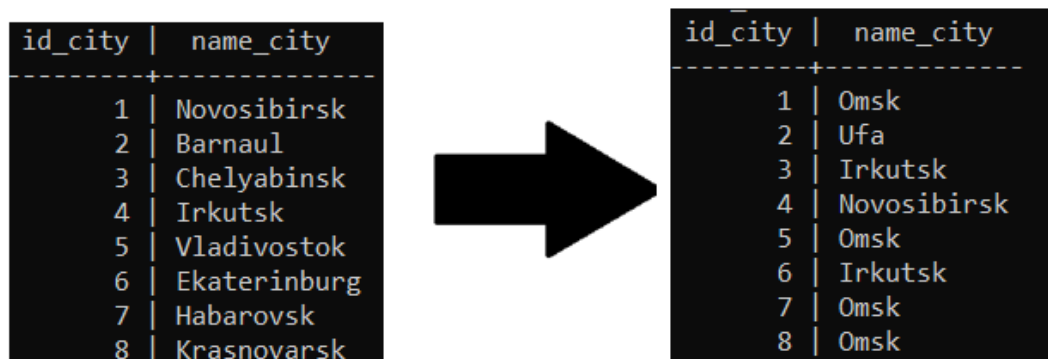
Описание метода:

Метод ValueReplacementFromFile позволяет обезличить таблицу путем замены всех записей в таблице на случайную запись из файла. В начале происходит считывание всего файла, после чего последовательно записи в базе данных заменяются на записи из строк файла, выбранные случайным образом, что не гарантирует уникальность (рисунок 3.14).

Пример создания объекта:

```
new ValueReplacementFromFile("city", "name_city", "city.txt");
```

Результат работы:



id_city	name_city
1	Novosibirsk
2	Barnaul
3	Chelyabinsk
4	Irkutsk
5	Vladivostok
6	Ekaterinburg
7	Habarovsk
8	Krasnoyarsk

id_city	name_city
1	Omsk
2	Ufa
3	Irkutsk
4	Novosibirsk
5	Omsk
6	Irkutsk
7	Omsk
8	Omsk

Рисунок 3.14 – Результат обезличивания методом ValueReplacementFromFile

3.2.12 ValueVariance

Параметры метода:

- nameTable – название таблицы, которую нужно обезличить;
- nameColumn – название столбца в выбранной таблице;
- percent – процент добавляемого шума;
- dataType – тип добавляемого шума: INTEGER, DOUBLE, DATE.

Описание метода:

Метод ValueVariance позволяет внедрить шум для выбранного пользователем атрибута. Пользователь определяет шум в виде процента «percent», который должен быть целым положительным числом. Затем алгоритм последовательно извлекает записи из базы данных и применяет к ним шум в виде процента от извлекаемого числа. Если шум применяется к дате, учитывается только месяц и день. Процент определяется как случайное значение «percent» в диапазоне от его отрицательного до положительного числа. Например, обезличим дату заболевания в таблице «infection», добавив шум в 50% (рисунок 3.15).

Пример создания объекта:

```
new ValueVariance("infection", "date_sickness", 50, ValueVariance.DataType.DATE);
```

Результат работы:



id_inf	date_sickness
10939	2008-05-08
10940	2008-02-22
10941	2008-02-25
10942	2008-02-25
10943	2008-02-25
10944	2008-02-25
10945	2008-02-25
10946	2008-02-25
10947	2008-02-25
10948	2008-02-26
10949	2008-02-26
10950	2008-02-25
10951	2008-02-26
10952	2008-02-22
10953	2008-02-25
10954	2008-02-25
10955	2008-02-24
10956	2008-02-25
10957	2008-02-26
10958	2008-02-26

id_inf	date_sickness
10939	2008-07-11
10940	2008-02-26
10941	2008-02-24
10942	2008-02-29
10943	2008-03-08
10944	2008-02-28
10945	2008-02-27
10946	2008-02-20
10947	2008-02-19
10948	2008-03-08
10949	2008-02-26
10950	2008-03-02
10951	2008-02-29
10952	2008-02-27
10953	2008-01-13
10954	2008-02-17
10955	2008-02-15
10956	2008-02-21
10957	2008-03-01
10958	2008-03-05

Рисунок 3.15 – Результат обезличивания методом ValueVariance

3.2.13 Decomposition

Параметры метода:

- nameTable – название таблицы, которую нужно обезличить;
- nameColumn – название столбца в выбранной таблице;
- nameNewTable – название новой передаваемой таблицы;
- controllerOfDatabaseToTransfer – соединение с базой данных, в которую передается таблица.

Описание метода:

Метод Decomposition позволяет вынести отдельный атрибут таблицы в отдельную таблицу, которая будет храниться в другой базе данных. Для этого создается подключение к новой базе данных, в которой создается новая таблица для переносимого атрибута, с полями «id» и «value». Затем случайным образом считываются все записи атрибута «nameColumn» и последовательно переносятся в другую базу данных. При этом в обезличиваемой таблице создается столбец идентификаторов, который будет ссылаться на записи в другой таблице. Например, декомпозируем таблицу периодов заболевания, вынеся оттуда атрибут города (рисунок 3.16).

Пример создания объекта:

```
ControllerDB controllerToTransfer = new  
ControllerDB("jdbc:postgresql://localhost:5432/", "test", "postgres", "mask");  
MaskItem item = new  
Decomposition("periods_infection", "name_city", "periods_infection_city", controllerT  
oTransfer);
```

Результат работы:

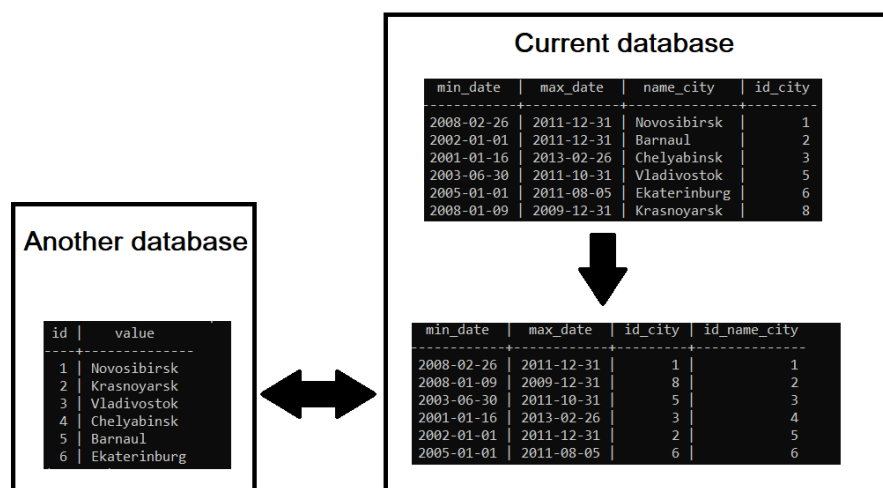


Рисунок 3.16 – Результат обезличивания методом Decomposition

3.3 Оценка скорости обезличивания данных на основе реализованных методов

Было выполнено исследование времени и скорости обезличивания данных при использовании методов обезличивания и примеров обезличивания, рассмотренных в пункте 3.2. В рамках исследования использовалась БД по инфекционной заболеваемости по городам России. Для каждого метода обезличивания выполнялся анализ зависимости времени и скорости обезличивания данных от объема БД (количества записей). В ходе эксперимента клонировалась часть исходной БД и создавались БД с разным количеством записей: от 5000 записей до 300000 записей с шагом 5000. Таким образом, было создано 60 исходных БД разных объемов, которые обезличивались в соответствии с выбранным методом обезличивания данных, для каждого метода проводилось всего 60 экспериментов. В каждом эксперименте фиксировалось время выполнения обезличивания данных для БД соответствующего объема.

Время выполнения обезличивания полностью зависит от характеристик компьютера, а именно от производительности процессора и времени записи на диск. В исследовании использовался компьютер со следующими характеристиками: процессор: Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz

2.70 GHz; оперативная память: 12,0 ГБ; видеокарта: NVIDIA GeForce 920MX; жесткий диск: ADATA SU650 520 Мбайт/сек, запись – 450 Мбайт/сек.

На рисунках 3.17-3.29 приведены графики изменения времени обезличивания БД от количества записей БД соответственно для каждого из методов обезличивания (Shuffle – перестановка, Replace – замена на заданное значение, Generalization – обобщение, Pattern – замена в соответствии с заданным паттерном, Identifier – введение идентификаторов). Для всех рассмотренных методов наблюдается линейная тенденция изменения времени обезличивания БД от количества записей БД. Отклонения от линейной зависимости (шум) связаны с загрузкой системы (нагрузка на процессор, загрузка оперативной памяти и жесткого диска). В идеальных условиях, где системе не будут «мешать» другие программы и процессы, зависимость будет линейной.

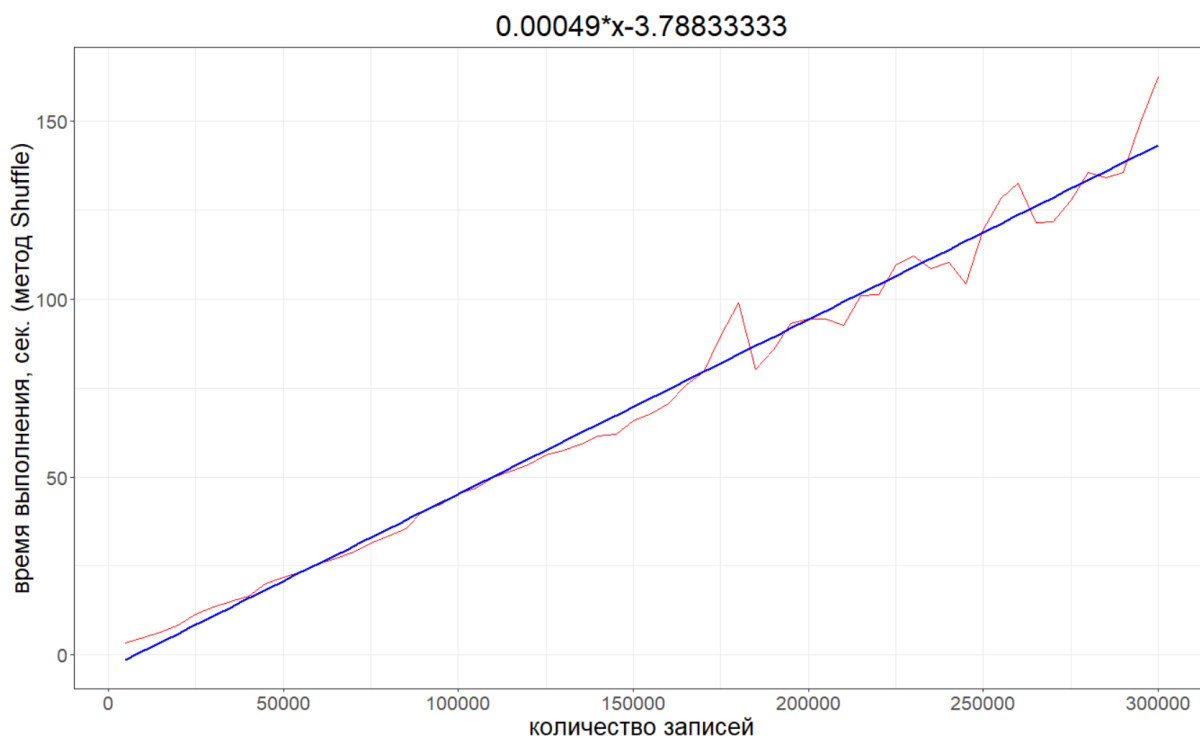
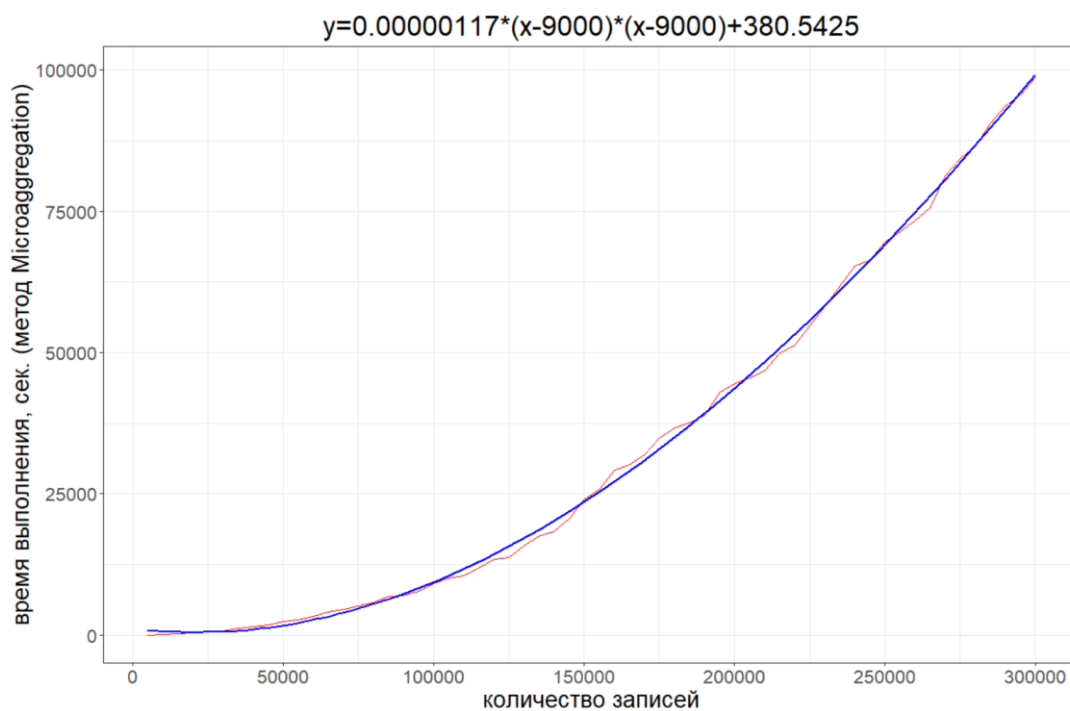
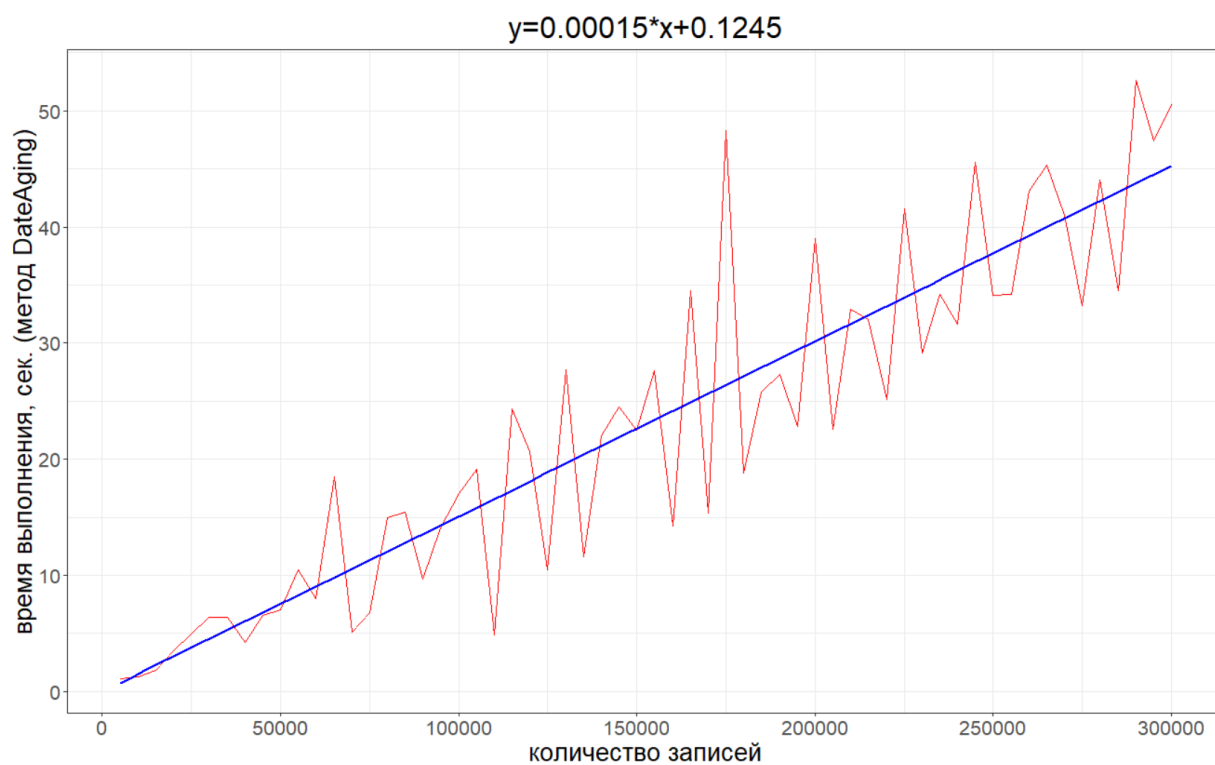


Рисунок 3.17 – Изменение времени обезличивания данных методом Shuffle от размера БД



*Рисунок 3.18 – Изменение времени обезличивания данных методом
Microaggregation от размера БД*



*Рисунок 3.19 – Изменение времени обезличивания данных методом DateAging
от размера БД*

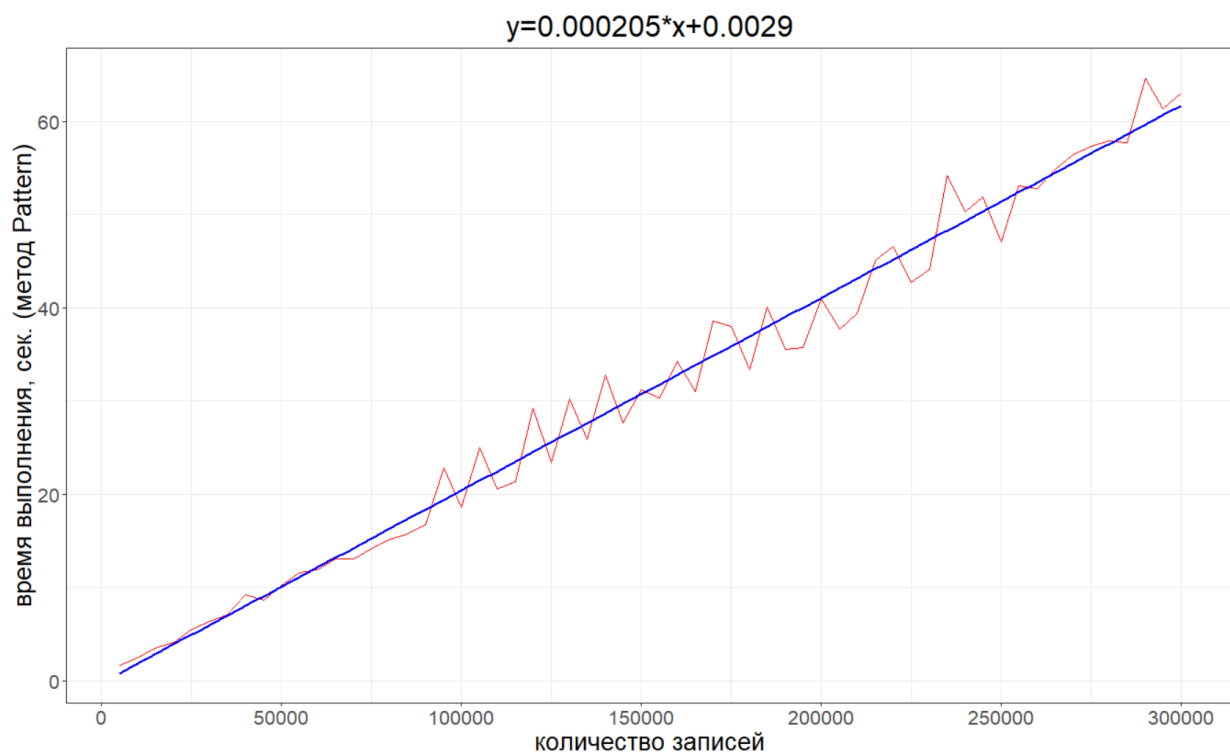


Рисунок 3.20 – Изменение времени обезличивания данных методом Pattern от размера БД

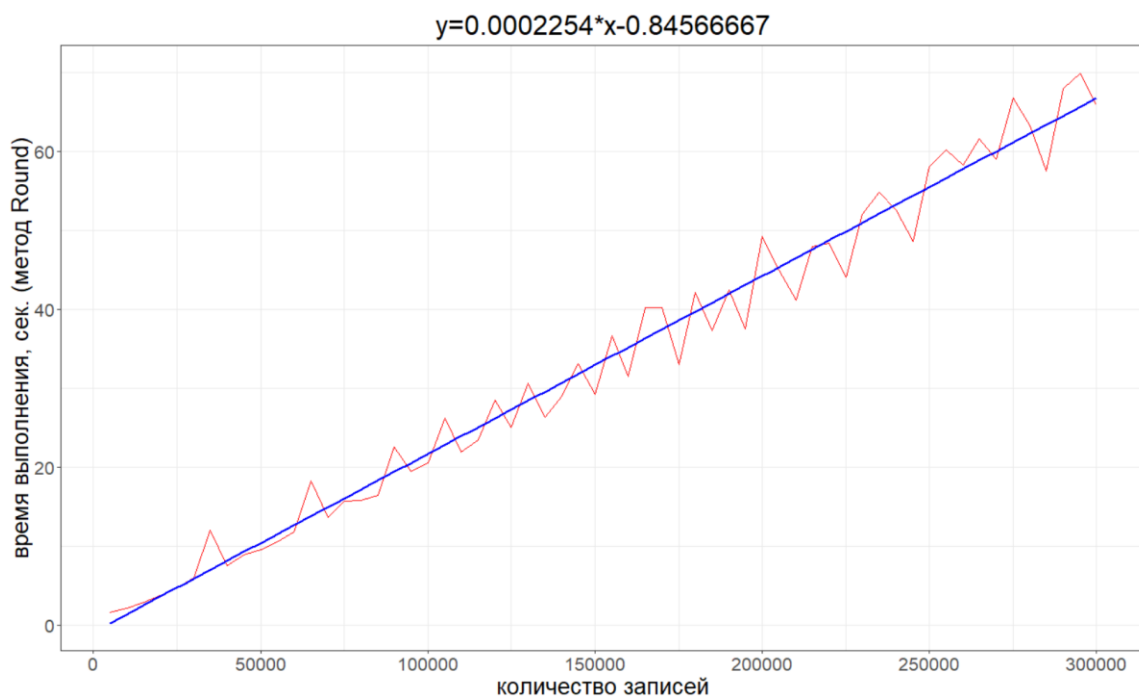


Рисунок 3.21 – Изменение времени обезличивания данных методом Round от размера БД

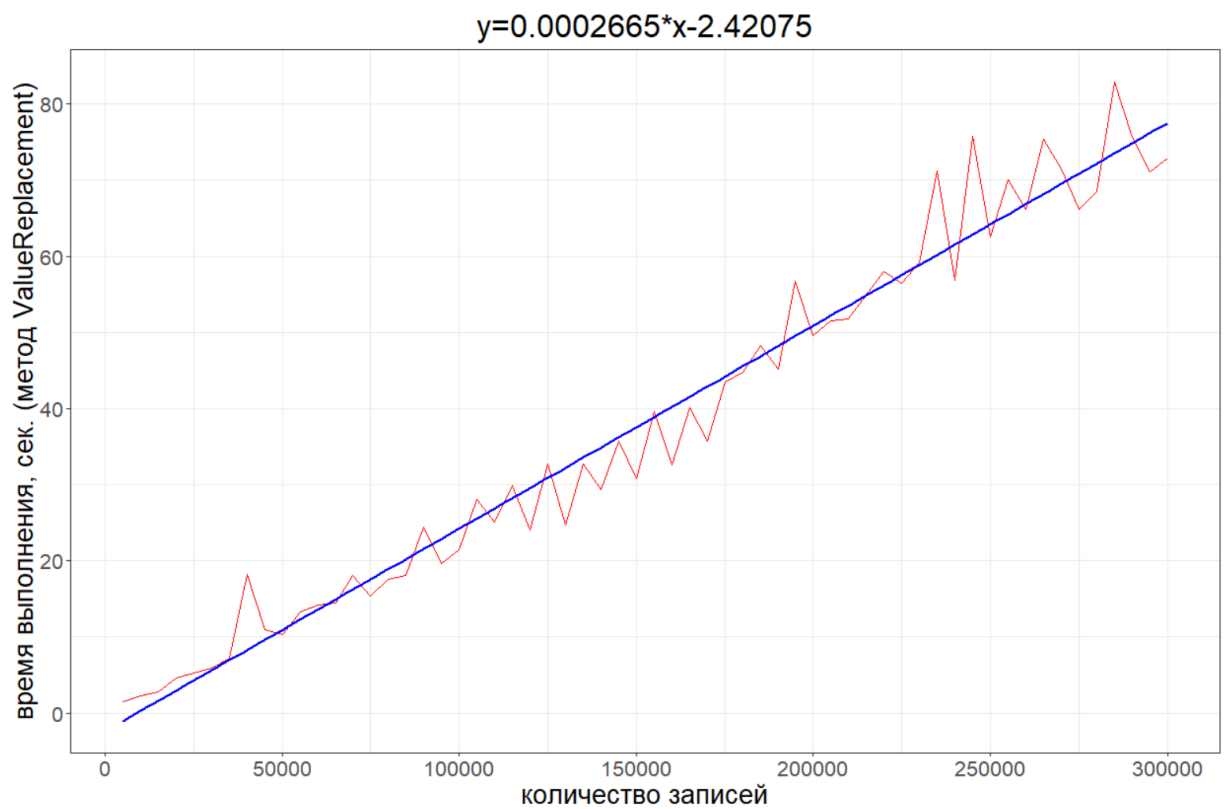


Рисунок 3.22 – Изменение времени обезличивания данных методом ValueReplacement от размера БД

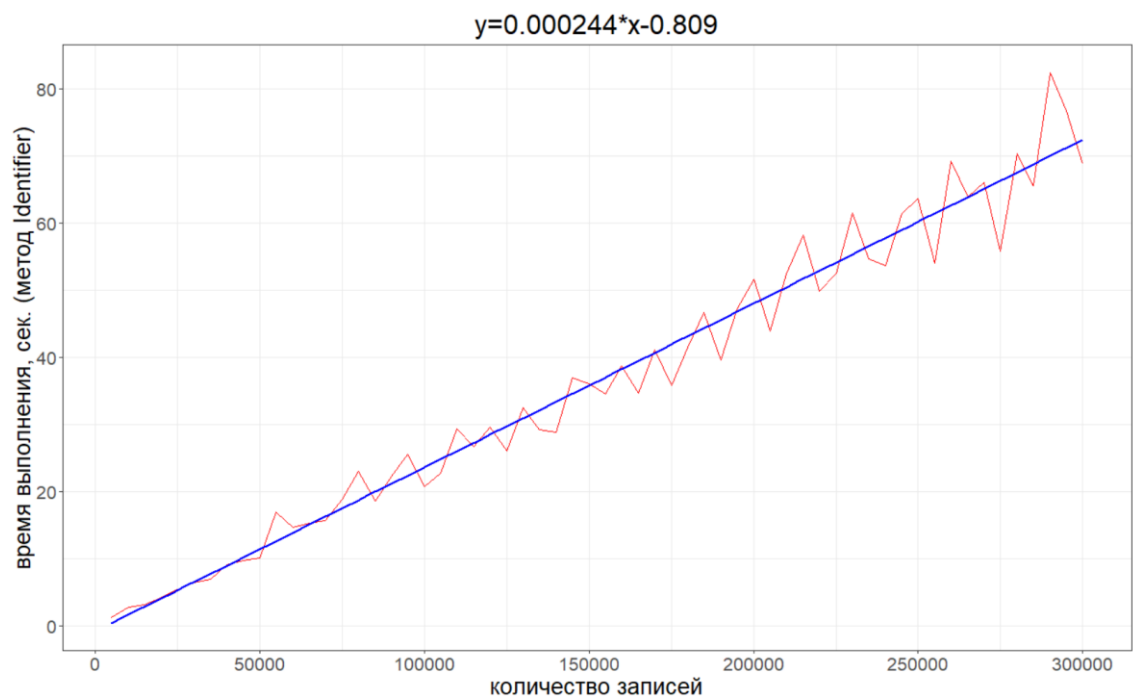


Рисунок 3.23 – Изменение времени обезличивания данных методом Identifier от размера БД

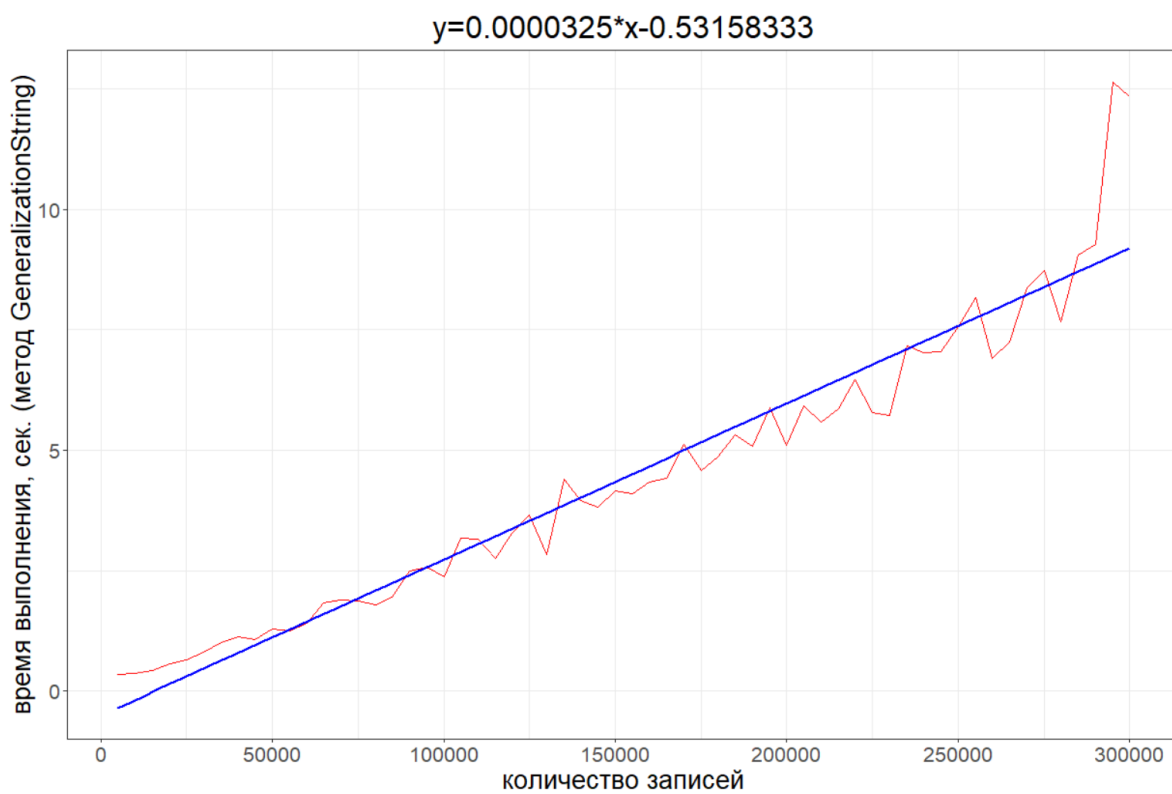


Рисунок 3.24 – Изменение времени обезличивания данных методом *GeneralizationString* от размера БД

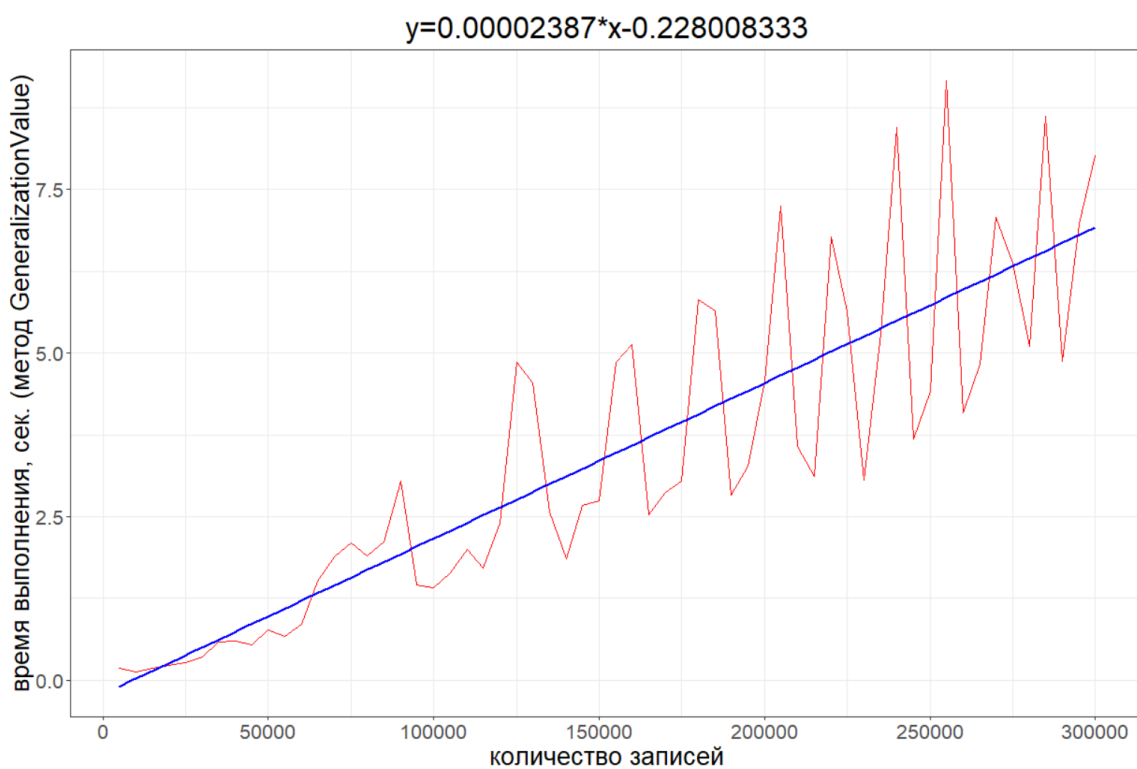


Рисунок 3.25 – Изменение времени обезличивания данных методом *GeneralizationValue* от размера БД

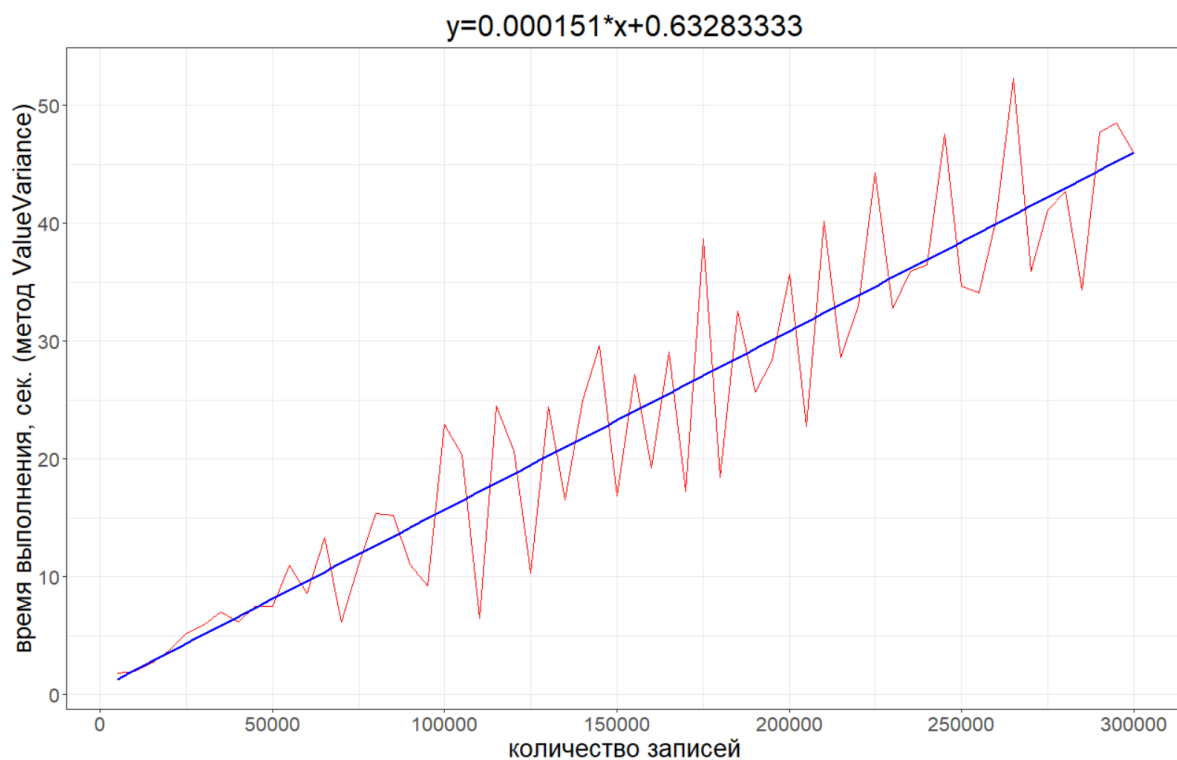


Рисунок 3.26 – Изменение времени обезличивания данных методом ValueVariance от размера БД

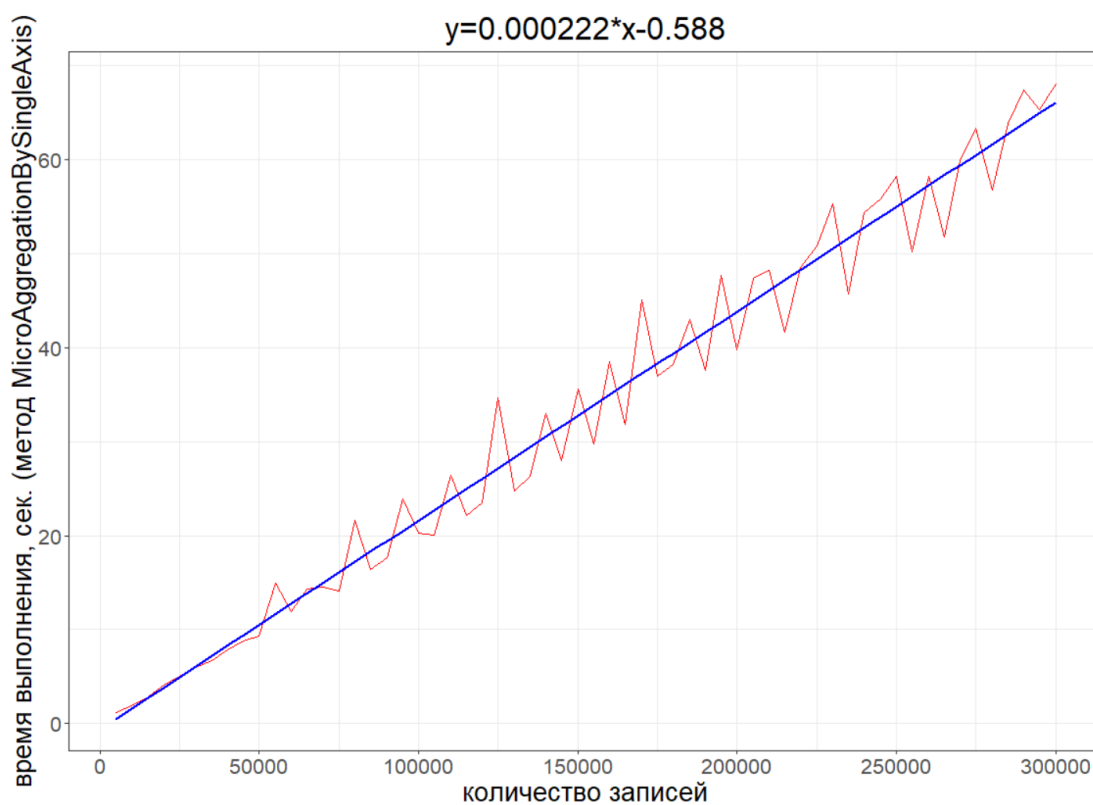


Рисунок 3.27 – Изменение времени обезличивания данных методом MicroAggregationBySingleAxis от размера БД

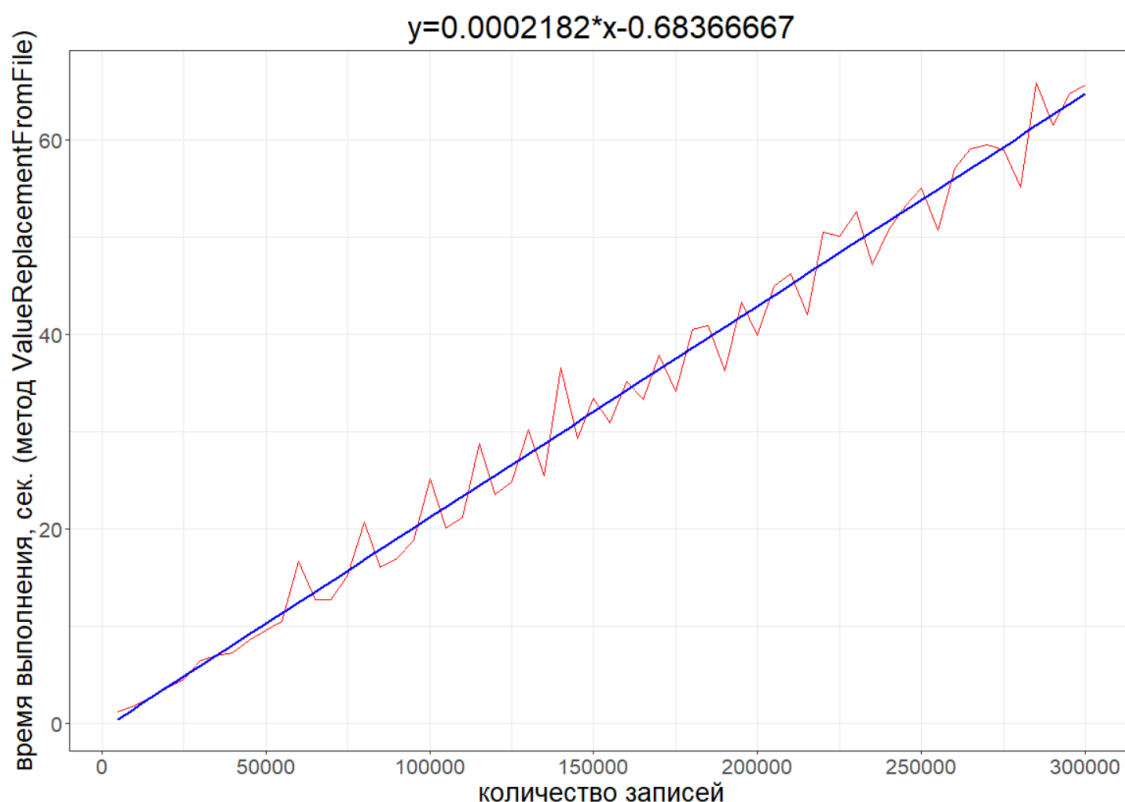


Рисунок 3.28 – Изменение времени обезличивания данных методом *ValueReplacementFromFile* от размера БД

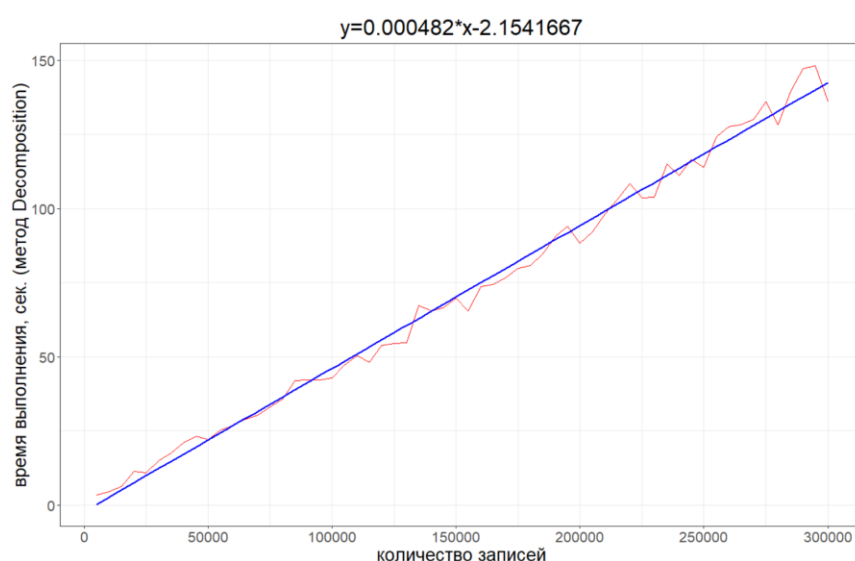


Рисунок 3.29 – Изменение времени обезличивания данных методом *Decomposition* от размера БД

В таблице 1 приведены характеристики точности линейной аппроксимации времени обезличивания БД от объема БД для каждого из рассматриваемых методов обезличивания (характеристики остатков: ME – средняя ошибка, MAE – средняя абсолютная ошибка, MAPE – средняя

абсолютная ошибка в процентах, и D – коэффициент детерминации). Точность аппроксимации зависит от вклада шумовой составляющей, который в свою очередь определяется загрузкой системы и количеством параллельно выполняемых процессов. По результатам эксперимента коэффициент детерминации, определяющий долю вариации времени обезличивания, объясненной моделью, составляет от 0,735 (GeneralizationValue) до 0,999 (Microaggregation), MAPE находится в диапазоне от 7,3% (Pattern) до 33,37% (GeneralizationValue).

Таблица 1 – Характеристики точности моделей

№	Метод обезличивания	Характеристики точности модели			
		ME	MAE	MAPE	D
1.	Microaggregation	0	889.94	30.95	0.999
2.	MicroAggregationBySingleAxis	0	2.57	9.33	0.973
3.	DateAging	0	4.62	27.93	0.82
4.	Pattern	0	1.86	7.3	0.983
5.	Round	0	2.33	9.66	0.978
6.	Shuffle	0	3.4	9.41	0.987
7.	Identifier	0	2.76	9.17	0.969
8.	ValueReplacement	0	3.07	14.35	0.967
9.	GeneralizationString	0	0.47	19.9	0.931
10.	GeneralizationValue	0	0.95	33.37	0.735
11.	ValueVariance	0	3.99	21.51	0.863
12.	ValueReplacementFromFile	0	1.96	8.21	0.983
13.	Decomposition	0	2.93	7.53	0.993

На рисунках 3.30-3.31 приведен график изменения времени обезличивания от количества записей БД для всех рассматриваемых методов, что позволяет сопоставить временные затраты, необходимые для реализации каждого из методов обезличивания. Наибольшее время выполнения процедуры обезличивания данных соответствует методу Shuffle, наименее ресурсоемкий – метод Identifier, методы Replace, Generalization, Pattern имеют близкие временные характеристики.

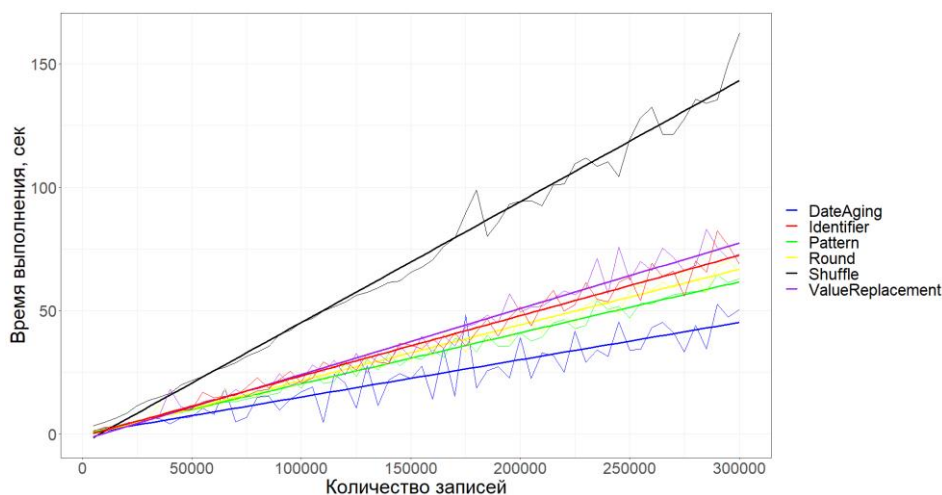


Рисунок 3.30 – Изменение времени обезличивания данных от размера БД при применении разных методов обезличивания

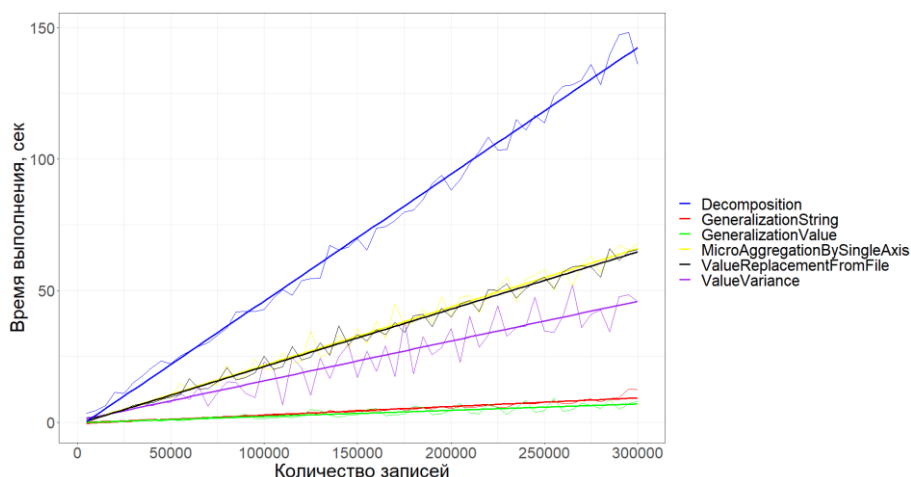


Рисунок 3.31 – Изменение времени обезличивания данных от размера БД при применении разных методов обезличивания

В таблице 2 приведены статистические характеристики скорости обезличивания данных (отношение общего времени обезличивания к объему БД – время на обработку одной записи в миллисекундах) для каждого метода по результатам 60 экспериментов, а именно минимальная, максимальная, средняя скорость и СКО скорости. На рисунках 3.32-3.35 показаны диаграммы размаха, иллюстрирующие изменение статистических характеристик скорости обезличивания данных в зависимости от используемого метода. На диаграмме размаха положение центральной линии определяет оценку медианы, границы прямоугольника соответствуют нижнему и верхнему квартилям, высота прямоугольника – интерквартильный размах (ИКР), нижний (верхний) «ус»

продолжается от нижней (верхней) границы прямоугольника до наименьшего (наибольшего) выборочного значения, находящегося в пределах расстояния $1,5 \times \text{ИКР}$ от этой границы. Значения за пределами «усов» могут считаться аномальными выбросами.

Таблица 2 – Статистические характеристики скорости обезличивания данных в зависимости от метода обезличивания

№	Метод обезличивания	Статистические характеристики			
		Среднее	СКО	Минимум	Максимум
1.	Microaggregation	161.515	100.674	6.3	328.638
2.	MicroAggregationBySingleAxis	0.216	0.024	0.186	0.2772
3.	DateAging	0.153	0.046	0.044	0.284
4.	Pattern	0.208	0.024	0.184	0.327
5.	Round	0.219	0.03	0.187	0.345
6.	Shuffle	0.459	0.043	0.411	0.696
7.	Identifier	0.237	0.025	0.198	0.308
8.	ValueReplacement	0.246	0.039	0.185	0.455
9.	GeneralizationString	0.029	0.006	0.022	0.068
10.	GeneralizationValue	0.021	0.008	0.011	0.039
11.	ValueVariance	0.163	0.047	0.059	0.376
12.	ValueReplacementFromFile	0.211	0.022	0.177	0.277
13.	Decomposition	0.469	0.039	0.419	0.674

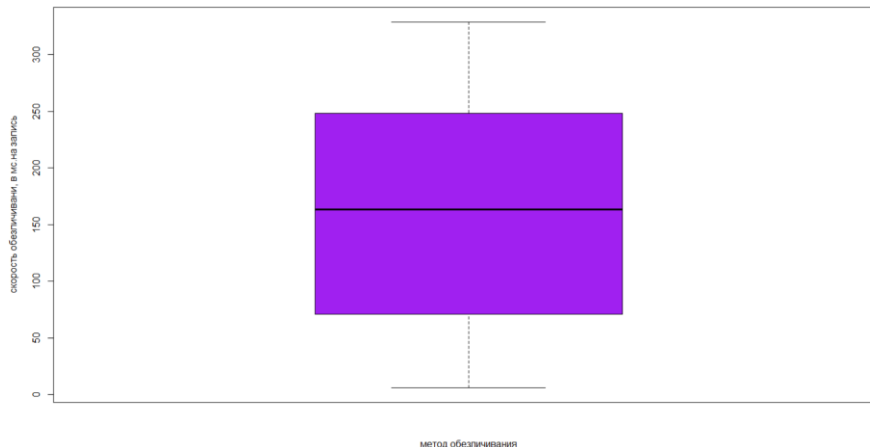


Рисунок 3.32 – Диаграмма статистических характеристик скорости обезличивания данных для метода Microaggregation

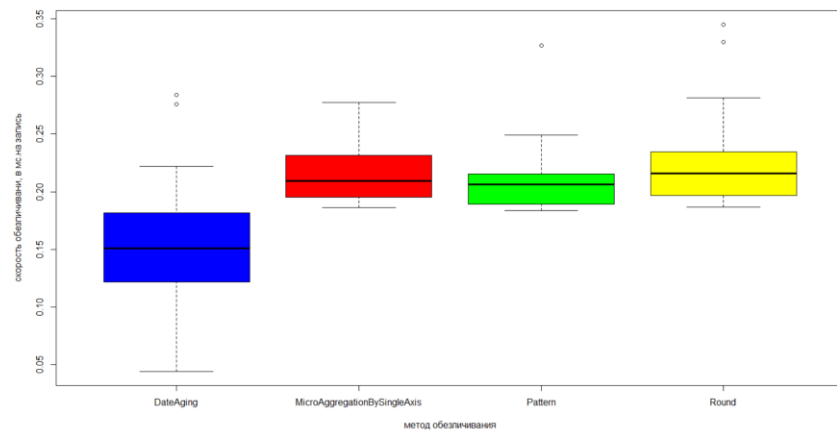


Рисунок 3.33 – Диаграмма статистических характеристик скорости обезличивания данных при применении разных методов обезличивания

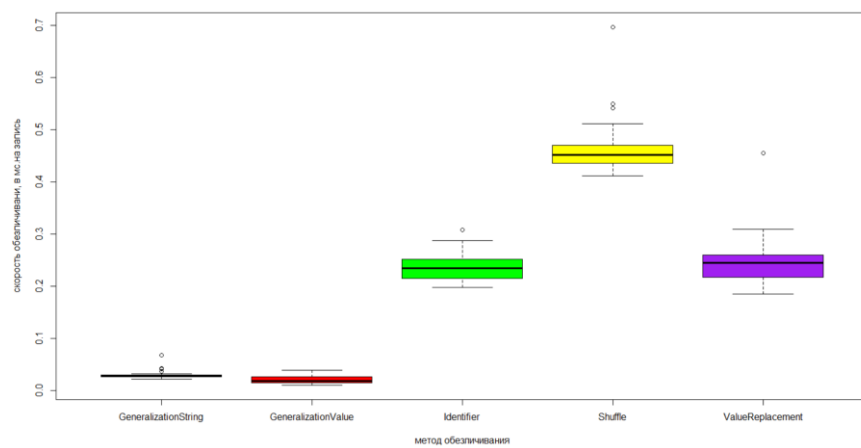


Рисунок 3.34 – Диаграмма статистических характеристик скорости обезличивания данных при применении разных методов обезличивания

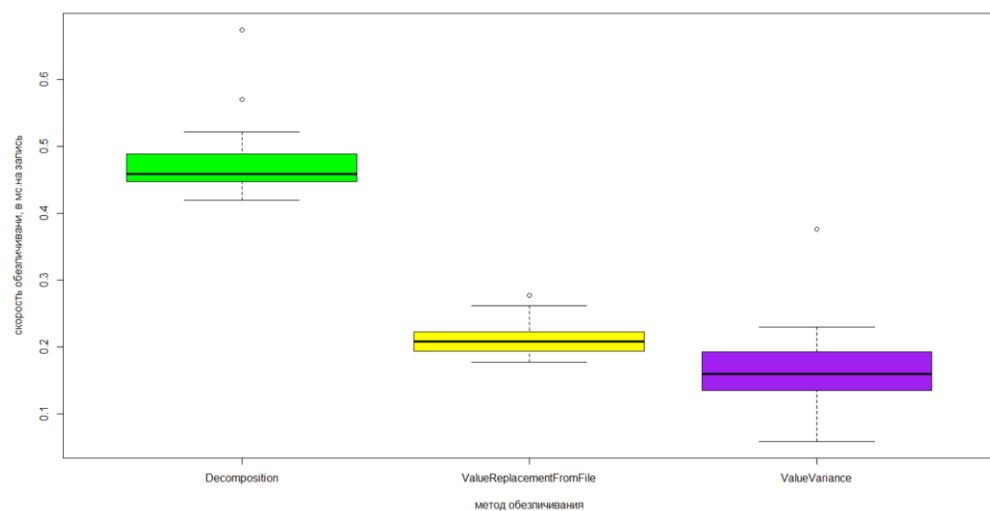


Рисунок 3.35 – Диаграмма статистических характеристик скорости обезличивания данных при применении разных методов обезличивания

Наибольшее среднее время на обезличивания одной записи соответствует методу Microaggregation(163.39±86,935), наименьшее время обезличивания принадлежит методу GeneralizationValue(0.01822±0.02091). Остальные методы обезличивания имеют следующие скорости обезличивания: GeneralizationString(0.02753±0.027925), MicroAggregationBySingleAxis(0.2094±0.21325), DateAging(0.15103±0.15174), Pattern(0.2062±0.20195), Round(0.2158±0.21555), Shuffle(0.4510±0.4529), Identifier(0.2342±0.23345), ValueReplacement(0.2443±0.2391), ValueVariance(0.15972±0.163575), ValueReplacementFromFile(0.2077±0.20845), Decomposition(0.459±0.46725).

3.4 Оценка информационных потерь, связанных с применением метода обезличивания

Далее оценим информационные потери, связанные с применением метода обезличивания. Так как после применения методов обезличивания к данным, не только уменьшается риск раскрытия информации, но и теряется часть полезной статистической информации по сравнению с изначальными данными, поэтому при обезличивании нужно сохранить, как можно больше полезных статистических данных для дальнейшего анализа.

Чтобы оценить, количество потерянной информации после обезличивания, нужно использовать специальные метрики. Если потери информации невелики, это значит, что обезличенные данные всё ещё содержат много полезной информации для анализа, и их статистические характеристики примерно соответствуют исходным данным.

Для оценивания информационных потерь можно использовать следующие метрики: среднеквадратическая ошибка, среднее отклонение и информационная энтропия Шеннона.

Среднеквадратическая ошибка (mean square error) вычисляется по формуле:

$$MSE = \frac{\sum_{j=1}^k \sum_{i=1}^n (x_{ij} - y_{ij})^2}{nk}, \quad (3.1)$$

где k – количество признаков; n – количество записей в БД; x_{ij} – значение j -го признака для i -ой записи в оригинальной БД; y_{ij} – значение j -го признака для i -ой записи в обезличенной БД.

Среднеквадратическая ошибка (root mean square error) вычисляется по формуле:

$$RMSE = \frac{\sum_{j=1}^p \sum_{i=1}^n (x_{ij} - y_{ij})^2}{np}, \quad (3.2)$$

где p – количество признаков; n – количество записей в БД; x_{ij} – значение j -го признака для i -ой записи в оригинальной БД; y_{ij} – значение j -го признака для i -ой записи в обезличенной БД.

Среднее отклонение (mean deviation) вычисляется по формуле:

$$MD = \frac{\sum_{j=1}^k \sum_{i=1}^n \frac{|x_{ij} - y_{ij}|}{\sqrt{2}S_j}}{nk}, \quad (3.3)$$

где S_j – среднеквадратическое отклонение j -го признака в оригинальной БД.

Преимущество данной меры заключается в нормировке и, как следствие, ее независимости от масштаба измерения признаков.

Для качественных и количественных признаков может использоваться информационная энтропия Шеннона, которая отображает меру разнообразия категорий в данных для j -го признака:

$$I_j = \sum_{s=1}^m p_s \log_2 p_s, \quad (3.4)$$

где p_s – вероятность s -ой категории в данных, m – количество категорий.

Информационные потери (в %) по j -ому признаку составят:

$$I_{j(loss)} = \left(1 - \frac{I_{j(depersion)}}{I_{j(initial)}}\right) * 100\%, \quad (3.5)$$

где $I_{j(depersion)}$, $I_{j(initial)}$ – мера Шеннона для j -го признака соответственно в обезличенной и оригинальной базе.

Проведем оценку информационных потерь по приведенным выше формулам (3.1) (3.2) (3.3) (3.5). Возьмем из базы данных по инфекции таблицу с

населением, и проведем обезличивания атрибутов newborn, map, woman при помощи метода микроагрегирования и разным параметром k (таблица 3).

Таблица 3 – Оценка информационных потерь на основе изменения статистических характеристик

№	1	2	3	4	5	6	7	8	9
k	2	4	6	8	10	12	16	18	20
md	0.001	1.46	0.77	1.46	20.92	6.67	4.35	1.04	100
mse	0.03	0.13	0.09	1.03	1.44	0.34	2.48	2.07	9.03
rmse	0.02	0.06	0.05	0.51	0.72	0.17	1.248	1.04	4.62
shannon	18.9	38.35	49.73	61.59	69.92	69.17	80.73	80.55	100

В таблице 3 представлены результаты MD, MSE, RMSE, Shannon указаны в виде разницы в статистических характеристиках между обезличенными и оригинальными данными. Для выбора варианта обезличивания стоит опираться на результаты расчетов MD, MSE, RMSE, Shannon. Исходя из таблицы 3, можно сделать вывод, что обезличивание при помощи микроагрегирования с параметром k равным 6, является наиболее удачным, так как меры MD, MSE, RMSE имеют минимальное искажение, при этом степень разнообразия уменьшилась в два раза по мере Shannon.

3.5 Выводы по главе 3

В рамках третьей главы были рассмотрены и выбраны основные программные средства для реализации модуля обезличивания данных в составе системы интеллектуального управления, обогащения и обезличивания данных. Предложено использовать связку программных средств и технологий: СУБД PostgreSQL и язык программирования Java, что обеспечит высокую скорость, надежность и полную функциональность работы модуля обезличивания данных.

Приведены примеры реализации методов обезличивания данных, относящихся к разным группам методов (введение идентификаторов, изменение состава и семантики, перемешивание) с использованием СУБД PostgreSQL и языка программирования Java. Проведены экспериментальные исследования времени и скорости обезличивания данных в зависимости от

размера исходной БД (количество записей) и выбранного метода обезличивания. Предложена методика проведения экспериментального исследования метода обезличивания данных, которая позволяет провести анализ эффективности применения рассматриваемого метода по критерию реализуемости (скорость работы) и сопоставить разные методы обезличивания или разные алгоритмы реализации одного метода обезличивания между собой. Скорость работы метода обезличивания может выступать важным критерием при выборе метода обезличивания или алгоритма его реализации в реальной конкретной ситуации.

4 РАЗРАБОТКА ПРОГРАММНОЙ СИСТЕМЫ ПО ОБЕЗЛИЧИВАНИЮ ДАННЫХ

4.1 Анализ требований

Разработанное программное обеспечение предназначено для системы «Интеллектуальное управление, обогащение и обезличивание данных», разрабатываемой в рамках платформы «Доверенная среда обмена информации» (проекты Центра компетенций НГТУ). Основная задача программного обеспечения – облегчить настройку и обеспечить обезличивание реляционной базы данных. Помимо этого, программа будет предоставлять оценку информационных потерь и риска раскрытия информации.

Разрабатываемая программная система должна обладать следующим функционалом:

- подготовка данных для обезличивания (выбор типа атрибута и метода заполнения пропусков);
- выбор и настройка методов обезличивания;
- выбор методов оценки рисков;
- выбор методов оценки информационных потерь;
- загрузка и сохранения конфигурации для обезличивания базы данных;
- перенастройка уже сконфигурированных методов обезличивания;
- проведения процесса обезличивания данных;
- отображение времени, за которое было выполнено обезличивание данных;
- вывод метрик оценки информационных потерь и риска раскрытия информации.

Так же программа должна удовлетворять следующим нефункциональным требованиям:

- русская локализация интерфейса;
- наличие графического интерфейса;
- наличие в программе только одного главного окна;

- интуитивно понятный интерфейс.

4.2 Проектирование программного обеспечения

4.2.1 Описание классов

Разработанная программная система представляет собой сложную структуру, основанную на классах графического интерфейса, каждый из которых занимается отображением различных окон приложения, обеспечивая пользователю доступ к необходимой информации и возможность взаимодействия. Эти классы не только отображают информацию, но и управляют пользовательским вводом и реагируют на его действия.

Важной частью этой системы являются классы, которые используют классы графического интерфейса. Например, среди этих классов присутствует класс, ответственный за взаимодействие с базой данных. Он осуществляет получение информации о структуре таблиц и их содержимом.

Еще одним важным аспектом является класс, содержащий методы для подготовки и обезличивания данных. Этот класс заботится о конфигурации методов обезличивания, оценке потенциальных информационных потерь и рисков раскрытия информации.

Для обеспечения процесса обезличивания данных в базе данных был разработан специализированный интерфейс. Классы, реализующие этот интерфейс, предназначены для конкретизации методов обезличивания, обеспечивая гибкость этого процесса в соответствии с требованиями и потребностями конкретного пользователя.

Дополнительный класс, созданный для взаимодействия с базой данных, играет ключевую роль в обеспечении коммуникации между программой и хранилищем данных. Он содержит реализацию всех необходимых методов, обеспечивающих доступ к данным, и обеспечивает надежную и эффективную работу программы с базой данных.

Подробную UML-диаграмму можно увидеть на рисунке 4.1.

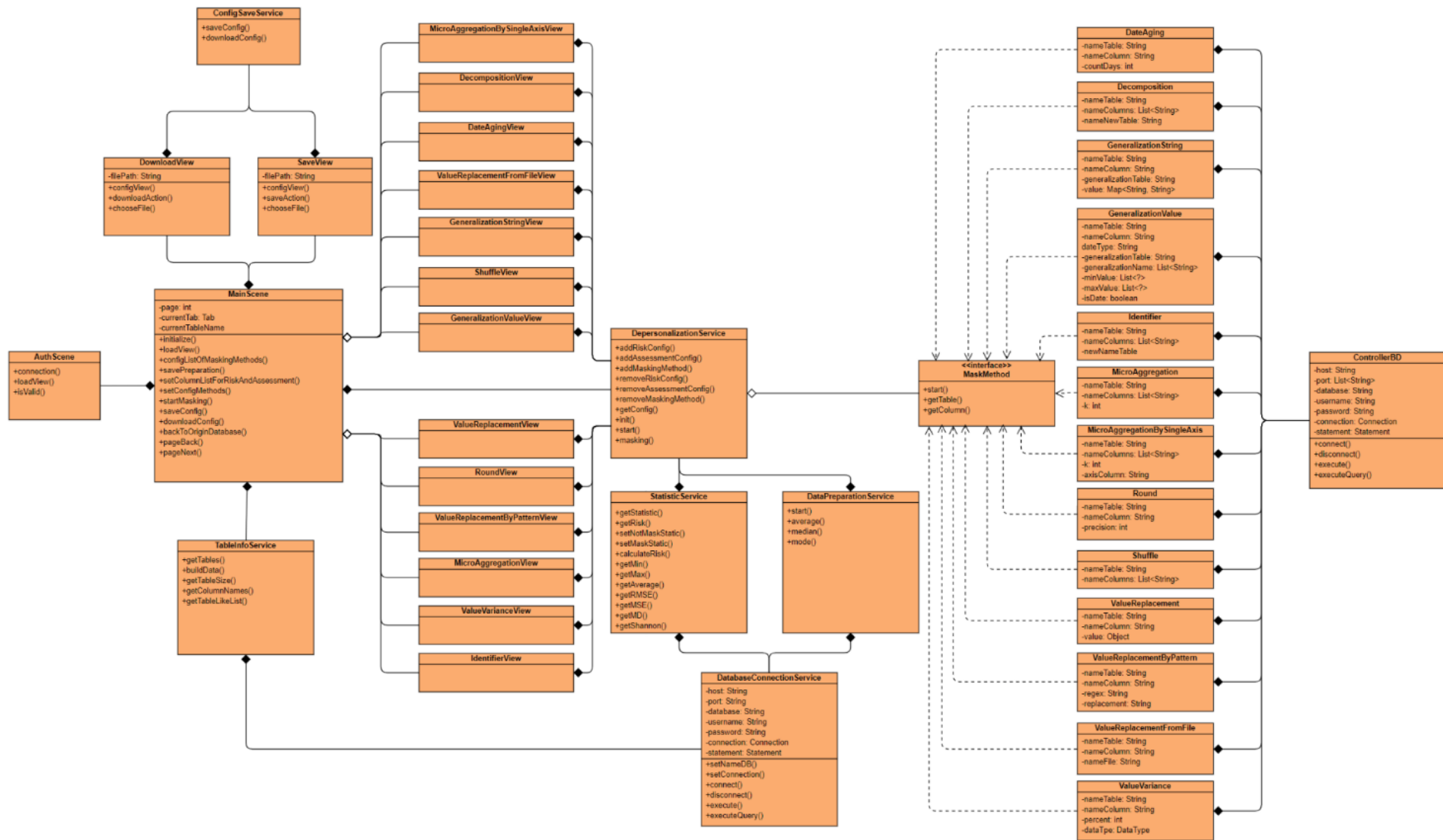


Рисунок 4.1 – UML-диаграмма структуры классов

Таким образом, разработанная программа представляет собой комплексную систему, включающую в себя различные классы и интерфейсы, обеспечивающие удобство использования, защиту данных и эффективное взаимодействие с базой данных.

Подробную информацию о классах можно увидеть в таблице 4.

Таблица 4 – Описание разработанных классов

Имя класс	Описание
ConfigSaveService	Класс необходимый для загрузки и сохранения конфигурационных файлов.
DatabaseConnectionService	Класс необходимый взаимодействия с базой данных.
DataPreparationService	Класс реализующий модуль подготовки данных.
DepersonalizationService	Класс реализующий модуль обезличивания данных.
StatisticService	Класс собирающий статистику, данный класс реализует модуль оценки рисков раскрытия информации и оценку потерь информации.
TableInfoService	Класс преобразующий информацию в удобный формат.
AuthScene	Класс отображающий окно, содержащие поля, необходимые для установки соединения с базой данных.
DownloadView	Класс отображающий окно, загрузки файла с конфигурацией.
SaveView	Класс отображающий окно, сохранения файла с конфигурацией.
MainScene	Класс отображающий главное окно, в данном окне происходит просмотр таблиц и настройка методов обезличивания, оценки потерь, оценки рисков и начальная подготовка данных.
DateAgingView	Класс отображающий окно, необходимое для конфигурации метода старения данных.
DecompositionView	Класс отображающий окно,

	необходимое для конфигурации метода декомпозиции.
GeneralizationStringView	Класс, отображающий окно, необходимое для конфигурации метода обобщения для количественных признаков или дат.
GeneralizationValueView	Класс, отображающий окно, необходимое для конфигурации метода обобщения для качественных признаков.
IdentifierView	Класс, отображающий окно, необходимое для конфигурации метода введения идентификатора.
MicroAggregationView	Класс, отображающий окно, необходимое для конфигурации метода микроагрегирования по выбранным атрибутам.
MicroAggregationBySingleAxisView	Класс, отображающий окно, необходимое для конфигурации метода микроагрегирования по выбранным атрибутам путем сортировки одного из них.
RoundView	Класс, отображающий окно, необходимое для конфигурации метода округления.
ShuffleView	Класс, отображающий окно, необходимое для конфигурации метода перемешивания.
ValueReplacementView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута.
ValueReplacementByPatternView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута согласно заданному образцу (паттерну).
ValueReplacementFromFileView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута на случайное значение из файла.
ValueVarianceView	Класс, отображающий окно, необходимое для конфигурации метода добавления шума.
DateAging	Класс содержащий реализацию

	метода старения данных.
Decomposition	Класс содержащий реализацию метода декомпозиции.
GeneralizationString	Класс содержащий реализацию метода обобщения для количественных признаков или дат.
GeneralizationValue	Класс содержащий реализацию метода обобщения для качественных признаков.
Identifier	Класс содержащий реализацию метода введения идентификатора.
MicroAggregation	Класс содержащий реализацию метода микроагрегирования по выбранным атрибутам.
MicroAggregationBySingleAxis	Класс содержащий реализацию метода микроагрегирования по выбранным атрибутам путем сортировки одного из них.
Round	Класс содержащий реализацию метода округления.
Shuffle	Класс содержащий реализацию метода перемешивания.
ValueReplacement	Класс содержащий реализацию метода замены значений атрибута.
ValueReplacementByPattern	Класс содержащий реализацию метода замены значений атрибута согласно заданному образцу (паттерну).
ValueReplacementFromFile	Класс содержащий реализацию метода замены значений атрибута на случайное значение из файла.
ValueVariance	Класс содержащий реализацию метода добавления шума.
ControllerDB	Внутренний класс, который используется в методах обезличивания, для взаимодействия с базой данных.

4.2.2 Архитектура программного обеспечения

На рисунке 4.2 представлена архитектура программного решения, состоящая из пяти основных компонентов. Первый компонент - пользовательское приложение, разработанное на основе Spring Boot и JavaFX,

отвечает за создание интерфейса пользователя и организацию взаимодействия с данными. Следующие четыре модуля выполняют различные функциональные задачи в рамках программы. Каждый модуль специализируется на определенной области функциональности и взаимодействует с остальными компонентами для обеспечения целостного функционирования программы. Такая архитектура позволяет эффективно структурировать программное решение и обеспечить его гибкость и масштабируемость.

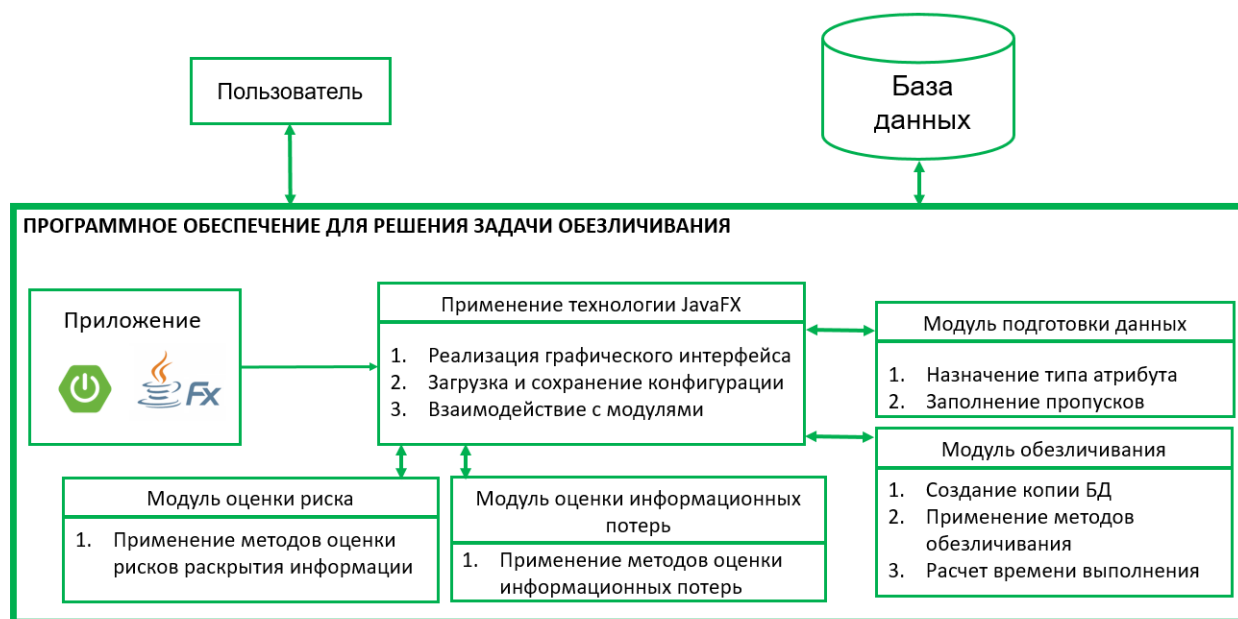


Рисунок 4.2 – Архитектура ПО для решения задачи обезличивания данных

На рисунке 4.3 показана структура программы по обезличиванию данных. Структура состоит из графического интерфейса, модуля загрузки данных, модуля подготовки данных перед проведением процедуры обезличивания, модуля обезличивания, модуля оценки риска раскрытия информации и модуля оценки информационных потерь.



Рисунок 4.3 – Структурная схема программного обеспечения

Графический интерфейс представляет собой важный компонент системы, обеспечивающий наглядное отображение данных в виде таблиц и предоставляющий пользователю доступ к результатам обезличивания.

Модуль "Подготовка данных перед проведением процедуры обезличивания" осуществляет заполнение пропусков в атрибутах, определенных пользователем, что обеспечивает более точные и надежные результаты обезличивания.

Модуль обезличивания играет ключевую роль в процессе создания новой базы данных, которая сохраняет структуру оригинальной базы, но при этом внедряет выбранные методы обезличивания, обеспечивая сохранение конфиденциальности данных. Одновременно ведется мониторинг времени, затраченного на процесс обезличивания, что позволяет оценить эффективность методов.

Модуль оценки риска раскрытия информации проводит анализ вероятности деобезличивания данных по атрибутам, определенным пользователями, что способствует выявлению потенциальных уязвимостей системы.

Модуль оценки информационных потерь осуществляет сопоставление исходной и обезличенной баз данных с целью определения объема утраченной информации, что помогает оценить степень сохранности данных после процедуры обезличивания.

В магистерской диссертации было разработано три модуля: модуль оценки информационных потерь, модуль обезличивания и модуль подготовки данных перед проведением процедуры обезличивания.

4.2.3 Алгоритм решения задачи обезличивания данных

Был разработан программный алгоритм обезличивания данных, который реализован в программной системе (рисунок 4.4).

Основная цель данного алгоритма заключается в обеспечении высокой эффективности процесса обезличивания. Для достижения этой цели алгоритм включает несколько этапов решения задачи, что обеспечивает комплексный подход к решению проблемы.

Этапы алгоритма обезличивания включают в себя ряд ключевых шагов, которые выполняются последовательно.

1. Создание копии оригинальной базы данных: Начальным этапом является создание дубликата исходной базы данных, что обеспечивает сохранность исходных данных в случае необходимости.

2. Анализ статистических данных атрибутов: Проводится расчет и анализ статистических характеристик атрибутов в оригинальной базе данных, которые планируется обезличить или которые требуются для дальнейшего исследования.

3. Предварительная обработка данных: На этом этапе происходит заполнение пропусков в атрибутах и определение типов атрибутов.

4. Применение методов обезличивания: На данном этапе применяются различные методы обезличивания данных согласно установленным целям и требованиям проекта. Это может включать в себя замену значений атрибутов, удаление идентифицирующих признаков или применение декомпозиции.

5. Расчет времени выполнения методов обезличивания: После завершения обезличивания проводится оценка времени выполнения, что позволяет оценить их эффективность и производительность.

6. Оценка рисков раскрытия информации: Применяются методы для оценки вероятности раскрытия информации после обезличивания.

7. Анализ статистических данных обезличенной БД: Проводится анализ статистических данных атрибутов в обезличенной базе данных.

8. Оценка информационных потерь: На основе сравнения статистических данных оригинальной и обезличенной баз данных производится оценка потерь информации.

9. Предоставление результатов пользователю: Наконец, пользователю предоставляются результаты обезличивания для дальнейшего использования.

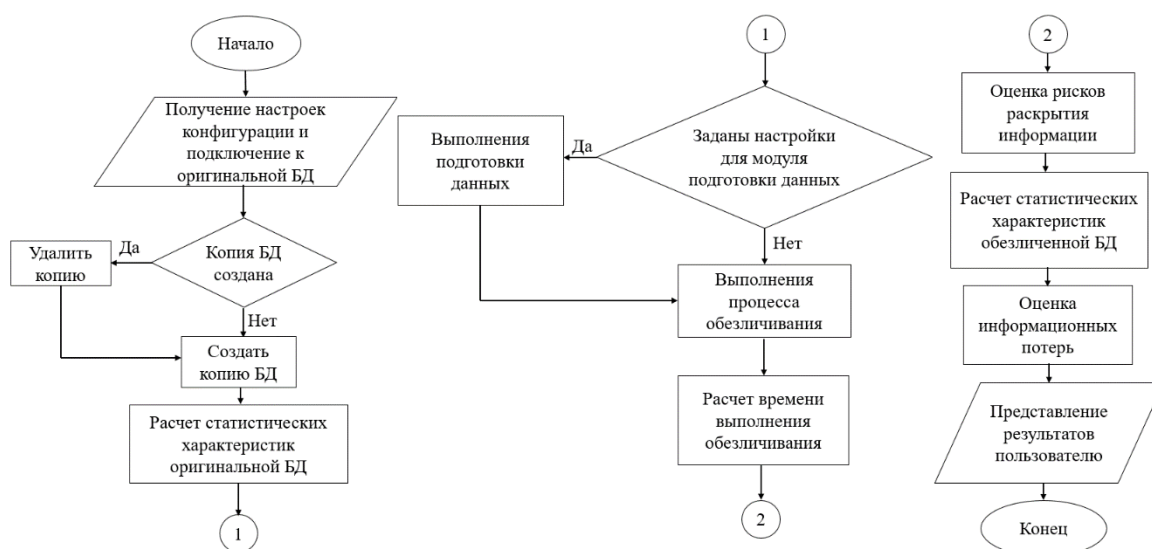


Рисунок 4.4 – Блок-схема алгоритма решения задачи обезличивания данных

4.2.4 Интерфейс программной системы

Графическая часть программы предполагает взаимодействие с пользователем через графический интерфейс в виде оконной формы настольного приложения. Приложение реализовано в виде одного главного окна, и вспомогательных окон, служащих для сохранения и загрузки конфигурации и настройки методов обезличивания.

Из главного меню возможны действия:

- просмотр базы данных;
- переход по 4 вкладкам, которые представляют модули программы (подготовка данных, обезличивания, риск, оценка потерь).
- переход на вкладку с конфигурацией;
- открытие окон, каждое из которых отвечает за конфигурирование своего метода;
- открытие окон для сохранения и загрузки конфигурации.

На рисунке 4.5 представлены все возможные варианты использования программы.

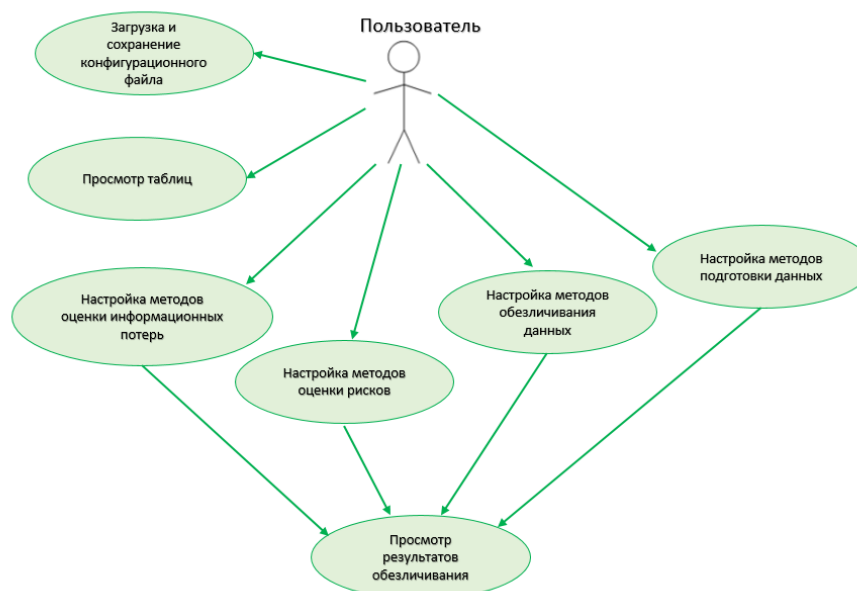


Рисунок 4.5 – Общая схема работы приложения

С учетом функциональных и нефункциональных требований был создан следующий прототип-макет графического интерфейса.

Разработанная программа имеет возможность подключиться к базам данных PostgreSQL (рисунок 4.6).

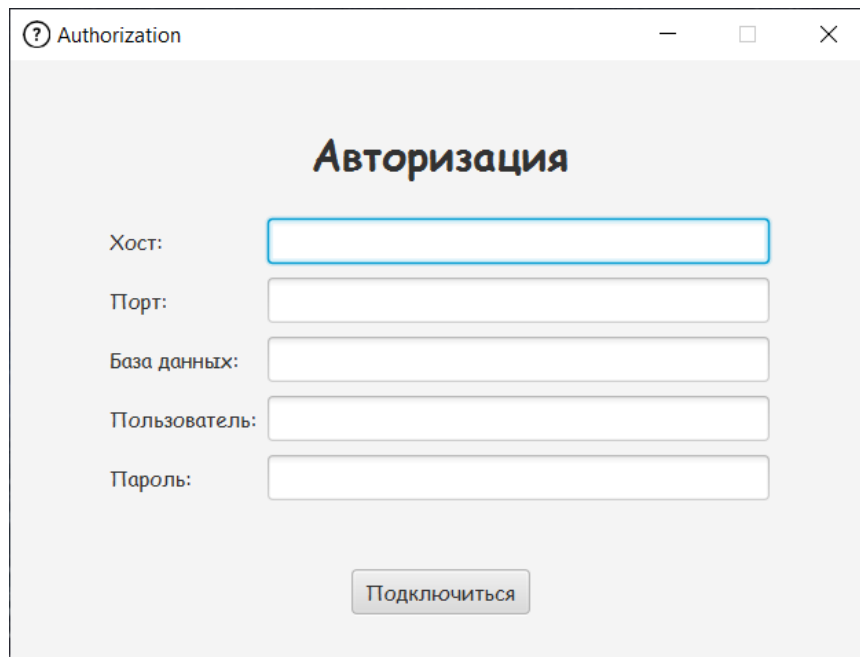


Рисунок 4.6 – Окно для подключения к базе данных

Основное окно программы (рисунок 4.7). Оно содержит следующие блоки:

- панель для отображения данных;
- верхняя панель;
- боковая правая панель содержит 5 вкладок, каждая вкладка, отвечает за один модуль программы, кроме вкладки конфигурации.

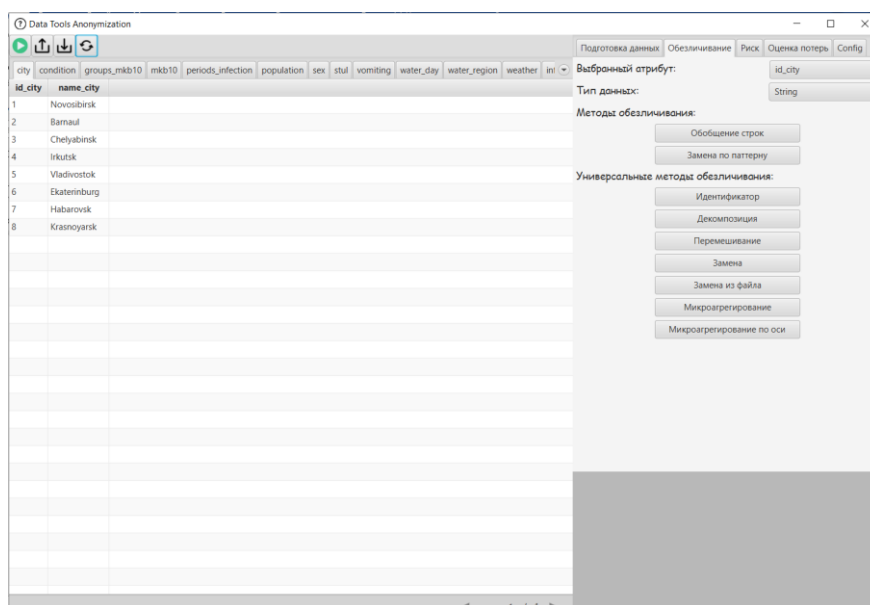


Рисунок 4.7 – Окно графического интерфейса программы

Панель для отображения данных из БД (рисунок 4.8), содержит в верхней части данной вкладки, представляющие собой имена таблиц, которые содержатся в базе. В нижней части справа расположен переключатель по страницам, каждая страница может отображать не более 500 записей. Между верхней и нижней панелью расположена область для вывода содержимого выбранной таблицы.

Рассмотрим подробнее панели:

[illegible]

Рисунок 4.8 – Панель отображения базы данных

В верхней части пользовательского интерфейса размещены четыре кнопки, как показано на рисунке 4.9. Первая кнопка инициирует процесс обезличивания данных. После запуска процесса появляется окно, сообщающее о текущем статусе обезличивания, как показано на рисунке 4.10. По завершении обезличивания в этом окне отображается время, затраченное на процесс, как показано на рисунке 4.11.

Вторая и третья кнопки отвечают соответственно за сохранение и загрузку конфигурации. Эти функции позволяют пользователю сохранить текущие настройки программы или загрузить ранее сохраненную конфигурацию.

Последняя кнопка предназначена для обновления программы. При её нажатии происходит возврат к отображению оригинальной базы данных. Это

важно после обезличивания, так как программа начинает отображать обезличенную базу. Кроме того, эта кнопка служит защитой от случайного повторного обезличивания уже обезличенной базы данных, предотвращая потенциальные ошибки и потерю данных.

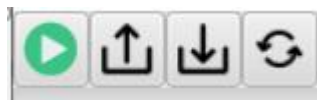


Рисунок 4.9 – Верхняя панель

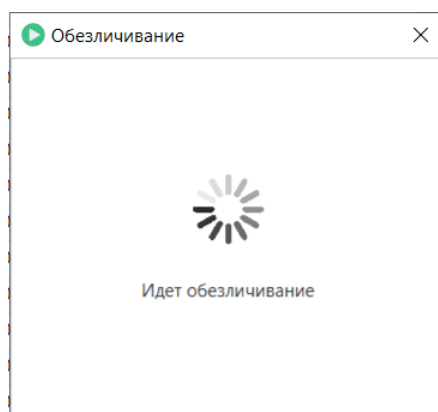


Рисунок 4.10 – Окно, сигнализирующее о процессе обезличивания

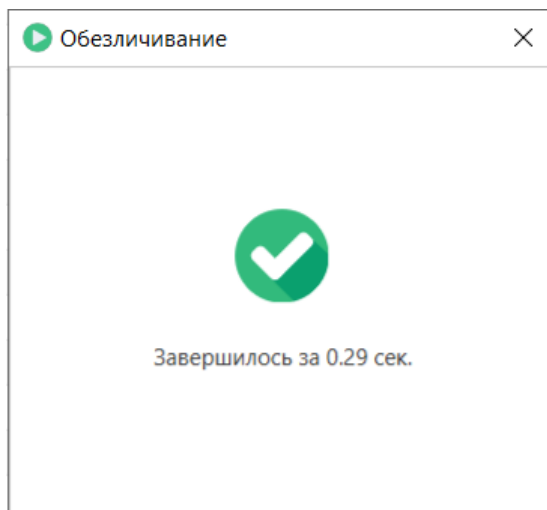


Рисунок 4.11 – Окно, после окончания процесса обезличивания

Последняя панель программы является боковая (рисунок 4.12). Данная панель делится на нижнюю и верхнюю.

Подготовка данных Обезличивание Риск Оценка потерь Config

Выбранный атрибут: id_city

Тип: Insensitive

Метод заполнения поля: none

Статистика по обезличиванию

MSE: 0.9989960689811811
MD: 8.969871474578262
Shannon: 45.734561707076374
ProsecutorMetricB: 0.2
ProsecutorMetricC: 0.19444444444444445

Рисунок 4.12 – Боковая панель вкладки «Подготовка данных»

В нижней части боковой панели представлены статистические данные, относящиеся к обезличенной базе данных. Эти данные включают в себя метрики оценки информационных потерь, которые представляют собой отношения между значениями оригинальной базы данных и обезличенной версии. Пользователь имеет возможность выбирать нужные метрики оценки потерь информации, такие как минимальное, максимальное, среднее значение, среднеквадратичное отклонение (RMSE), среднеквадратическая ошибка (MSE), медианное значение (MD) и индекс Шеннона.

Кроме того, также предоставляется выбор методов оценки рисков, таких как ProsecutorMetricA, ProsecutorMetricB, ProsecutorMetricC и GlobalRisk. Эти методы позволяют пользователю анализировать возможные риски, связанные с обезличиванием данных, и принимать информированные решения о дальнейших шагах в обработке данных.

В верхней части боковой панели расположены 5 вкладок, каждая из которых представляет собой отдельный модуль функциональности: подготовка данных, обезличивание, оценка рисков и потерь, а также конфигурация.

Первая вкладка, представленная в виде модуля подготовки данных, открывает доступ к возможности назначения выбранному атрибуту соответствующего типа данных (например, insensitive, sensitive, quasi-sensitive, identifying). Кроме того, пользователь может выбрать метод заполнения пропущенных значений для указанного атрибута из предложенных вариантов (например, none, average, median, mode). Эти опции обеспечивают пользователю контроль над предварительной обработкой данных перед обезличиванием.

Во второй вкладке располагается модуль обезличивания данных, представленный на рисунке 4.13. Здесь пользователь имеет возможность назначить тип атрибута данных, а также применить один или несколько методов обезличивания.

Подготовка данных Обезличивание Риск Оценка потерь Config

Выбранный атрибут: id_city

Тип данных: String

Методы обезличивания:

Обобщение строк

Замена по паттерну

Универсальные методы обезличивания:

Идентификатор

Декомпозиция

Перемешивание

Замена

Замена из файла

Микроагрегирование

Микроагрегирование по оси

Рисунок 4.13 – Боковая панель вкладки «Обезличивание»

Существует два вида методов обезличивания: универсальные, которые могут быть применены к атрибутам независимо от их типа данных, и специализированные, предназначенные только для определенных типов данных (String, Integer, Float или Date).

К универсальным методам обезличивания относятся:

- Идентификатор
- Декомпозиция

- Перемешивание
- Замена
- Замена из файла
- Микроагрегирование по нескольким признакам
- Микроагрегирование по одному признаку
- Методы обезличивания для строковых атрибутов включают:
- Обобщение строк
- Замена по шаблону

Для целочисленных, вещественных и датových атрибутов доступны методы:

- Обобщение значений дат
- Шум

Для вещественных атрибутов также можно применить метод округления, а для даты - округление и старение даты. Эти методы обеспечивают разнообразные способы обезличивания данных в зависимости от их типа и требований безопасности.

Также для каждого метода обезличивания существует свое собственное окно по настройке метода. Рассмотрим окно настройки на примере метода микроагрегирования (рисунок 4.14).

Настройка метода (Микроагрегирование)

Название:

Таблица:

k:

id_city
years
newborn
p1_6
p0_15
p15_more
p7_15
man

→
←

Сохранить

Рисунок 4.14 – Окно настройки для метода микроагрегирования

На рисунке 4.14 показано окно для настройки метода микроагрегирования, в котором присутствуют поля, требующие заполнения для правильной настройки данного метода. Во всех окнах для настройки методов обезличивания присутствует поле "Название", которое предназначено исключительно для внутреннего использования программы и обозначает имя инструкции, заданное пользователем. Это имя позволит пользователю легко найти данный метод во вкладке конфигурации для последующего редактирования или удаления.

Затем идет текстовое поле "Таблица", которое автоматически заполняется в зависимости от того, какую таблицу пользователь просматривал при открытии окна с настройками метода. Далее следует поле под названием "k", предназначенное для настройки количества групп, которые будет формировать метод микроагрегирования.

Последний блок для заполнения состоит из двух колонок, в которых расположены имена атрибутов. В правой колонке перечислены атрибуты, на которые будет применен метод микроагрегирования, а в левой остаются атрибуты, которые не будут затронуты в процессе обезличивания данным методом. Эти поля позволяют пользователю настроить метод

микроагрегирования с учетом конкретных требований и характеристик его данных.

На вкладках «Риск» и «Оценка потерь» (рисунки 4.15 и 4.16) реализованы модули, похожие по своей функциональности, и отвечающие соответственно за оценку рисков и оценку потерь информации. На этих вкладках пользователю предоставлены методы для проведения оценки рисков и информационных потерь.

The screenshot shows the 'Risk' tab of a software interface. At the top, there are five tabs: 'Подготовка данных', 'Обезличивание', 'Риск', 'Оценка потерь', and 'Config'. The 'Риск' tab is active. Below the tabs, there are several configuration options: 'ProsecutorMetricA' with a value of 1.0, 'ProsecutorMetricB' checked, 'ProsecutorMetricC' checked, and 'GlobalRisk' with a value of 1.0. Below these, there is a list of variables with checkboxes: 'id_city', 'years', 'newborn' (checked), 'p1_6', 'p0_15', 'p15_more', 'p7_15', 'man' (checked), 'woman' (checked), and 'total' (checked).

Рисунок 4.15 – Боковая панель вкладки «Риск»

The screenshot shows the 'Loss Evaluation' tab of a software interface. At the top, there are five tabs: 'Подготовка данных', 'Обезличивание', 'Риск', 'Оценка потерь', and 'Config'. The 'Оценка потерь' tab is active. Below the tabs, there are several configuration options: 'min', 'max', 'avg', 'RMSE', 'MSE' (checked), 'MD' (checked), and 'Shannon' (checked). Below these, there is a list of variables with checkboxes: 'id_city', 'years', 'newborn' (checked), 'p1_6', 'p0_15', 'p15_more', 'p7_15', 'man' (checked), 'woman' (checked), and 'total' (checked).

Рисунок 4.16 – Боковая панель вкладки «Оценка потерь»

Пользователь может выбрать один или несколько методов оценки рисков и информационных потерь из представленного списка. Затем необходимо определить атрибуты, по которым нужно провести оценку рисков и/или потерь информации, из таблицы данных, выбранной на данный момент.

На вкладке «Риск» представлены четыре метода оценки рисков: ProsecutorMetricA, ProsecutorMetricB, ProsecutorMetricC и GlobalRisk. Эти методы помогают пользователю анализировать потенциальные риски, связанные с обезличиванием данных, и принимать соответствующие меры для их уменьшения или управления.

На вкладке «Оценка потерь» представлены метрики оценки потерь информации, такие как минимальное, максимальное, среднее значение, среднеквадратичное отклонение (RMSE), среднеквадратическая ошибка (MSE), медианное значение (MD) и индекс Шеннона. Эти метрики позволяют пользователю оценить степень потери информации в результате обезличивания и принять необходимые меры для её минимизации.

В боковой панели также присутствует вкладка конфигурации (рисунок 4.17), где отображаются имена инструкций, ранее созданных пользователем, в виде списка. Нажатие на имя инструкции открывает окно редактирования ранее созданного метода обезличивания (рисунок 4.18).

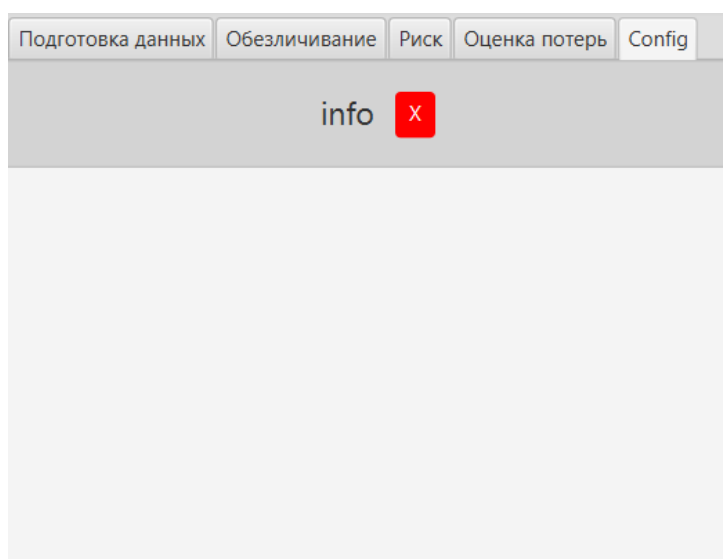


Рисунок 4.17 – Боковая панель вкладки «Конфигурация»

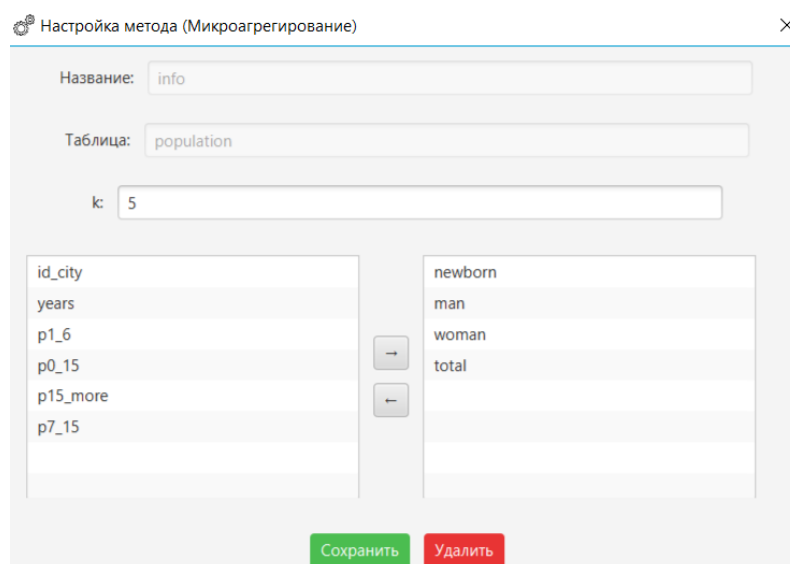


Рисунок 4.18 – Окно настройки для метода микроагрегирования, открытое через вкладку конфигурации

В окне редактирования метода обезличивания пользователь может внести изменения в настройки этого метода. Кроме того, нажав на кнопку удаления в этом окне, можно удалить сконфигурированный метод обезличивания.

Таким образом, вкладка конфигурации обеспечивает удобный доступ к ранее созданным методам обезличивания, а окно редактирования позволяет пользователям легко изменять или удалять настройки методов в соответствии с их потребностями.

4.3 Тестирование программного обеспечения

При запуске исполняемого файла, непосредственно происходит активация окна авторизации, как показано на рисунке 4.19. Затем, проводим конфигурацию для установки связи и осуществляем подключение к базе данных. После успешного установления соединения, появляется главное окно приложения, отображенное на рисунке 4.20.

?

Authorization

—□×

Авторизация

Хост:

localhost

Порт:

5432

База данных:

mad

Пользователь:

postgres

Пароль:

....

Подключиться

Рисунок 4.19 – Окно авторизации

[illegible]

Рисунок 4.20 – Главное окно

Далее, следует процесс обращения к таблице «population» и ее открытие. В разделе подготовки данных производится отбор необходимых атрибутов, при этом каждому из них назначается статус «Sensitive», что показано на рисунке 4.21.

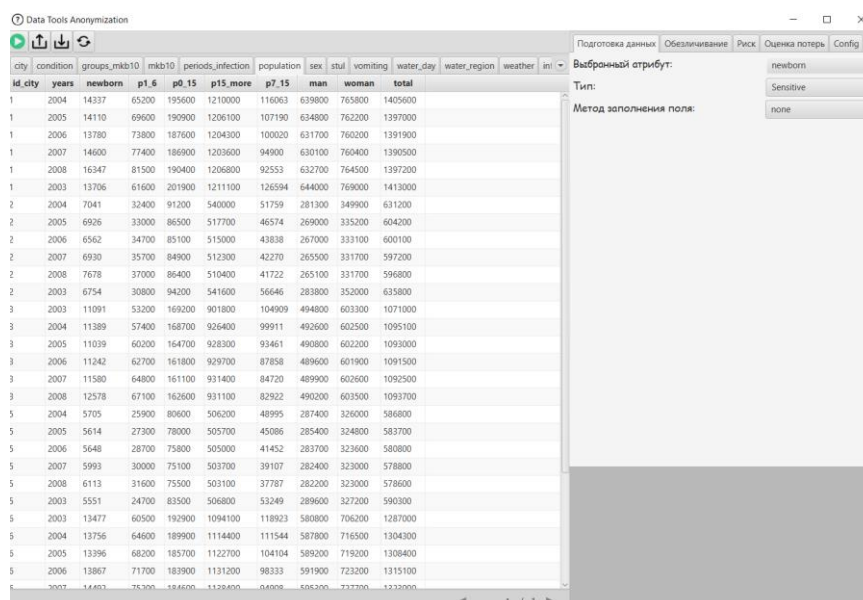


Рисунок 4.21 – Настройка типа атрибута

Продвигаясь далее, переносимся в раздел обезличивания, изображенный на рисунке 4.22, где создаем стратегию применения методов микроагрегации, перемешивания и замены, щелкая по соответствующим кнопкам, как отражено на рисунках 4.23, 4.24 и 4.25 соответственно.

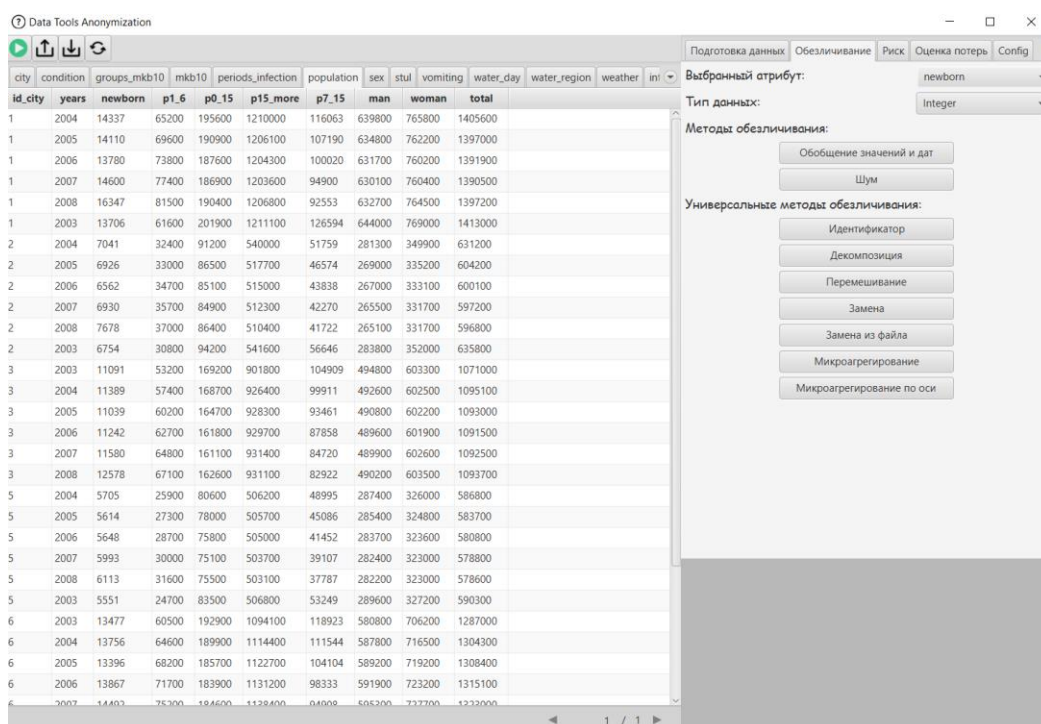


Рисунок 4.22 – Открытие вкладки «Обезличивание»

Настройка метода (Микроагрегирование)
✕

Название:

Таблица:

k:

id_city
years
p1_6
p0_15
p15_more
p7_15

→
←

newborn
man
woman
total

Сохранить

Рисунок 4.23 – Настройка метода микроагрегирования

Настройка метода (Перемешивание)
✕

Название:

Таблица:

id_city
years
newborn
man
woman
total

→
←

p1_6
p0_15
p15_more
p7_15

Сохранить

Рисунок 4.24 – Настройка метода перемешивания

Настройка метода (Замена)

Название: Инструкция для замены

Таблица: city

Атрибут: name_city

Значение для замены: none

Сохранить

Рисунок 4.25 – Настройка метода замены

Завершив настройку методов обезличивания, переключаемся на вкладку «риски» (рисунок 4.26), где выбираем методы ProsecutorMetricB и ProsecutorMetricC вместе с атрибутами newborn, man, woman и total. Кроме того, осуществляем выбор методов MD, MSE, RMSE и Shannon для оценки потерь, описанных также и на рисунке 4.27.

Подготовка данных Обезличивание Риск Оценка потерь Config

☐ ProsecutorMetricA 1.0

☒ ProsecutorMetricB

☒ ProsecutorMetricC

☐ GlobalRisk 1.0

☐ id_city

☐ years

☒ newborn

☐ p1_6

☐ p0_15

☐ p15_more

☐ p7_15

☒ man

☒ woman

☒ total

Рисунок 4.26 – Настройка методов оценки рисков

Подготовка данных	Обезличивание	Риск	Оценка потерь	Config
☐ min				
☐ max				
☐ avg				
☒ RMSE				
☒ MSE				
☒ MD				
☒ Shannon				
☐ id_city				
☐ years				
☒ newborn				
☐ p1_6				
☐ p0_15				
☐ p15_more				
☐ p7_15				
☒ man				
☒ woman				
☒ total				

Рисунок 4.27 – Настройка методов оценки информационных потерь

После завершения данных настроек, производится процесс обезличивания. Длительность этой операции составила 0.4 секунды, что показано на рисунке 4.28. В последующем, проанализировав полученные статистические характеристики, изображенные на рисунке 4.29, мы наблюдаем обезличенные версии таблиц «city» и «population», представленные соответственно на рисунках 4.30 и 4.31.

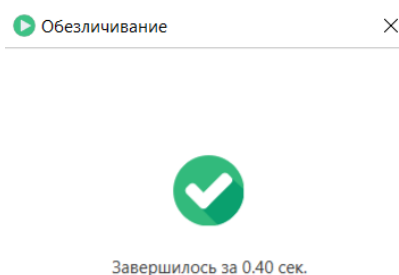


Рисунок 4.28 – Завершение обезличивания

Статистика по обезличиванию
 RMSE: 0.5008411923055511
 MSE: 0.9989960689811811
 MD: 8.96987147457827
 Shannon: 45.734561707076374
 ProsecutorMetricB: 0.2
 ProsecutorMetricC: 0.19444444444444445

Рисунок 4.29 – Получение статистики

id_city	name_city
1	none
2	none
3	none
4	none
5	none
6	none
7	none
8	none

Рисунок 4.30 – Результат работы обезличивания таблица «city»

id_city	years	newborn	p1_6	p0_15	p15_more	p7_15	man	woman	total
1	2004	14456	27300	78000	505700	45086	636600	764340	1400940
1	2005	14456	81500	190400	1206800	92553	636600	764340	1400940
1	2008	14456	31600	75500	503100	37787	636600	764340	1400940
1	2006	14456	62700	161800	929700	87858	636600	764340	1400940
1	2003	14456	60500	192900	1094100	118923	636600	764340	1400940
1	2007	14435	25900	80600	506200	48995	601000	732860	1333860
2	2007	6729	47300	143500	773700	86203	271240	331780	597720
2	2006	6729	69600	190900	1206100	107190	271240	331780	597720
2	2005	6729	35700	84900	512300	42270	271240	331780	597720
2	2004	9732	67100	162600	931100	82922	410183	504067	909733
2	2003	9732	75200	184600	1138400	94908	410183	504067	909733
2	2008	6729	30800	94200	541600	56646	271240	331780	597720
3	2007	9732	24700	83500	506800	53249	410183	504067	909733
3	2005	12448	56400	138500	797900	71417	528440	646180	1174620
3	2008	12448	28700	75800	505000	41452	528440	646180	1174620
3	2004	12448	68200	185700	1122700	104104	528440	646180	1174620
3	2006	9732	37000	86400	510400	41722	410183	504067	909733
3	2003	9732	64600	189900	1114400	111544	410183	504067	909733
5	2006	5815	30000	75100	503700	39107	284220	324080	581740
5	2008	5815	32400	91200	540000	51759	284220	324080	581740
5	2007	5815	64800	161100	931400	84720	284220	324080	581740
5	2003	6729	50000	140300	780600	80262	271240	331780	597720
5	2005	5815	57400	168700	926400	99911	284220	324080	581740
5	2004	5815	53200	169200	901800	104909	284220	324080	581740
6	2007	14435	79000	188200	1144100	93380	601000	732860	1333860
6	2004	12448	33000	86500	517700	46574	528440	646180	1174620
6	2003	12448	34700	85100	515000	43838	528440	646180	1174620
6	2008	14435	60200	164700	928300	93461	601000	732860	1333860
6	2006	14425	52100	128400	788800	75008	601000	732860	1333860

Рисунок 4.31 – Результат работы обезличивания таблица «population»

4.4 Выводы по главе 4

В данной главе было описано программное обеспечение, предназначенное для поддержки процесса обезличивания реляционных баз данных и предоставления оценки информационных потерь и риска раскрытия

информации. Оно должно обладать функциями подготовки данных, выбора методов обезличивания и оценки рисков, а также удовлетворять нефункциональным требованиям, таким как русскоязычный интерфейс и интуитивная навигация.

При разработке архитектуры программы использовался паттерн MVC (модель, вид, контроль). Модель представляет собой классы, отвечающие за методы обезличивания данных. Представление включает классы графического интерфейса, отображающие информацию пользователю и обеспечивающие взаимодействие. Контроллер выступает посредником между моделью и пользовательским интерфейсом, управляя вызовами методов модели и управлением потоком данных в приложении.

Был представлен интерфейс программы и описаны основные окна разработанной программы. А также было проведено тестовое конфигурирование для обезличивания 2 таблиц: «city» и «population».

ЗАКЛЮЧЕНИЕ

В ходе научно-исследовательской работы были достигнуты следующие результаты:

1. Сформулированы цель и задачи магистерского исследования, ориентированные на реализацию и исследование методов обезличивания данных.

2. Сделан обзор современных технологий и программных инструментов, предназначенных для поддержки процесса обезличивания данных.

3. Проведено исследование SQL Server Management Studio, включая анализ его возможностей и ограничений использования, в контексте обезличивания данных.

4. Проанализированы основные классы методов обезличивания, которые представлены в приказе Роскомнадзора от 5 сентября 2013 г. N 996 "Об утверждении требований и методов по обезличиванию персональных данных" и выбраны методы для реализации в ПО.

5. Сделан выбор программных средств и технологий для реализации программной системы обезличивания данных, учитывая требования и особенности задачи.

6. Разработаны алгоритм обезличивания данных на основе применения разных классов методов обезличивания.

7. Исследована скорость работы разных методов обезличивания данных в зависимости от размера исходной БД.

8. Выполнена оценка информационных потерь, связанных с применением метода микроагрегирования, в зависимости от заданного значения параметра метода.

9. Разработана архитектура, функциональное наполнение, пользовательский интерфейс программной системы обезличивания данных.

10. Выполнено тестирование разработанного программного обеспечения.

Актуальными направлениями для дальнейших исследований являются разработка новых методов обезличивания данных, оценки рисков и оценки

информационных потерь, внедрение параллелизма в процесс обезличивания, а также создание подхода, который позволит оценивать риски в зависимости от заданных информационных потерь.

Таким образом, выполненная научно-исследовательская работа не только определяет актуальность и важность проблемы обезличивания данных, но и предлагает конкретные методы и инструменты для её решения, а также оценивает эффективность предлагаемых подходов.

СПИСОК ЛИТЕРАТУРЫ

1. Методические рекомендации по применению приказа Роскомнадзора от 5 сентября 2013 г. N 996 "Об утверждении требований и методов по обезличиванию персональных данных".
2. Мавринская Т. В. Обезличивание персональных данных и технологии «больших данных» (bigdata) / Т. В. Мавринская, А. В. Лошкарёв, Е. Н. Чуракова // Интерактивная наука. – 2017. – С. 78-80. – ISSN 2414-9411. – DOI 10.21661/r-130405.
3. Static vs Dynamic Data Masking. – URL: <https://www.imperva.com/learn/data-security/static-dynamic-data-masking/> (дата обращения: 28.04.2023).
4. Anonymization & Data Masking for PostgreSQL. – URL: <https://postgresql-anonymizer.readthedocs.io/en/stable/> (дата обращения: 28.04.2023)
5. Data Masking in MySQL. – URL: <https://dev.mysql.com/blog-archive/data-masking-in-mysql/> (дата обращения: 28.04.2023).
6. Static Data Masking (SSMS 18.0 Preview). – URL: <https://sqlespresso.com/2018/12/19/static-data-masking-ssms-18-0-preview/> (дата обращения: 28.04.2023).
7. General Data Protection Regulation. – URL: <https://gdpr-info.eu/> (дата обращения: 28.04.2023).
8. Health Insurance Portability and Accountability Act of 1996 (HIPAA). – URL: <https://www.cdc.gov/phlp/publications/topic/hipaa.html> (дата обращения: 28.04.2023).
9. Payment Card Industry Data Security Standard. – URL: <https://www.techtarget.com/searchsecurity/definition/PCI-DSS-Payment-Card-Industry-Data-Security-Standard> (дата обращения: 28.04.2023).
10. Что такое реляционная база данных. – URL: <https://www.oracle.com/cis/database/what-is-a-relational-database/> (дата обращения: 28.04.2023).

11. МАСКИРОВКА ДАННЫХ ПРОТИВ ШИФРОВАНИЯ. – URL: <https://merehead.com/ru/blog/data-masking-vs-encryption/> (дата обращения: 28.04.2023).

12. mu-ARGUS user's manual. – URL: https://www.researchgate.net/publication/304535536_mu-ARGUS_user's_manual_version_25 (дата обращения: 28.04.2023).

13. Micro-aggregation Techniques. – URL: <http://www2.stat-athens.aueb.gr/~jpan/diatrives/Tzavidis/chapter2.pdf> (дата обращения: 28.04.2023).